FILE: simplebank/api/account.go

```
package api

import (
    "errors"
    "net/http"

    "github.com/gin-gonic/gin"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
)

type createAccountRequest struct {
    Currency string `json:"currency" binding:"required,currency"`
}

func (server *Server) createAccount(ctx *gin.Context) {
    var req createAccountRequest
    if err := ctx.ShouldBindJSON(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    authPayload := ctx.MustGet(authorizationPayloadKey).(*token.Payload)
    arg := db.CreateAccountParams{
        Owner:    authPayload.Username,
        Currency: req.Currency,
        Balance:  0,
    }

    account, err := server.store.CreateAccount(ctx, arg)
    if err != nil {
        errCode := db.ErrorCode(err)
        if errCode == db.ForeignKeyViolation || errCode == db.UniqueViolation {
            ctx.JSON(http.StatusForbidden, errorResponse(err))
            return
        }
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    ctx.JSON(http.StatusOK, account)
}

type getAccountRequest struct {
    ID int64 `uri:"id" binding:"required,min=1"`
}

func (server *Server) getAccount(ctx *gin.Context) {
    var req getAccountRequest
    if err := ctx.ShouldBindUri(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    account, err := server.store.GetAccount(ctx, req.ID)
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            ctx.JSON(http.StatusNotFound, errorResponse(err))
            return
        }

        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    authPayload := ctx.MustGet(authorizationPayloadKey).(*token.Payload)
    if account.Owner != authPayload.Username {
        err := errors.New("account doesn't belong to the authenticated user")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    ctx.JSON(http.StatusOK, account)
}

type listAccountRequest struct {
    PageID   int32 `form:"page_id" binding:"required,min=1"`
    PageSize int32 `form:"page_size" binding:"required,min=5,max=10"`
}

func (server *Server) listAccounts(ctx *gin.Context) {
    var req listAccountRequest
    if err := ctx.ShouldBindQuery(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    authPayload := ctx.MustGet(authorizationPayloadKey).(*token.Payload)
    arg := db.ListAccountsParams{
        Owner:  authPayload.Username,
        Limit:  req.PageSize,
        Offset: (req.PageID - 1) * req.PageSize,
    }

    accounts, err := server.store.ListAccounts(ctx, arg)
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    ctx.JSON(http.StatusOK, accounts)
}
```

FILE: simplebank/api/account_test.go

```
package api

import (
    "bytes"
    "database/sql"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang/mock/gomock"
    "github.com/stretchr/testify/require"
    mockdb "github.com/techschool/simplebank/db/mock"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
)

func TestGetAccountAPI(t *testing.T) {
    user, _ := randomUser(t)
    account := randomAccount(user.Username)

    testCases := []struct {
        name          string
        accountID     int64
        setupAuth     func(t *testing.T, request *http.Request, tokenMaker token.Maker)
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(t *testing.T, recoder *httptest.ResponseRecorder)
    }{
        {
            name:      "OK",
            accountID: account.ID,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Eq(account.ID)).
                    Times(1).
                    Return(account, nil)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
                requireBodyMatchAccount(t, recorder.Body, account)
            },
        },
        {
            name:      "UnauthorizedUser",
            accountID: account.ID,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
"unauthorized_user", util.DepositorRole, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Eq(account.ID)).
                    Times(1).
                    Return(account, nil)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name:      "NoAuthorization",
            accountID: account.ID,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name:      "NotFound",
            accountID: account.ID,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },

            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Eq(account.ID)).
                    Times(1).
                    Return(db.Account{}, db.ErrRecordNotFound)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusNotFound, recorder.Code)
            },
        },
        {
            name:      "InternalError",
            accountID: account.ID,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Eq(account.ID)).
```

```
                    Times(1).
                    Return(db.Account{}, sql.ErrConnDone)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name:      "InvalidID",
            accountID: 0,
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetAccount(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            url := fmt.Sprintf("/accounts/%d", tc.accountID)
            request, err := http.NewRequest(http.MethodGet, url, nil)
            require.NoError(t, err)

            tc.setupAuth(t, request, server.tokenMaker)
            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(t, recorder)
        })
    }
}

func TestCreateAccountAPI(t *testing.T) {
    user, _ := randomUser(t)
    account := randomAccount(user.Username)

    testCases := []struct {
        name          string
        body          gin.H
        setupAuth     func(t *testing.T, request *http.Request, tokenMaker token.Maker)
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(recoder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            body: gin.H{
                "currency": account.Currency,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                arg := db.CreateAccountParams{
                    Owner:    account.Owner,
                    Currency: account.Currency,
                    Balance:  0,
                }

                store.EXPECT().
                    CreateAccount(gomock.Any(), gomock.Eq(arg)).
                    Times(1).
                    Return(account, nil)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
                requireBodyMatchAccount(t, recorder.Body, account)
            },
        },
        {
            name: "NoAuthorization",
            body: gin.H{
                "currency": account.Currency,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateAccount(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "InternalError",
            body: gin.H{
                "currency": account.Currency,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateAccount(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.Account{}, sql.ErrConnDone)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name: "InvalidCurrency",
            body: gin.H{
                "currency": "invalid",
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateAccount(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            // Marshal body data to JSON
            data, err := json.Marshal(tc.body)
            require.NoError(t, err)

            url := "/accounts"
            request, err := http.NewRequest(http.MethodPost, url, bytes.NewReader(data))
            require.NoError(t, err)

            tc.setupAuth(t, request, server.tokenMaker)
            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(recorder)
        })
    }
}

func TestListAccountsAPI(t *testing.T) {
    user, _ := randomUser(t)

    n := 5
    accounts := make([]db.Account, n)
    for i := 0; i < n; i++ {
        accounts[i] = randomAccount(user.Username)
    }

    type Query struct {
        pageID   int
        pageSize int
    }

    testCases := []struct {
        name          string
        query         Query
        setupAuth     func(t *testing.T, request *http.Request, tokenMaker token.Maker)
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(recoder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            query: Query{
                pageID:   1,
                pageSize: n,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                arg := db.ListAccountsParams{
                    Owner:  user.Username,
                    Limit:  int32(n),
                    Offset: 0,
                }

                store.EXPECT().
                    ListAccounts(gomock.Any(), gomock.Eq(arg)).
                    Times(1).
                    Return(accounts, nil)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
                requireBodyMatchAccounts(t, recorder.Body, accounts)
            },
```

```
        },
        {
            name: "NoAuthorization",
            query: Query{
                pageID:   1,
                pageSize: n,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    ListAccounts(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "InternalError",
            query: Query{
                pageID:   1,
                pageSize: n,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    ListAccounts(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return([]db.Account{}, sql.ErrConnDone)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name: "InvalidPageID",
            query: Query{
                pageID:   -1,
                pageSize: n,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    ListAccounts(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "InvalidPageSize",
            query: Query{
                pageID:   1,
                pageSize: 100000,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user.Username, user.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    ListAccounts(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            url := "/accounts"
            request, err := http.NewRequest(http.MethodGet, url, nil)
            require.NoError(t, err)

            // Add query parameters to request URL
            q := request.URL.Query()
            q.Add("page_id", fmt.Sprintf("%d", tc.query.pageID))
            q.Add("page_size", fmt.Sprintf("%d", tc.query.pageSize))
            request.URL.RawQuery = q.Encode()

            tc.setupAuth(t, request, server.tokenMaker)
            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(recorder)
        })
    }
}
```

```
func randomAccount(owner string) db.Account {
    return db.Account{
        ID:       util.RandomInt(1, 1000),
        Owner:    owner,
        Balance:  util.RandomMoney(),
        Currency: util.RandomCurrency(),
    }
}

func requireBodyMatchAccount(t *testing.T, body *bytes.Buffer, account db.Account) {
    data, err := io.ReadAll(body)
    require.NoError(t, err)

    var gotAccount db.Account
    err = json.Unmarshal(data, &gotAccount)
    require.NoError(t, err)
    require.Equal(t, account, gotAccount)
}

func requireBodyMatchAccounts(t *testing.T, body *bytes.Buffer, accounts []db.Account) {
    data, err := io.ReadAll(body)
    require.NoError(t, err)

    var gotAccounts []db.Account
    err = json.Unmarshal(data, &gotAccounts)
    require.NoError(t, err)
    require.Equal(t, accounts, gotAccounts)
}
```

FILE: simplebank/api/main_test.go

```
package api

import (
    "os"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/stretchr/testify/require"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/util"
)

func newTestServer(t *testing.T, store db.Store) *Server {
    config := util.Config{
        TokenSymmetricKey:   util.RandomString(32),
        AccessTokenDuration: time.Minute,
    }

    server, err := NewServer(config, store)
    require.NoError(t, err)

    return server
}

func TestMain(m *testing.M) {
    gin.SetMode(gin.TestMode)

    os.Exit(m.Run())
}
```

FILE: simplebank/api/middleware.go

```
package api

import (
    "errors"
    "fmt"
    "net/http"
    "strings"

    "github.com/gin-gonic/gin"
    "github.com/techschool/simplebank/token"
)

const (
    authorizationHeaderKey  = "authorization"
    authorizationTypeBearer = "bearer"
    authorizationPayloadKey = "authorization_payload"
)

// AuthMiddleware creates a gin middleware for authorization
func authMiddleware(tokenMaker token.Maker) gin.HandlerFunc {
    return func(ctx *gin.Context) {
        authorizationHeader := ctx.GetHeader(authorizationHeaderKey)

        if len(authorizationHeader) == 0 {
            err := errors.New("authorization header is not provided")
            ctx.AbortWithStatusJSON(http.StatusUnauthorized, errorResponse(err))
            return
        }

        fields := strings.Fields(authorizationHeader)
        if len(fields) < 2 {
            err := errors.New("invalid authorization header format")
            ctx.AbortWithStatusJSON(http.StatusUnauthorized, errorResponse(err))
            return
        }

        authorizationType := strings.ToLower(fields[0])
        if authorizationType != authorizationTypeBearer {
            err := fmt.Errorf("unsupported authorization type %s", authorizationType)
            ctx.AbortWithStatusJSON(http.StatusUnauthorized, errorResponse(err))
            return
        }

        accessToken := fields[1]
        payload, err := tokenMaker.VerifyToken(accessToken, token.TokenTypeAccessToken)
        if err != nil {
```

```
            ctx.AbortWithStatusJSON(http.StatusUnauthorized, errorResponse(err))
            return
        }

        ctx.Set(authorizationPayloadKey, payload)
        ctx.Next()
    }
}
```

FILE: simplebank/api/middleware_test.go

```
package api

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
)

func addAuthorization(
    t *testing.T,
    request *http.Request,
    tokenMaker token.Maker,
    authorizationType string,
    username string,
    role string,
    duration time.Duration,
) {
    token, payload, err := tokenMaker.CreateToken(username, role, duration,
token.TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, payload)

    authorizationHeader := fmt.Sprintf("%s %s", authorizationType, token)
    request.Header.Set(authorizationHeaderKey, authorizationHeader)
}

func TestAuthMiddleware(t *testing.T) {
    username := util.RandomOwner()
    role := util.DepositorRole

    testCases := []struct {
        name          string
        setupAuth     func(t *testing.T, request *http.Request, tokenMaker token.Maker)
        checkResponse func(t *testing.T, recorder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer, username,
role, time.Minute)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
            },
        },
        {
            name: "NoAuthorization",
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "UnsupportedAuthorization",
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, "unsupported", username, role,
time.Minute)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "InvalidAuthorizationFormat",
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, "", username, role, time.Minute)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "ExpiredToken",
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer, username,
role, -time.Minute)
            },
            checkResponse: func(t *testing.T, recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            server := newTestServer(t, nil)
            authPath := "/auth"
            server.router.GET(
                authPath,
                authMiddleware(server.tokenMaker),
                func(ctx *gin.Context) {
                    ctx.JSON(http.StatusOK, gin.H{})
                },
            )

            recorder := httptest.NewRecorder()
            request, err := http.NewRequest(http.MethodGet, authPath, nil)
            require.NoError(t, err)

            tc.setupAuth(t, request, server.tokenMaker)
            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(t, recorder)
        })
    }
}
```

FILE: simplebank/api/server.go

```
package api

import (
    "fmt"

    "github.com/gin-gonic/gin"
    "github.com/gin-gonic/gin/binding"
    "github.com/go-playground/validator/v10"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
)

// Server serves HTTP requests for our banking service.
type Server struct {
    config     util.Config
    store      db.Store
    tokenMaker token.Maker
    router     *gin.Engine
}

// NewServer creates a new HTTP server and set up routing.
func NewServer(config util.Config, store db.Store) (*Server, error) {
    tokenMaker, err := token.NewPasetoMaker(config.TokenSymmetricKey)
    if err != nil {
        return nil, fmt.Errorf("cannot create token maker: %w", err)
    }

    server := &Server{
        config:     config,
        store:      store,
        tokenMaker: tokenMaker,
    }

    if v, ok := binding.Validator.Engine().(*validator.Validate); ok {
        v.RegisterValidation("currency", validCurrency)
    }

    server.setupRouter()
    return server, nil
}

func (server *Server) setupRouter() {
    router := gin.Default()

    router.POST("/users", server.createUser)
    router.POST("/users/login", server.loginUser)
    router.POST("/tokens/renew_access", server.renewAccessToken)

    authRoutes := router.Group("/").Use(authMiddleware(server.tokenMaker))
    authRoutes.POST("/accounts", server.createAccount)
    authRoutes.GET("/accounts/:id", server.getAccount)
    authRoutes.GET("/accounts", server.listAccounts)

    authRoutes.POST("/transfers", server.createTransfer)

    server.router = router
}

// Start runs the HTTP server on a specific address.
func (server *Server) Start(address string) error {
    return server.router.Run(address)
}

func errorResponse(err error) gin.H {
    return gin.H{"error": err.Error()}
}
```

FILE: simplebank/api/token.go

```
package api

import (
    "errors"
    "fmt"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
)

type renewAccessTokenRequest struct {
    RefreshToken string `json:"refresh_token" binding:"required"`
}

type renewAccessTokenResponse struct {
    AccessToken          string    `json:"access_token"`
    AccessTokenExpiresAt time.Time `json:"access_token_expires_at"`
```

```
}

func (server *Server) renewAccessToken(ctx *gin.Context) {
    var req renewAccessTokenRequest
    if err := ctx.ShouldBindJSON(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    refreshPayload, err := server.tokenMaker.VerifyToken(req.RefreshToken,
token.TokenTypeRefreshToken)
    if err != nil {
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    session, err := server.store.GetSession(ctx, refreshPayload.ID)
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            ctx.JSON(http.StatusNotFound, errorResponse(err))
            return
        }
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    if session.IsBlocked {
        err := fmt.Errorf("blocked session")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    if session.Username != refreshPayload.Username {
        err := fmt.Errorf("incorrect session user")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    if session.RefreshToken != req.RefreshToken {
        err := fmt.Errorf("mismatched session token")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    if time.Now().After(session.ExpiresAt) {
        err := fmt.Errorf("expired session")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    accessToken, accessPayload, err := server.tokenMaker.CreateToken(
        refreshPayload.Username,
        refreshPayload.Role,
        server.config.AccessTokenDuration,
        token.TokenTypeAccessToken,
    )
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    rsp := renewAccessTokenResponse{
        AccessToken:          accessToken,
        AccessTokenExpiresAt: accessPayload.ExpiredAt,
    }
    ctx.JSON(http.StatusOK, rsp)
}
```

FILE: simplebank/api/transfer.go

```
package api

import (
    "errors"
    "fmt"
    "net/http"

    "github.com/gin-gonic/gin"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
)

type transferRequest struct {
    FromAccountID int64  `json:"from_account_id" binding:"required,min=1"`
    ToAccountID   int64  `json:"to_account_id" binding:"required,min=1"`
    Amount        int64  `json:"amount" binding:"required,gt=0"`
    Currency      string `json:"currency" binding:"required,currency"`
}

func (server *Server) createTransfer(ctx *gin.Context) {
    var req transferRequest
    if err := ctx.ShouldBindJSON(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    fromAccount, valid := server.validAccount(ctx, req.FromAccountID, req.Currency)
    if !valid {
        return
    }

    authPayload := ctx.MustGet(authorizationPayloadKey).(*token.Payload)
    if fromAccount.Owner != authPayload.Username {
        err := errors.New("from account doesn't belong to the authenticated user")
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    _, valid = server.validAccount(ctx, req.ToAccountID, req.Currency)
    if !valid {
        return
    }

    arg := db.TransferTxParams{
        FromAccountID: req.FromAccountID,
        ToAccountID:   req.ToAccountID,
        Amount:        req.Amount,
    }

    result, err := server.store.TransferTx(ctx, arg)
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    ctx.JSON(http.StatusOK, result)
}

func (server *Server) validAccount(ctx *gin.Context, accountID int64, currency string)
(db.Account, bool) {
    account, err := server.store.GetAccount(ctx, accountID)
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            ctx.JSON(http.StatusNotFound, errorResponse(err))
            return account, false
        }

        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return account, false
    }

    if account.Currency != currency {
        err := fmt.Errorf("account [%d] currency mismatch: %s vs %s", account.ID,
account.Currency, currency)
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return account, false
    }

    return account, true
}
```

FILE: simplebank/api/transfer_test.go

```
package api

import (
    "bytes"
    "database/sql"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang/mock/gomock"
    "github.com/stretchr/testify/require"
    mockdb "github.com/techschool/simplebank/db/mock"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
)

func TestTransferAPI(t *testing.T) {
    amount := int64(10)

    user1, _ := randomUser(t)
    user2, _ := randomUser(t)
    user3, _ := randomUser(t)

    account1 := randomAccount(user1.Username)
    account2 := randomAccount(user2.Username)
    account3 := randomAccount(user3.Username)

    account1.Currency = util.USD
    account2.Currency = util.USD
    account3.Currency = util.EUR

    testCases := []struct {
        name          string
        body          gin.H
        setupAuth     func(t *testing.T, request *http.Request, tokenMaker token.Maker)
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(recoder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(account1, nil)
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account2.ID)).Times(1).Return(account2, nil)

                arg := db.TransferTxParams{
                    FromAccountID: account1.ID,
                    ToAccountID:   account2.ID,
                    Amount:        amount,
                }
```

```
                store.EXPECT().TransferTx(gomock.Any(), gomock.Eq(arg)).Times(1)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
            },
        },
        {
            name: "UnauthorizedUser",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user2.Username, user2.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(account1, nil)
                store.EXPECT().GetAccount(gomock.Any(), gomock.Eq(account2.ID)).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "NoAuthorization",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(), gomock.Any()).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "FromAccountNotFound",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(db.Account{}, db.ErrRecordNotFound)
                store.EXPECT().GetAccount(gomock.Any(), gomock.Eq(account2.ID)).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusNotFound, recorder.Code)
            },
        },
        {
            name: "ToAccountNotFound",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(account1, nil)
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account2.ID)).Times(1).Return(db.Account{}, db.ErrRecordNotFound)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusNotFound, recorder.Code)
            },
        },
        {
            name: "FromAccountCurrencyMismatch",
            body: gin.H{
                "from_account_id": account3.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user3.Username, user3.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account3.ID)).Times(1).Return(account3, nil)
                store.EXPECT().GetAccount(gomock.Any(), gomock.Eq(account2.ID)).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "ToAccountCurrencyMismatch",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account3.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(account1, nil)
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account3.ID)).Times(1).Return(account3, nil)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "InvalidCurrency",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        "XYZ",
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(), gomock.Any()).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "NegativeAmount",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          -amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(), gomock.Any()).Times(0)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "GetAccountError",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Any()).Times(1).Return(db.Account{}, sql.ErrConnDone)
                store.EXPECT().TransferTx(gomock.Any(), gomock.Any()).Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name: "TransferTxError",
            body: gin.H{
                "from_account_id": account1.ID,
                "to_account_id":   account2.ID,
                "amount":          amount,
                "currency":        util.USD,
            },
            setupAuth: func(t *testing.T, request *http.Request, tokenMaker token.Maker) {
                addAuthorization(t, request, tokenMaker, authorizationTypeBearer,
user1.Username, user1.Role, time.Minute)
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account1.ID)).Times(1).Return(account1, nil)
                store.EXPECT().GetAccount(gomock.Any(),
gomock.Eq(account2.ID)).Times(1).Return(account2, nil)
                store.EXPECT().TransferTx(gomock.Any(),
gomock.Any()).Times(1).Return(db.TransferTxResult{}, sql.ErrTxDone)
```

```
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            // Marshal body data to JSON
            data, err := json.Marshal(tc.body)
            require.NoError(t, err)

            url := "/transfers"
            request, err := http.NewRequest(http.MethodPost, url, bytes.NewReader(data))
            require.NoError(t, err)

            tc.setupAuth(t, request, server.tokenMaker)
            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(recorder)
        })
    }
}
```

FILE: simplebank/api/user.go

```
package api

import (
    "errors"
    "net/http"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/google/uuid"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
)

type createUserRequest struct {
    Username string `json:"username" binding:"required,alphanum"`
    Password string `json:"password" binding:"required,min=6"`
    FullName string `json:"full_name" binding:"required"`
    Email    string `json:"email" binding:"required,email"`
}

type userResponse struct {
    Username          string    `json:"username"`
    FullName          string    `json:"full_name"`
    Email             string    `json:"email"`
    PasswordChangedAt time.Time `json:"password_changed_at"`
    CreatedAt         time.Time `json:"created_at"`
}

func newUserResponse(user db.User) userResponse {
    return userResponse{
        Username:          user.Username,
        FullName:          user.FullName,
        Email:             user.Email,
        PasswordChangedAt: user.PasswordChangedAt,
        CreatedAt:         user.CreatedAt,
    }
}

func (server *Server) createUser(ctx *gin.Context) {
    var req createUserRequest
    if err := ctx.ShouldBindJSON(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    hashedPassword, err := util.HashPassword(req.Password)
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    arg := db.CreateUserParams{
        Username:       req.Username,
        HashedPassword: hashedPassword,
        FullName:       req.FullName,
        Email:          req.Email,
    }

    user, err := server.store.CreateUser(ctx, arg)
    if err != nil {
        if db.ErrorCode(err) == db.UniqueViolation {
            ctx.JSON(http.StatusForbidden, errorResponse(err))
            return
        }
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    rsp := newUserResponse(user)
    ctx.JSON(http.StatusOK, rsp)
}

type loginUserRequest struct {
    Username string `json:"username" binding:"required,alphanum"`
    Password string `json:"password" binding:"required,min=6"`
}

type loginUserResponse struct {
    SessionID             uuid.UUID    `json:"session_id"`
    AccessToken           string       `json:"access_token"`
    AccessTokenExpiresAt  time.Time    `json:"access_token_expires_at"`
    RefreshToken          string       `json:"refresh_token"`
    RefreshTokenExpiresAt time.Time    `json:"refresh_token_expires_at"`
    User                  userResponse `json:"user"`
}

func (server *Server) loginUser(ctx *gin.Context) {
    var req loginUserRequest
    if err := ctx.ShouldBindJSON(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, errorResponse(err))
        return
    }

    user, err := server.store.GetUser(ctx, req.Username)
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            ctx.JSON(http.StatusNotFound, errorResponse(err))
            return
        }
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    err = util.CheckPassword(req.Password, user.HashedPassword)
    if err != nil {
        ctx.JSON(http.StatusUnauthorized, errorResponse(err))
        return
    }

    accessToken, accessPayload, err := server.tokenMaker.CreateToken(
        user.Username,
        user.Role,
        server.config.AccessTokenDuration,
        token.TokenTypeAccessToken,
    )
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    refreshToken, refreshPayload, err := server.tokenMaker.CreateToken(
        user.Username,
        user.Role,
        server.config.RefreshTokenDuration,
        token.TokenTypeRefreshToken,
    )
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    session, err := server.store.CreateSession(ctx, db.CreateSessionParams{
        ID:           refreshPayload.ID,
        Username:     user.Username,
        RefreshToken: refreshToken,
        UserAgent:    ctx.Request.UserAgent(),
        ClientIp:     ctx.ClientIP(),
        IsBlocked:    false,
        ExpiresAt:    refreshPayload.ExpiredAt,
    })
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, errorResponse(err))
        return
    }

    rsp := loginUserResponse{
        SessionID:             session.ID,
        AccessToken:           accessToken,
        AccessTokenExpiresAt:  accessPayload.ExpiredAt,
        RefreshToken:          refreshToken,
        RefreshTokenExpiresAt: refreshPayload.ExpiredAt,
        User:                  newUserResponse(user),
    }
    ctx.JSON(http.StatusOK, rsp)
}
```

FILE: simplebank/api/user_test.go

```
package api

import (
    "bytes"
    "database/sql"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/http/httptest"
    "reflect"
    "testing"

    "github.com/gin-gonic/gin"
    "github.com/golang/mock/gomock"
    "github.com/stretchr/testify/require"
    mockdb "github.com/techschool/simplebank/db/mock"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/util"
)

type eqCreateUserParamsMatcher struct {
```

```
    arg      db.CreateUserParams
    password string
}

func (e eqCreateUserParamsMatcher) Matches(x interface{}) bool {
    arg, ok := x.(db.CreateUserParams)
    if !ok {
        return false
    }

    err := util.CheckPassword(e.password, arg.HashedPassword)
    if err != nil {
        return false
    }

    e.arg.HashedPassword = arg.HashedPassword
    return reflect.DeepEqual(e.arg, arg)
}

func (e eqCreateUserParamsMatcher) String() string {
    return fmt.Sprintf("matches arg %v and password %v", e.arg, e.password)
}

func EqCreateUserParams(arg db.CreateUserParams, password string) gomock.Matcher {
    return eqCreateUserParamsMatcher{arg, password}
}

func TestCreateUserAPI(t *testing.T) {
    user, password := randomUser(t)

    testCases := []struct {
        name          string
        body          gin.H
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(recoder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            body: gin.H{
                "username":  user.Username,
                "password":  password,
                "full_name": user.FullName,
                "email":     user.Email,
            },
            buildStubs: func(store *mockdb.MockStore) {
                arg := db.CreateUserParams{
                    Username: user.Username,
                    FullName: user.FullName,
                    Email:    user.Email,
                }
                store.EXPECT().
                    CreateUser(gomock.Any(), EqCreateUserParams(arg, password)).
                    Times(1).
                    Return(user, nil)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
                requireBodyMatchUser(t, recorder.Body, user)
            },
        },
        {
            name: "InternalError",
            body: gin.H{
                "username":  user.Username,
                "password":  password,
                "full_name": user.FullName,
                "email":     user.Email,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateUser(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.User{}, sql.ErrConnDone)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name: "DuplicateUsername",
            body: gin.H{
                "username":  user.Username,
                "password":  password,
                "full_name": user.FullName,
                "email":     user.Email,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateUser(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.User{}, db.ErrUniqueViolation)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusForbidden, recorder.Code)
            },
        },
        {
            name: "InvalidUsername",
            body: gin.H{
                "username":  "invalid-user#1",
                "password":  password,
                "full_name": user.FullName,
                "email":     user.Email,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "InvalidEmail",
            body: gin.H{
                "username":  user.Username,
                "password":  password,
                "full_name": user.FullName,
                "email":     "invalid-email",
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
        {
            name: "TooShortPassword",
            body: gin.H{
                "username":  user.Username,
                "password":  "123",
                "full_name": user.FullName,
                "email":     user.Email,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    CreateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            // Marshal body data to JSON
            data, err := json.Marshal(tc.body)
            require.NoError(t, err)

            url := "/users"
            request, err := http.NewRequest(http.MethodPost, url, bytes.NewReader(data))
            require.NoError(t, err)

            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(recorder)
        })
    }
}

func TestLoginUserAPI(t *testing.T) {
    user, password := randomUser(t)

    testCases := []struct {
        name          string
        body          gin.H
        buildStubs    func(store *mockdb.MockStore)
        checkResponse func(recoder *httptest.ResponseRecorder)
    }{
        {
            name: "OK",
            body: gin.H{
                "username": user.Username,
                "password": password,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetUser(gomock.Any(), gomock.Eq(user.Username)).
                    Times(1).
                    Return(user, nil)
                store.EXPECT().
                    CreateSession(gomock.Any(), gomock.Any()).
                    Times(1)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusOK, recorder.Code)
            },
        },
        {
            name: "UserNotFound",
            body: gin.H{
                "username": "NotFound",
                "password": password,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetUser(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.User{}, db.ErrRecordNotFound)
            },
```

```
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusNotFound, recorder.Code)
            },
        },
        {
            name: "IncorrectPassword",
            body: gin.H{
                "username": user.Username,
                "password": "incorrect",
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetUser(gomock.Any(), gomock.Eq(user.Username)).
                    Times(1).
                    Return(user, nil)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusUnauthorized, recorder.Code)
            },
        },
        {
            name: "InternalError",
            body: gin.H{
                "username": user.Username,
                "password": password,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetUser(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.User{}, sql.ErrConnDone)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusInternalServerError, recorder.Code)
            },
        },
        {
            name: "InvalidUsername",
            body: gin.H{
                "username": "invalid-user#1",
                "password": password,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    GetUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(recorder *httptest.ResponseRecorder) {
                require.Equal(t, http.StatusBadRequest, recorder.Code)
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            ctrl := gomock.NewController(t)
            defer ctrl.Finish()

            store := mockdb.NewMockStore(ctrl)
            tc.buildStubs(store)

            server := newTestServer(t, store)
            recorder := httptest.NewRecorder()

            // Marshal body data to JSON
            data, err := json.Marshal(tc.body)
            require.NoError(t, err)

            url := "/users/login"
            request, err := http.NewRequest(http.MethodPost, url, bytes.NewReader(data))
            require.NoError(t, err)

            server.router.ServeHTTP(recorder, request)
            tc.checkResponse(recorder)
        })
    }
}

func randomUser(t *testing.T) (user db.User, password string) {
    password = util.RandomString(6)
    hashedPassword, err := util.HashPassword(password)
    require.NoError(t, err)

    user = db.User{
        Username:       util.RandomOwner(),
        HashedPassword: hashedPassword,
        FullName:       util.RandomOwner(),
        Email:          util.RandomEmail(),
    }
    return
}

func requireBodyMatchUser(t *testing.T, body *bytes.Buffer, user db.User) {
    data, err := io.ReadAll(body)
    require.NoError(t, err)

    var gotUser db.User
    err = json.Unmarshal(data, &gotUser)

    require.NoError(t, err)
    require.Equal(t, user.Username, gotUser.Username)
    require.Equal(t, user.FullName, gotUser.FullName)
    require.Equal(t, user.Email, gotUser.Email)
    require.Empty(t, gotUser.HashedPassword)
}
```

FILE: simplebank/api/validator.go

```
package api

import (
    "github.com/go-playground/validator/v10"
    "github.com/techschool/simplebank/util"
)

var validCurrency validator.Func = func(fieldLevel validator.FieldLevel) bool {
    if currency, ok := fieldLevel.Field().Interface().(string); ok {
        return util.IsSupportedCurrency(currency)
    }
    return false
}
```

FILE: simplebank/db/mock/store.go

```
// Code generated by MockGen. DO NOT EDIT.
// Source: github.com/techschool/simplebank/db/sqlc (interfaces: Store)

// Package mockdb is a generated GoMock package.
package mockdb

import (
    context "context"
    gomock "github.com/golang/mock/gomock"
    uuid "github.com/google/uuid"
    db "github.com/techschool/simplebank/db/sqlc"
    reflect "reflect"
)

// MockStore is a mock of Store interface
type MockStore struct {
    ctrl     *gomock.Controller
    recorder *MockStoreMockRecorder
}

// MockStoreMockRecorder is the mock recorder for MockStore
type MockStoreMockRecorder struct {
    mock *MockStore
}

// NewMockStore creates a new mock instance
func NewMockStore(ctrl *gomock.Controller) *MockStore {
    mock := &MockStore{ctrl: ctrl}
    mock.recorder = &MockStoreMockRecorder{mock}
    return mock
}

// EXPECT returns an object that allows the caller to indicate expected use
func (m *MockStore) EXPECT() *MockStoreMockRecorder {
    return m.recorder
}

// AddAccountBalance mocks base method
func (m *MockStore) AddAccountBalance(arg0 context.Context, arg1 db.AddAccountBalanceParams)
(db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "AddAccountBalance", arg0, arg1)
    ret0, _ := ret[0].(db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// AddAccountBalance indicates an expected call of AddAccountBalance
func (mr *MockStoreMockRecorder) AddAccountBalance(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "AddAccountBalance",
reflect.TypeOf((*MockStore)(nil).AddAccountBalance), arg0, arg1)
}

// CreateAccount mocks base method
func (m *MockStore) CreateAccount(arg0 context.Context, arg1 db.CreateAccountParams)
(db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateAccount", arg0, arg1)
    ret0, _ := ret[0].(db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateAccount indicates an expected call of CreateAccount
func (mr *MockStoreMockRecorder) CreateAccount(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateAccount",
reflect.TypeOf((*MockStore)(nil).CreateAccount), arg0, arg1)
}

// CreateEntry mocks base method
func (m *MockStore) CreateEntry(arg0 context.Context, arg1 db.CreateEntryParams) (db.Entry,
error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateEntry", arg0, arg1)
    ret0, _ := ret[0].(db.Entry)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateEntry indicates an expected call of CreateEntry
func (mr *MockStoreMockRecorder) CreateEntry(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateEntry",
reflect.TypeOf((*MockStore)(nil).CreateEntry), arg0, arg1)
}

// CreateSession mocks base method
func (m *MockStore) CreateSession(arg0 context.Context, arg1 db.CreateSessionParams)
(db.Session, error) {
```

```
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateSession", arg0, arg1)
    ret0, _ := ret[0].(db.Session)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateSession indicates an expected call of CreateSession
func (mr *MockStoreMockRecorder) CreateSession(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateSession",
reflect.TypeOf((*MockStore)(nil).CreateSession), arg0, arg1)
}

// CreateTransfer mocks base method
func (m *MockStore) CreateTransfer(arg0 context.Context, arg1 db.CreateTransferParams)
(db.Transfer, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateTransfer", arg0, arg1)
    ret0, _ := ret[0].(db.Transfer)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateTransfer indicates an expected call of CreateTransfer
func (mr *MockStoreMockRecorder) CreateTransfer(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateTransfer",
reflect.TypeOf((*MockStore)(nil).CreateTransfer), arg0, arg1)
}

// CreateUser mocks base method
func (m *MockStore) CreateUser(arg0 context.Context, arg1 db.CreateUserParams) (db.User, error)
{
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateUser", arg0, arg1)
    ret0, _ := ret[0].(db.User)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateUser indicates an expected call of CreateUser
func (mr *MockStoreMockRecorder) CreateUser(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateUser",
reflect.TypeOf((*MockStore)(nil).CreateUser), arg0, arg1)
}

// CreateUserTx mocks base method
func (m *MockStore) CreateUserTx(arg0 context.Context, arg1 db.CreateUserTxParams)
(db.CreateUserTxResult, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateUserTx", arg0, arg1)
    ret0, _ := ret[0].(db.CreateUserTxResult)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateUserTx indicates an expected call of CreateUserTx
func (mr *MockStoreMockRecorder) CreateUserTx(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateUserTx",
reflect.TypeOf((*MockStore)(nil).CreateUserTx), arg0, arg1)
}

// CreateVerifyEmail mocks base method
func (m *MockStore) CreateVerifyEmail(arg0 context.Context, arg1 db.CreateVerifyEmailParams)
(db.VerifyEmail, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "CreateVerifyEmail", arg0, arg1)
    ret0, _ := ret[0].(db.VerifyEmail)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// CreateVerifyEmail indicates an expected call of CreateVerifyEmail
func (mr *MockStoreMockRecorder) CreateVerifyEmail(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "CreateVerifyEmail",
reflect.TypeOf((*MockStore)(nil).CreateVerifyEmail), arg0, arg1)
}

// DeleteAccount mocks base method
func (m *MockStore) DeleteAccount(arg0 context.Context, arg1 int64) error {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "DeleteAccount", arg0, arg1)
    ret0, _ := ret[0].(error)
    return ret0
}

// DeleteAccount indicates an expected call of DeleteAccount
func (mr *MockStoreMockRecorder) DeleteAccount(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "DeleteAccount",
reflect.TypeOf((*MockStore)(nil).DeleteAccount), arg0, arg1)
}

// GetAccount mocks base method
func (m *MockStore) GetAccount(arg0 context.Context, arg1 int64) (db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetAccount", arg0, arg1)
    ret0, _ := ret[0].(db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetAccount indicates an expected call of GetAccount
func (mr *MockStoreMockRecorder) GetAccount(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetAccount",
reflect.TypeOf((*MockStore)(nil).GetAccount), arg0, arg1)
}

// GetAccountForUpdate mocks base method
func (m *MockStore) GetAccountForUpdate(arg0 context.Context, arg1 int64) (db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetAccountForUpdate", arg0, arg1)
    ret0, _ := ret[0].(db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetAccountForUpdate indicates an expected call of GetAccountForUpdate
func (mr *MockStoreMockRecorder) GetAccountForUpdate(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetAccountForUpdate",
reflect.TypeOf((*MockStore)(nil).GetAccountForUpdate), arg0, arg1)
}

// GetEntry mocks base method
func (m *MockStore) GetEntry(arg0 context.Context, arg1 int64) (db.Entry, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetEntry", arg0, arg1)
    ret0, _ := ret[0].(db.Entry)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetEntry indicates an expected call of GetEntry
func (mr *MockStoreMockRecorder) GetEntry(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetEntry",
reflect.TypeOf((*MockStore)(nil).GetEntry), arg0, arg1)
}

// GetSession mocks base method
func (m *MockStore) GetSession(arg0 context.Context, arg1 uuid.UUID) (db.Session, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetSession", arg0, arg1)
    ret0, _ := ret[0].(db.Session)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetSession indicates an expected call of GetSession
func (mr *MockStoreMockRecorder) GetSession(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetSession",
reflect.TypeOf((*MockStore)(nil).GetSession), arg0, arg1)
}

// GetTransfer mocks base method
func (m *MockStore) GetTransfer(arg0 context.Context, arg1 int64) (db.Transfer, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetTransfer", arg0, arg1)
    ret0, _ := ret[0].(db.Transfer)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetTransfer indicates an expected call of GetTransfer
func (mr *MockStoreMockRecorder) GetTransfer(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetTransfer",
reflect.TypeOf((*MockStore)(nil).GetTransfer), arg0, arg1)
}

// GetUser mocks base method
func (m *MockStore) GetUser(arg0 context.Context, arg1 string) (db.User, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "GetUser", arg0, arg1)
    ret0, _ := ret[0].(db.User)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// GetUser indicates an expected call of GetUser
func (mr *MockStoreMockRecorder) GetUser(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "GetUser",
reflect.TypeOf((*MockStore)(nil).GetUser), arg0, arg1)
}

// ListAccounts mocks base method
func (m *MockStore) ListAccounts(arg0 context.Context, arg1 db.ListAccountsParams)
([]db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "ListAccounts", arg0, arg1)
    ret0, _ := ret[0].([]db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// ListAccounts indicates an expected call of ListAccounts
func (mr *MockStoreMockRecorder) ListAccounts(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "ListAccounts",
reflect.TypeOf((*MockStore)(nil).ListAccounts), arg0, arg1)
}

// ListEntries mocks base method
func (m *MockStore) ListEntries(arg0 context.Context, arg1 db.ListEntriesParams) ([]db.Entry,
error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "ListEntries", arg0, arg1)
    ret0, _ := ret[0].([]db.Entry)
```

```
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// ListEntries indicates an expected call of ListEntries
func (mr *MockStoreMockRecorder) ListEntries(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "ListEntries",
reflect.TypeOf((*MockStore)(nil).ListEntries), arg0, arg1)
}

// ListTransfers mocks base method
func (m *MockStore) ListTransfers(arg0 context.Context, arg1 db.ListTransfersParams)
([]db.Transfer, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "ListTransfers", arg0, arg1)
    ret0, _ := ret[0].([]db.Transfer)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// ListTransfers indicates an expected call of ListTransfers
func (mr *MockStoreMockRecorder) ListTransfers(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "ListTransfers",
reflect.TypeOf((*MockStore)(nil).ListTransfers), arg0, arg1)
}

// TransferTx mocks base method
func (m *MockStore) TransferTx(arg0 context.Context, arg1 db.TransferTxParams)
(db.TransferTxResult, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "TransferTx", arg0, arg1)
    ret0, _ := ret[0].(db.TransferTxResult)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// TransferTx indicates an expected call of TransferTx
func (mr *MockStoreMockRecorder) TransferTx(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "TransferTx",
reflect.TypeOf((*MockStore)(nil).TransferTx), arg0, arg1)
}

// UpdateAccount mocks base method
func (m *MockStore) UpdateAccount(arg0 context.Context, arg1 db.UpdateAccountParams)
(db.Account, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "UpdateAccount", arg0, arg1)
    ret0, _ := ret[0].(db.Account)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// UpdateAccount indicates an expected call of UpdateAccount
func (mr *MockStoreMockRecorder) UpdateAccount(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "UpdateAccount",
reflect.TypeOf((*MockStore)(nil).UpdateAccount), arg0, arg1)
}

// UpdateUser mocks base method
func (m *MockStore) UpdateUser(arg0 context.Context, arg1 db.UpdateUserParams) (db.User, error)
{
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "UpdateUser", arg0, arg1)
    ret0, _ := ret[0].(db.User)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// UpdateUser indicates an expected call of UpdateUser
func (mr *MockStoreMockRecorder) UpdateUser(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "UpdateUser",
reflect.TypeOf((*MockStore)(nil).UpdateUser), arg0, arg1)
}

// UpdateVerifyEmail mocks base method
func (m *MockStore) UpdateVerifyEmail(arg0 context.Context, arg1 db.UpdateVerifyEmailParams)
(db.VerifyEmail, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "UpdateVerifyEmail", arg0, arg1)
    ret0, _ := ret[0].(db.VerifyEmail)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// UpdateVerifyEmail indicates an expected call of UpdateVerifyEmail
func (mr *MockStoreMockRecorder) UpdateVerifyEmail(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "UpdateVerifyEmail",
reflect.TypeOf((*MockStore)(nil).UpdateVerifyEmail), arg0, arg1)
}

// VerifyEmailTx mocks base method
func (m *MockStore) VerifyEmailTx(arg0 context.Context, arg1 db.VerifyEmailTxParams)
(db.VerifyEmailTxResult, error) {
    m.ctrl.T.Helper()
    ret := m.ctrl.Call(m, "VerifyEmailTx", arg0, arg1)
    ret0, _ := ret[0].(db.VerifyEmailTxResult)
    ret1, _ := ret[1].(error)
    return ret0, ret1
}

// VerifyEmailTx indicates an expected call of VerifyEmailTx
func (mr *MockStoreMockRecorder) VerifyEmailTx(arg0, arg1 interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "VerifyEmailTx",
reflect.TypeOf((*MockStore)(nil).VerifyEmailTx), arg0, arg1)
}
```

FILE: simplebank/db/sqlc/account.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: account.sql

package db

import (
    "context"
)

const addAccountBalance = `-- name: AddAccountBalance :one
UPDATE accounts
SET balance = balance + $1
WHERE id = $2
RETURNING id, owner, balance, currency, created_at
`


type AddAccountBalanceParams struct {
    Amount int64 `json:"amount"`
    ID     int64 `json:"id"`
}

func (q *Queries) AddAccountBalance(ctx context.Context, arg AddAccountBalanceParams) (Account,
error) {
    row := q.db.QueryRow(ctx, addAccountBalance, arg.Amount, arg.ID)
    var i Account
    err := row.Scan(
        &i.ID,
        &i.Owner,
        &i.Balance,
        &i.Currency,
        &i.CreatedAt,
    )
    return i, err
}

const createAccount = `-- name: CreateAccount :one
INSERT INTO accounts (
  owner,
  balance,
  currency
) VALUES (
  $1, $2, $3
) RETURNING id, owner, balance, currency, created_at
`


type CreateAccountParams struct {
    Owner    string `json:"owner"`
    Balance  int64  `json:"balance"`
    Currency string `json:"currency"`
}

func (q *Queries) CreateAccount(ctx context.Context, arg CreateAccountParams) (Account, error)
{
    row := q.db.QueryRow(ctx, createAccount, arg.Owner, arg.Balance, arg.Currency)
    var i Account
    err := row.Scan(
        &i.ID,
        &i.Owner,
        &i.Balance,
        &i.Currency,
        &i.CreatedAt,
    )
    return i, err
}

const deleteAccount = `-- name: DeleteAccount :exec
DELETE FROM accounts
WHERE id = $1
`


func (q *Queries) DeleteAccount(ctx context.Context, id int64) error {
    _, err := q.db.Exec(ctx, deleteAccount, id)
    return err
}

const getAccount = `-- name: GetAccount :one
SELECT id, owner, balance, currency, created_at FROM accounts
WHERE id = $1 LIMIT 1
`


func (q *Queries) GetAccount(ctx context.Context, id int64) (Account, error) {
    row := q.db.QueryRow(ctx, getAccount, id)
    var i Account
    err := row.Scan(
        &i.ID,
        &i.Owner,
        &i.Balance,
        &i.Currency,
        &i.CreatedAt,
    )
    return i, err
}

const getAccountForUpdate = `-- name: GetAccountForUpdate :one
SELECT id, owner, balance, currency, created_at FROM accounts
WHERE id = $1 LIMIT 1
FOR NO KEY UPDATE
`


func (q *Queries) GetAccountForUpdate(ctx context.Context, id int64) (Account, error) {
```

```
    row := q.db.QueryRow(ctx, getAccountForUpdate, id)
    var i Account
    err := row.Scan(
        &i.ID,
        &i.Owner,
        &i.Balance,
        &i.Currency,
        &i.CreatedAt,
    )
    return i, err
}

const listAccounts = `-- name: ListAccounts :many
SELECT id, owner, balance, currency, created_at FROM accounts
WHERE owner = $1
ORDER BY id
LIMIT $2
OFFSET $3
`

type ListAccountsParams struct {
    Owner  string `json:"owner"`
    Limit  int32  `json:"limit"`
    Offset int32  `json:"offset"`
}

func (q *Queries) ListAccounts(ctx context.Context, arg ListAccountsParams) ([]Account, error)
{
    rows, err := q.db.Query(ctx, listAccounts, arg.Owner, arg.Limit, arg.Offset)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    items := []Account{}
    for rows.Next() {
        var i Account
        if err := rows.Scan(
            &i.ID,
            &i.Owner,
            &i.Balance,
            &i.Currency,
            &i.CreatedAt,
        ); err != nil {
            return nil, err
        }
        items = append(items, i)
    }
    if err := rows.Err(); err != nil {
        return nil, err
    }
    return items, nil
}

const updateAccount = `-- name: UpdateAccount :one
UPDATE accounts
SET balance = $2
WHERE id = $1
RETURNING id, owner, balance, currency, created_at
`

type UpdateAccountParams struct {
    ID      int64 `json:"id"`
    Balance int64 `json:"balance"`
}

func (q *Queries) UpdateAccount(ctx context.Context, arg UpdateAccountParams) (Account, error)
{
    row := q.db.QueryRow(ctx, updateAccount, arg.ID, arg.Balance)
    var i Account
    err := row.Scan(
        &i.ID,
        &i.Owner,
        &i.Balance,
        &i.Currency,
        &i.CreatedAt,
    )
    return i, err
}
```

FILE: simplebank/db/sqlc/account_test.go

```
package db

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func createRandomAccount(t *testing.T) Account {
    user := createRandomUser(t)

    arg := CreateAccountParams{
        Owner:    user.Username,
        Balance:  util.RandomMoney(),
        Currency: util.RandomCurrency(),
    }

    account, err := testStore.CreateAccount(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, account)

    require.Equal(t, arg.Owner, account.Owner)
    require.Equal(t, arg.Balance, account.Balance)
    require.Equal(t, arg.Currency, account.Currency)

    require.NotZero(t, account.ID)
    require.NotZero(t, account.CreatedAt)

    return account
}

func TestCreateAccount(t *testing.T) {
    createRandomAccount(t)
}

func TestGetAccount(t *testing.T) {
    account1 := createRandomAccount(t)
    account2, err := testStore.GetAccount(context.Background(), account1.ID)
    require.NoError(t, err)
    require.NotEmpty(t, account2)

    require.Equal(t, account1.ID, account2.ID)
    require.Equal(t, account1.Owner, account2.Owner)
    require.Equal(t, account1.Balance, account2.Balance)
    require.Equal(t, account1.Currency, account2.Currency)
    require.WithinDuration(t, account1.CreatedAt, account2.CreatedAt, time.Second)
}

func TestUpdateAccount(t *testing.T) {
    account1 := createRandomAccount(t)

    arg := UpdateAccountParams{
        ID:      account1.ID,
        Balance: util.RandomMoney(),
    }

    account2, err := testStore.UpdateAccount(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, account2)

    require.Equal(t, account1.ID, account2.ID)
    require.Equal(t, account1.Owner, account2.Owner)
    require.Equal(t, arg.Balance, account2.Balance)
    require.Equal(t, account1.Currency, account2.Currency)
    require.WithinDuration(t, account1.CreatedAt, account2.CreatedAt, time.Second)
}

func TestDeleteAccount(t *testing.T) {
    account1 := createRandomAccount(t)
    err := testStore.DeleteAccount(context.Background(), account1.ID)
    require.NoError(t, err)

    account2, err := testStore.GetAccount(context.Background(), account1.ID)
    require.Error(t, err)
    require.EqualError(t, err, ErrRecordNotFound.Error())
    require.Empty(t, account2)
}

func TestListAccounts(t *testing.T) {
    var lastAccount Account
    for i := 0; i < 10; i++ {
        lastAccount = createRandomAccount(t)
    }

    arg := ListAccountsParams{
        Owner:  lastAccount.Owner,
        Limit:  5,
        Offset: 0,
    }

    accounts, err := testStore.ListAccounts(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, accounts)

    for _, account := range accounts {
        require.NotEmpty(t, account)
        require.Equal(t, lastAccount.Owner, account.Owner)
    }
}
```

FILE: simplebank/db/sqlc/db.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0

package db

import (
    "context"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

type DBTX interface {
    Exec(context.Context, string, ...interface{}) (pgconn.CommandTag, error)
    Query(context.Context, string, ...interface{}) (pgx.Rows, error)
    QueryRow(context.Context, string, ...interface{}) pgx.Row
}

func New(db DBTX) *Queries {
    return &Queries{db: db}
}

type Queries struct {
    db DBTX
}

func (q *Queries) WithTx(tx pgx.Tx) *Queries {
    return &Queries{
        db: tx,
    }
}
```

FILE: simplebank/db/sqlc/entry.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: entry.sql

package db

import (
    "context"
)

const createEntry = `-- name: CreateEntry :one
INSERT INTO entries (
  account_id,
  amount
) VALUES (
  $1, $2
) RETURNING id, account_id, amount, created_at
`

type CreateEntryParams struct {
    AccountID int64 `json:"account_id"`
    Amount    int64 `json:"amount"`
}

func (q *Queries) CreateEntry(ctx context.Context, arg CreateEntryParams) (Entry, error) {
    row := q.db.QueryRow(ctx, createEntry, arg.AccountID, arg.Amount)
    var i Entry
    err := row.Scan(
        &i.ID,
        &i.AccountID,
        &i.Amount,
        &i.CreatedAt,
    )
    return i, err
}

const getEntry = `-- name: GetEntry :one
SELECT id, account_id, amount, created_at FROM entries
WHERE id = $1 LIMIT 1
`

func (q *Queries) GetEntry(ctx context.Context, id int64) (Entry, error) {
    row := q.db.QueryRow(ctx, getEntry, id)
    var i Entry
    err := row.Scan(
        &i.ID,
        &i.AccountID,
        &i.Amount,
        &i.CreatedAt,
    )
    return i, err
}

const listEntries = `-- name: ListEntries :many
SELECT id, account_id, amount, created_at FROM entries
WHERE account_id = $1
ORDER BY id
LIMIT $2
OFFSET $3
`

type ListEntriesParams struct {
    AccountID int64 `json:"account_id"`
    Limit     int32 `json:"limit"`
    Offset    int32 `json:"offset"`
}

func (q *Queries) ListEntries(ctx context.Context, arg ListEntriesParams) ([]Entry, error) {
    rows, err := q.db.Query(ctx, listEntries, arg.AccountID, arg.Limit, arg.Offset)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    items := []Entry{}
    for rows.Next() {
        var i Entry
        if err := rows.Scan(
            &i.ID,
            &i.AccountID,
            &i.Amount,
            &i.CreatedAt,
        ); err != nil {
            return nil, err
        }
        items = append(items, i)
    }
    if err := rows.Err(); err != nil {
        return nil, err
    }
    return items, nil
}
```

FILE: simplebank/db/sqlc/entry_test.go

```
package db

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func createRandomEntry(t *testing.T, account Account) Entry {
    arg := CreateEntryParams{
        AccountID: account.ID,
        Amount:    util.RandomMoney(),
    }

    entry, err := testStore.CreateEntry(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, entry)

    require.Equal(t, arg.AccountID, entry.AccountID)
    require.Equal(t, arg.Amount, entry.Amount)

    require.NotZero(t, entry.ID)
    require.NotZero(t, entry.CreatedAt)

    return entry
}

func TestCreateEntry(t *testing.T) {
    account := createRandomAccount(t)
    createRandomEntry(t, account)
}

func TestGetEntry(t *testing.T) {
    account := createRandomAccount(t)
    entry1 := createRandomEntry(t, account)
    entry2, err := testStore.GetEntry(context.Background(), entry1.ID)
    require.NoError(t, err)
    require.NotEmpty(t, entry2)

    require.Equal(t, entry1.ID, entry2.ID)
    require.Equal(t, entry1.AccountID, entry2.AccountID)
    require.Equal(t, entry1.Amount, entry2.Amount)
    require.WithinDuration(t, entry1.CreatedAt, entry2.CreatedAt, time.Second)
}

func TestListEntries(t *testing.T) {
    account := createRandomAccount(t)
    for i := 0; i < 10; i++ {
        createRandomEntry(t, account)
    }

    arg := ListEntriesParams{
        AccountID: account.ID,
        Limit:     5,
        Offset:    5,
    }

    entries, err := testStore.ListEntries(context.Background(), arg)
    require.NoError(t, err)
    require.Len(t, entries, 5)

    for _, entry := range entries {
        require.NotEmpty(t, entry)
        require.Equal(t, arg.AccountID, entry.AccountID)
    }
}
```

FILE: simplebank/db/sqlc/error.go

```
package db

import (
    "errors"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
)

const (
    ForeignKeyViolation = "23503"
    UniqueViolation     = "23505"
)

var ErrRecordNotFound = pgx.ErrNoRows

var ErrUniqueViolation = &pgconn.PgError{
    Code: UniqueViolation,
}

func ErrorCode(err error) string {
    var pgErr *pgconn.PgError
    if errors.As(err, &pgErr) {
        return pgErr.Code
    }
    return ""
}
```

FILE: simplebank/db/sqlc/exec_tx.go

```
package db

import (
    "context"
    "fmt"
)

// ExecTx executes a function within a database transaction
func (store *SQLStore) execTx(ctx context.Context, fn func(*Queries) error) error {
    tx, err := store.connPool.Begin(ctx)
    if err != nil {
        return err
    }

    q := New(tx)
    err = fn(q)
    if err != nil {
        if rbErr := tx.Rollback(ctx); rbErr != nil {
            return fmt.Errorf("tx err: %v, rb err: %v", err, rbErr)
        }
```

```
        return err
    }

    return tx.Commit(ctx)
}
```

FILE: simplebank/db/sqlc/main_test.go

```
package db

import (
    "context"
    "log"
    "os"
    "testing"

    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/techschool/simplebank/util"
)

var testStore Store

func TestMain(m *testing.M) {
    config, err := util.LoadConfig("../..")
    if err != nil {
        log.Fatal("cannot load config:", err)
    }

    connPool, err := pgxpool.New(context.Background(), config.DBSource)
    if err != nil {
        log.Fatal("cannot connect to db:", err)
    }

    testStore = NewStore(connPool)
    os.Exit(m.Run())
}
```

FILE: simplebank/db/sqlc/models.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0

package db

import (
    "time"

    "github.com/google/uuid"
)

type Account struct {
    ID        int64     `json:"id"`
    Owner     string    `json:"owner"`
    Balance   int64     `json:"balance"`
    Currency  string    `json:"currency"`
    CreatedAt time.Time `json:"created_at"`
}

type Entry struct {
    ID        int64 `json:"id"`
    AccountID int64 `json:"account_id"`
    // can be negative or positive
    Amount    int64     `json:"amount"`
    CreatedAt time.Time `json:"created_at"`
}

type Session struct {
    ID           uuid.UUID `json:"id"`
    Username     string    `json:"username"`
    RefreshToken string    `json:"refresh_token"`
    UserAgent    string    `json:"user_agent"`
    ClientIp     string    `json:"client_ip"`
    IsBlocked    bool      `json:"is_blocked"`
    ExpiresAt    time.Time `json:"expires_at"`
    CreatedAt    time.Time `json:"created_at"`
}

type Transfer struct {
    ID            int64 `json:"id"`
    FromAccountID int64 `json:"from_account_id"`
    ToAccountID   int64 `json:"to_account_id"`
    // must be positive
    Amount    int64     `json:"amount"`
    CreatedAt time.Time `json:"created_at"`
}

type User struct {
    Username          string    `json:"username"`
    HashedPassword    string    `json:"hashed_password"`
    FullName          string    `json:"full_name"`
    Email             string    `json:"email"`
    PasswordChangedAt time.Time `json:"password_changed_at"`
    CreatedAt         time.Time `json:"created_at"`
    IsEmailVerified   bool      `json:"is_email_verified"`
    Role              string    `json:"role"`
}

type VerifyEmail struct {
    ID         int64     `json:"id"`
    Username   string    `json:"username"`
    Email      string    `json:"email"`
    SecretCode string    `json:"secret_code"`
    IsUsed     bool      `json:"is_used"`
    CreatedAt  time.Time `json:"created_at"`
    ExpiredAt  time.Time `json:"expired_at"`
}
```

FILE: simplebank/db/sqlc/querier.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0

package db

import (
    "context"

    "github.com/google/uuid"
)

type Querier interface {
    AddAccountBalance(ctx context.Context, arg AddAccountBalanceParams) (Account, error)
    CreateAccount(ctx context.Context, arg CreateAccountParams) (Account, error)
    CreateEntry(ctx context.Context, arg CreateEntryParams) (Entry, error)
    CreateSession(ctx context.Context, arg CreateSessionParams) (Session, error)
    CreateTransfer(ctx context.Context, arg CreateTransferParams) (Transfer, error)
    CreateUser(ctx context.Context, arg CreateUserParams) (User, error)
    CreateVerifyEmail(ctx context.Context, arg CreateVerifyEmailParams) (VerifyEmail, error)
    DeleteAccount(ctx context.Context, id int64) error
    GetAccount(ctx context.Context, id int64) (Account, error)
    GetAccountForUpdate(ctx context.Context, id int64) (Account, error)
    GetEntry(ctx context.Context, id int64) (Entry, error)
    GetSession(ctx context.Context, id uuid.UUID) (Session, error)
    GetTransfer(ctx context.Context, id int64) (Transfer, error)
    GetUser(ctx context.Context, username string) (User, error)
    ListAccounts(ctx context.Context, arg ListAccountsParams) ([]Account, error)
    ListEntries(ctx context.Context, arg ListEntriesParams) ([]Entry, error)
    ListTransfers(ctx context.Context, arg ListTransfersParams) ([]Transfer, error)
    UpdateAccount(ctx context.Context, arg UpdateAccountParams) (Account, error)
    UpdateUser(ctx context.Context, arg UpdateUserParams) (User, error)
    UpdateVerifyEmail(ctx context.Context, arg UpdateVerifyEmailParams) (VerifyEmail, error)
}

var _ Querier = (*Queries)(nil)
```

FILE: simplebank/db/sqlc/session.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: session.sql

package db

import (
    "context"
    "time"

    "github.com/google/uuid"
)

const createSession = `-- name: CreateSession :one
INSERT INTO sessions (
  id,
  username,
  refresh_token,
  user_agent,
  client_ip,
  is_blocked,
  expires_at
) VALUES (
  $1, $2, $3, $4, $5, $6, $7
) RETURNING id, username, refresh_token, user_agent, client_ip, is_blocked, expires_at,
created_at
`

type CreateSessionParams struct {
    ID           uuid.UUID `json:"id"`
    Username     string    `json:"username"`
    RefreshToken string    `json:"refresh_token"`
    UserAgent    string    `json:"user_agent"`
    ClientIp     string    `json:"client_ip"`
    IsBlocked    bool      `json:"is_blocked"`
    ExpiresAt    time.Time `json:"expires_at"`
}

func (q *Queries) CreateSession(ctx context.Context, arg CreateSessionParams) (Session, error)
{
    row := q.db.QueryRow(ctx, createSession,
        arg.ID,
        arg.Username,
        arg.RefreshToken,
        arg.UserAgent,
        arg.ClientIp,
        arg.IsBlocked,
        arg.ExpiresAt,
    )
    var i Session
    err := row.Scan(
        &i.ID,
        &i.Username,
        &i.RefreshToken,
        &i.UserAgent,
        &i.ClientIp,
        &i.IsBlocked,
        &i.ExpiresAt,
        &i.CreatedAt,
    )
    return i, err
}

const getSession = `-- name: GetSession :one
SELECT id, username, refresh_token, user_agent, client_ip, is_blocked, expires_at, created_at
FROM sessions
WHERE id = $1 LIMIT 1
`

func (q *Queries) GetSession(ctx context.Context, id uuid.UUID) (Session, error) {
```

```
    row := q.db.QueryRow(ctx, getSession, id)
    var i Session
    err := row.Scan(
        &i.ID,
        &i.Username,
        &i.RefreshToken,
        &i.UserAgent,
        &i.ClientIp,
        &i.IsBlocked,
        &i.ExpiresAt,
        &i.CreatedAt,
    )
    return i, err
}
```

FILE: simplebank/db/sqlc/store.go

```
package db

import (
    "context"

    "github.com/jackc/pgx/v5/pgxpool"
)

// Store defines all functions to execute db queries and transactions
type Store interface {
    Querier
    TransferTx(ctx context.Context, arg TransferTxParams) (TransferTxResult, error)
    CreateUserTx(ctx context.Context, arg CreateUserTxParams) (CreateUserTxResult, error)
    VerifyEmailTx(ctx context.Context, arg VerifyEmailTxParams) (VerifyEmailTxResult, error)
}

// SQLStore provides all functions to execute SQL queries and transactions
type SQLStore struct {
    connPool *pgxpool.Pool
    *Queries
}

// NewStore creates a new store
func NewStore(connPool *pgxpool.Pool) Store {
    return &SQLStore{
        connPool: connPool,
        Queries:  New(connPool),
    }
}
```

FILE: simplebank/db/sqlc/store_test.go

```
package db

import (
    "context"
    "fmt"
    "testing"

    "github.com/stretchr/testify/require"
)

func TestTransferTx(t *testing.T) {
    account1 := createRandomAccount(t)
    account2 := createRandomAccount(t)
    fmt.Println(">> before:", account1.Balance, account2.Balance)

    n := 5
    amount := int64(10)

    errs := make(chan error)
    results := make(chan TransferTxResult)

    // run n concurrent transfer transaction
    for i := 0; i < n; i++ {
        go func() {
            result, err := testStore.TransferTx(context.Background(), TransferTxParams{
                FromAccountID: account1.ID,
                ToAccountID:   account2.ID,
                Amount:        amount,
            })

            errs <- err
            results <- result
        }()
    }

    // check results
    existed := make(map[int]bool)

    for i := 0; i < n; i++ {
        err := <-errs
        require.NoError(t, err)

        result := <-results
        require.NotEmpty(t, result)

        // check transfer
        transfer := result.Transfer
        require.NotEmpty(t, transfer)
        require.Equal(t, account1.ID, transfer.FromAccountID)
        require.Equal(t, account2.ID, transfer.ToAccountID)
        require.Equal(t, amount, transfer.Amount)
        require.NotZero(t, transfer.ID)
        require.NotZero(t, transfer.CreatedAt)

        _, err = testStore.GetTransfer(context.Background(), transfer.ID)
        require.NoError(t, err)

        // check entries
        fromEntry := result.FromEntry
        require.NotEmpty(t, fromEntry)
        require.Equal(t, account1.ID, fromEntry.AccountID)
        require.Equal(t, -amount, fromEntry.Amount)
        require.NotZero(t, fromEntry.ID)
        require.NotZero(t, fromEntry.CreatedAt)

        _, err = testStore.GetEntry(context.Background(), fromEntry.ID)
        require.NoError(t, err)

        toEntry := result.ToEntry
        require.NotEmpty(t, toEntry)
        require.Equal(t, account2.ID, toEntry.AccountID)
        require.Equal(t, amount, toEntry.Amount)
        require.NotZero(t, toEntry.ID)
        require.NotZero(t, toEntry.CreatedAt)

        _, err = testStore.GetEntry(context.Background(), toEntry.ID)
        require.NoError(t, err)

        // check accounts
        fromAccount := result.FromAccount
        require.NotEmpty(t, fromAccount)
        require.Equal(t, account1.ID, fromAccount.ID)

        toAccount := result.ToAccount
        require.NotEmpty(t, toAccount)
        require.Equal(t, account2.ID, toAccount.ID)

        // check balances
        fmt.Println(">> tx:", fromAccount.Balance, toAccount.Balance)

        diff1 := account1.Balance - fromAccount.Balance
        diff2 := toAccount.Balance - account2.Balance
        require.Equal(t, diff1, diff2)
        require.True(t, diff1 > 0)
        require.True(t, diff1%amount == 0) // 1 * amount, 2 * amount, 3 * amount, ..., n *
amount

        k := int(diff1 / amount)
        require.True(t, k >= 1 && k <= n)
        require.NotContains(t, existed, k)
        existed[k] = true
    }

    // check the final updated balance
    updatedAccount1, err := testStore.GetAccount(context.Background(), account1.ID)
    require.NoError(t, err)

    updatedAccount2, err := testStore.GetAccount(context.Background(), account2.ID)
    require.NoError(t, err)

    fmt.Println(">> after:", updatedAccount1.Balance, updatedAccount2.Balance)

    require.Equal(t, account1.Balance-int64(n)*amount, updatedAccount1.Balance)
    require.Equal(t, account2.Balance+int64(n)*amount, updatedAccount2.Balance)
}

func TestTransferTxDeadlock(t *testing.T) {
    account1 := createRandomAccount(t)
    account2 := createRandomAccount(t)
    fmt.Println(">> before:", account1.Balance, account2.Balance)

    n := 10
    amount := int64(10)
    errs := make(chan error)

    for i := 0; i < n; i++ {
        fromAccountID := account1.ID
        toAccountID := account2.ID

        if i%2 == 1 {
            fromAccountID = account2.ID
            toAccountID = account1.ID
        }

        go func() {
            _, err := testStore.TransferTx(context.Background(), TransferTxParams{
                FromAccountID: fromAccountID,
                ToAccountID:   toAccountID,
                Amount:        amount,
            })

            errs <- err
        }()
    }

    for i := 0; i < n; i++ {
        err := <-errs
        require.NoError(t, err)
    }

    // check the final updated balance
    updatedAccount1, err := testStore.GetAccount(context.Background(), account1.ID)
    require.NoError(t, err)

    updatedAccount2, err := testStore.GetAccount(context.Background(), account2.ID)
    require.NoError(t, err)

    fmt.Println(">> after:", updatedAccount1.Balance, updatedAccount2.Balance)
    require.Equal(t, account1.Balance, updatedAccount1.Balance)
    require.Equal(t, account2.Balance, updatedAccount2.Balance)
}
```

FILE: simplebank/db/sqlc/transfer.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: transfer.sql

package db
```

```
import (
    "context"
)

const createTransfer = `-- name: CreateTransfer :one
INSERT INTO transfers (
  from_account_id,
  to_account_id,
  amount
) VALUES (
  $1, $2, $3
) RETURNING id, from_account_id, to_account_id, amount, created_at
`

type CreateTransferParams struct {
    FromAccountID int64 `json:"from_account_id"`
    ToAccountID   int64 `json:"to_account_id"`
    Amount        int64 `json:"amount"`
}

func (q *Queries) CreateTransfer(ctx context.Context, arg CreateTransferParams) (Transfer,
error) {
    row := q.db.QueryRow(ctx, createTransfer, arg.FromAccountID, arg.ToAccountID, arg.Amount)
    var i Transfer
    err := row.Scan(
        &i.ID,
        &i.FromAccountID,
        &i.ToAccountID,
        &i.Amount,
        &i.CreatedAt,
    )
    return i, err
}

const getTransfer = `-- name: GetTransfer :one
SELECT id, from_account_id, to_account_id, amount, created_at FROM transfers
WHERE id = $1 LIMIT 1
`

func (q *Queries) GetTransfer(ctx context.Context, id int64) (Transfer, error) {
    row := q.db.QueryRow(ctx, getTransfer, id)
    var i Transfer
    err := row.Scan(
        &i.ID,
        &i.FromAccountID,
        &i.ToAccountID,
        &i.Amount,
        &i.CreatedAt,
    )
    return i, err
}

const listTransfers = `-- name: ListTransfers :many
SELECT id, from_account_id, to_account_id, amount, created_at FROM transfers
WHERE
    from_account_id = $1 OR
    to_account_id = $2
ORDER BY id
LIMIT $3
OFFSET $4
`

type ListTransfersParams struct {
    FromAccountID int64 `json:"from_account_id"`
    ToAccountID   int64 `json:"to_account_id"`
    Limit         int32 `json:"limit"`
    Offset        int32 `json:"offset"`
}

func (q *Queries) ListTransfers(ctx context.Context, arg ListTransfersParams) ([]Transfer,
error) {
    rows, err := q.db.Query(ctx, listTransfers,
        arg.FromAccountID,
        arg.ToAccountID,
        arg.Limit,
        arg.Offset,
    )
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    items := []Transfer{}
    for rows.Next() {
        var i Transfer
        if err := rows.Scan(
            &i.ID,
            &i.FromAccountID,
            &i.ToAccountID,
            &i.Amount,
            &i.CreatedAt,
        ); err != nil {
            return nil, err
        }
        items = append(items, i)
    }
    if err := rows.Err(); err != nil {
        return nil, err
    }
    return items, nil
}
```

FILE: simplebank/db/sqlc/transfer_test.go

```
package db

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func createRandomTransfer(t *testing.T, account1, account2 Account) Transfer {
    arg := CreateTransferParams{
        FromAccountID: account1.ID,
        ToAccountID:   account2.ID,
        Amount:        util.RandomMoney(),
    }

    transfer, err := testStore.CreateTransfer(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, transfer)

    require.Equal(t, arg.FromAccountID, transfer.FromAccountID)
    require.Equal(t, arg.ToAccountID, transfer.ToAccountID)
    require.Equal(t, arg.Amount, transfer.Amount)

    require.NotZero(t, transfer.ID)
    require.NotZero(t, transfer.CreatedAt)

    return transfer
}

func TestCreateTransfer(t *testing.T) {
    account1 := createRandomAccount(t)
    account2 := createRandomAccount(t)
    createRandomTransfer(t, account1, account2)
}

func TestGetTransfer(t *testing.T) {
    account1 := createRandomAccount(t)
    account2 := createRandomAccount(t)
    transfer1 := createRandomTransfer(t, account1, account2)

    transfer2, err := testStore.GetTransfer(context.Background(), transfer1.ID)
    require.NoError(t, err)
    require.NotEmpty(t, transfer2)

    require.Equal(t, transfer1.ID, transfer2.ID)
    require.Equal(t, transfer1.FromAccountID, transfer2.FromAccountID)
    require.Equal(t, transfer1.ToAccountID, transfer2.ToAccountID)
    require.Equal(t, transfer1.Amount, transfer2.Amount)
    require.WithinDuration(t, transfer1.CreatedAt, transfer2.CreatedAt, time.Second)
}

func TestListTransfer(t *testing.T) {
    account1 := createRandomAccount(t)
    account2 := createRandomAccount(t)

    for i := 0; i < 5; i++ {
        createRandomTransfer(t, account1, account2)
        createRandomTransfer(t, account2, account1)
    }

    arg := ListTransfersParams{
        FromAccountID: account1.ID,
        ToAccountID:   account1.ID,
        Limit:         5,
        Offset:        5,
    }

    transfers, err := testStore.ListTransfers(context.Background(), arg)
    require.NoError(t, err)
    require.Len(t, transfers, 5)

    for _, transfer := range transfers {
        require.NotEmpty(t, transfer)
        require.True(t, transfer.FromAccountID == account1.ID || transfer.ToAccountID ==
account1.ID)
    }
}
```

FILE: simplebank/db/sqlc/tx_create_user.go

```
package db

import "context"

type CreateUserTxParams struct {
    CreateUserParams
    AfterCreate func(user User) error
}

type CreateUserTxResult struct {
    User User
}

func (store *SQLStore) CreateUserTx(ctx context.Context, arg CreateUserTxParams)
(CreateUserTxResult, error) {
    var result CreateUserTxResult

    err := store.execTx(ctx, func(q *Queries) error {
        var err error

        result.User, err = q.CreateUser(ctx, arg.CreateUserParams)
        if err != nil {
            return err
        }

        return arg.AfterCreate(result.User)
    })

    return result, err
}
```

FILE: simplebank/db/sqlc/tx_transfer.go

```
package db

import "context"

// TransferTxParams contains the input parameters of the transfer transaction
type TransferTxParams struct {
    FromAccountID int64 `json:"from_account_id"`
    ToAccountID   int64 `json:"to_account_id"`
    Amount        int64 `json:"amount"`
}

// TransferTxResult is the result of the transfer transaction
type TransferTxResult struct {
    Transfer    Transfer `json:"transfer"`
    FromAccount Account  `json:"from_account"`
    ToAccount   Account  `json:"to_account"`
    FromEntry   Entry    `json:"from_entry"`
    ToEntry     Entry    `json:"to_entry"`
}

// TransferTx performs a money transfer from one account to the other.
// It creates the transfer, add account entries, and update accounts' balance within a database
transaction
func (store *SQLStore) TransferTx(ctx context.Context, arg TransferTxParams) (TransferTxResult,
error) {
    var result TransferTxResult

    err := store.execTx(ctx, func(q *Queries) error {
        var err error

        result.Transfer, err = q.CreateTransfer(ctx, CreateTransferParams{
            FromAccountID: arg.FromAccountID,
            ToAccountID:   arg.ToAccountID,
            Amount:        arg.Amount,
        })
        if err != nil {
            return err
        }

        result.FromEntry, err = q.CreateEntry(ctx, CreateEntryParams{
            AccountID: arg.FromAccountID,
            Amount:    -arg.Amount,
        })
        if err != nil {
            return err
        }

        result.ToEntry, err = q.CreateEntry(ctx, CreateEntryParams{
            AccountID: arg.ToAccountID,
            Amount:    arg.Amount,
        })
        if err != nil {
            return err
        }

        if arg.FromAccountID < arg.ToAccountID {
            result.FromAccount, result.ToAccount, err = addMoney(ctx, q, arg.FromAccountID,
-arg.Amount, arg.ToAccountID, arg.Amount)
        } else {
            result.ToAccount, result.FromAccount, err = addMoney(ctx, q, arg.ToAccountID,
arg.Amount, arg.FromAccountID, -arg.Amount)
        }

        return err
    })

    return result, err
}

func addMoney(
    ctx context.Context,
    q *Queries,
    accountID1 int64,
    amount1 int64,
    accountID2 int64,
    amount2 int64,
) (account1 Account, account2 Account, err error) {
    account1, err = q.AddAccountBalance(ctx, AddAccountBalanceParams{
        ID:     accountID1,
        Amount: amount1,
    })
    if err != nil {
        return
    }

    account2, err = q.AddAccountBalance(ctx, AddAccountBalanceParams{
        ID:     accountID2,
        Amount: amount2,
    })
    return
}
```

FILE: simplebank/db/sqlc/tx_verify_email.go

```
package db

import (
    "context"

    "github.com/jackc/pgx/v5/pgtype"
)

type VerifyEmailTxParams struct {
    EmailId    int64
    SecretCode string
}

type VerifyEmailTxResult struct {
    User        User
    VerifyEmail VerifyEmail
}

func (store *SQLStore) VerifyEmailTx(ctx context.Context, arg VerifyEmailTxParams)
(VerifyEmailTxResult, error) {
    var result VerifyEmailTxResult

    err := store.execTx(ctx, func(q *Queries) error {
        var err error

        result.VerifyEmail, err = q.UpdateVerifyEmail(ctx, UpdateVerifyEmailParams{
            ID:         arg.EmailId,
            SecretCode: arg.SecretCode,
        })
        if err != nil {
            return err
        }

        result.User, err = q.UpdateUser(ctx, UpdateUserParams{
            Username: result.VerifyEmail.Username,
            IsEmailVerified: pgtype.Bool{
                Bool:  true,
                Valid: true,
            },
        })
        return err
    })

    return result, err
}
```

FILE: simplebank/db/sqlc/user.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: user.sql

package db

import (
    "context"

    "github.com/jackc/pgx/v5/pgtype"
)

const createUser = `-- name: CreateUser :one
INSERT INTO users (
  username,
  hashed_password,
  full_name,
  email
) VALUES (
  $1, $2, $3, $4
) RETURNING username, hashed_password, full_name, email, password_changed_at, created_at,
is_email_verified, role
`

type CreateUserParams struct {
    Username       string `json:"username"`
    HashedPassword string `json:"hashed_password"`
    FullName       string `json:"full_name"`
    Email          string `json:"email"`
}

func (q *Queries) CreateUser(ctx context.Context, arg CreateUserParams) (User, error) {
    row := q.db.QueryRow(ctx, createUser,
        arg.Username,
        arg.HashedPassword,
        arg.FullName,
        arg.Email,
    )
    var i User
    err := row.Scan(
        &i.Username,
        &i.HashedPassword,
        &i.FullName,
        &i.Email,
        &i.PasswordChangedAt,
        &i.CreatedAt,
        &i.IsEmailVerified,
        &i.Role,
    )
    return i, err
}

const getUser = `-- name: GetUser :one
SELECT username, hashed_password, full_name, email, password_changed_at, created_at,
is_email_verified, role FROM users
WHERE username = $1 LIMIT 1
`

func (q *Queries) GetUser(ctx context.Context, username string) (User, error) {
    row := q.db.QueryRow(ctx, getUser, username)
    var i User
    err := row.Scan(
        &i.Username,
        &i.HashedPassword,
        &i.FullName,
        &i.Email,
        &i.PasswordChangedAt,
        &i.CreatedAt,
        &i.IsEmailVerified,
        &i.Role,
    )
    return i, err
}
```

```
const updateUser = `-- name: UpdateUser :one
UPDATE users
SET
  hashed_password = COALESCE($1, hashed_password),
  password_changed_at = COALESCE($2, password_changed_at),
  full_name = COALESCE($3, full_name),
  email = COALESCE($4, email),
  is_email_verified = COALESCE($5, is_email_verified)
WHERE
  username = $6
RETURNING username, hashed_password, full_name, email, password_changed_at, created_at,
is_email_verified, role
`

type UpdateUserParams struct {
    HashedPassword    pgtype.Text        `json:"hashed_password"`
    PasswordChangedAt pgtype.Timestamptz `json:"password_changed_at"`
    FullName          pgtype.Text        `json:"full_name"`
    Email             pgtype.Text        `json:"email"`
    IsEmailVerified   pgtype.Bool        `json:"is_email_verified"`
    Username          string             `json:"username"`
}

func (q *Queries) UpdateUser(ctx context.Context, arg UpdateUserParams) (User, error) {
    row := q.db.QueryRow(ctx, updateUser,
        arg.HashedPassword,
        arg.PasswordChangedAt,
        arg.FullName,
        arg.Email,
        arg.IsEmailVerified,
        arg.Username,
    )
    var i User
    err := row.Scan(
        &i.Username,
        &i.HashedPassword,
        &i.FullName,
        &i.Email,
        &i.PasswordChangedAt,
        &i.CreatedAt,
        &i.IsEmailVerified,
        &i.Role,
    )
    return i, err
}
```

FILE: simplebank/db/sqlc/user_test.go

```
package db

import (
    "context"
    "testing"
    "time"

    "github.com/jackc/pgx/v5/pgtype"
    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func createRandomUser(t *testing.T) User {
    hashedPassword, err := util.HashPassword(util.RandomString(6))
    require.NoError(t, err)

    arg := CreateUserParams{
        Username:       util.RandomOwner(),
        HashedPassword: hashedPassword,
        FullName:       util.RandomOwner(),
        Email:          util.RandomEmail(),
    }

    user, err := testStore.CreateUser(context.Background(), arg)
    require.NoError(t, err)
    require.NotEmpty(t, user)

    require.Equal(t, arg.Username, user.Username)
    require.Equal(t, arg.HashedPassword, user.HashedPassword)
    require.Equal(t, arg.FullName, user.FullName)
    require.Equal(t, arg.Email, user.Email)
    require.True(t, user.PasswordChangedAt.IsZero())
    require.NotZero(t, user.CreatedAt)

    return user
}

func TestCreateUser(t *testing.T) {
    createRandomUser(t)
}

func TestGetUser(t *testing.T) {
    user1 := createRandomUser(t)
    user2, err := testStore.GetUser(context.Background(), user1.Username)
    require.NoError(t, err)
    require.NotEmpty(t, user2)

    require.Equal(t, user1.Username, user2.Username)
    require.Equal(t, user1.HashedPassword, user2.HashedPassword)
    require.Equal(t, user1.FullName, user2.FullName)
    require.Equal(t, user1.Email, user2.Email)
    require.WithinDuration(t, user1.PasswordChangedAt, user2.PasswordChangedAt, time.Second)
    require.WithinDuration(t, user1.CreatedAt, user2.CreatedAt, time.Second)
}

func TestUpdateUserOnlyFullName(t *testing.T) {
    oldUser := createRandomUser(t)

    newFullName := util.RandomOwner()
    updatedUser, err := testStore.UpdateUser(context.Background(), UpdateUserParams{
        Username: oldUser.Username,
        FullName: pgtype.Text{
            String: newFullName,
            Valid:  true,
        },
    })

    require.NoError(t, err)
    require.NotEqual(t, oldUser.FullName, updatedUser.FullName)
    require.Equal(t, newFullName, updatedUser.FullName)
    require.Equal(t, oldUser.Email, updatedUser.Email)
    require.Equal(t, oldUser.HashedPassword, updatedUser.HashedPassword)
}

func TestUpdateUserOnlyEmail(t *testing.T) {
    oldUser := createRandomUser(t)

    newEmail := util.RandomEmail()
    updatedUser, err := testStore.UpdateUser(context.Background(), UpdateUserParams{
        Username: oldUser.Username,
        Email: pgtype.Text{
            String: newEmail,
            Valid:  true,
        },
    })

    require.NoError(t, err)
    require.NotEqual(t, oldUser.Email, updatedUser.Email)
    require.Equal(t, newEmail, updatedUser.Email)
    require.Equal(t, oldUser.FullName, updatedUser.FullName)
    require.Equal(t, oldUser.HashedPassword, updatedUser.HashedPassword)
}

func TestUpdateUserOnlyPassword(t *testing.T) {
    oldUser := createRandomUser(t)

    newPassword := util.RandomString(6)
    newHashedPassword, err := util.HashPassword(newPassword)
    require.NoError(t, err)

    updatedUser, err := testStore.UpdateUser(context.Background(), UpdateUserParams{
        Username: oldUser.Username,
        HashedPassword: pgtype.Text{
            String: newHashedPassword,
            Valid:  true,
        },
    })

    require.NoError(t, err)
    require.NotEqual(t, oldUser.HashedPassword, updatedUser.HashedPassword)
    require.Equal(t, newHashedPassword, updatedUser.HashedPassword)
    require.Equal(t, oldUser.FullName, updatedUser.FullName)
    require.Equal(t, oldUser.Email, updatedUser.Email)
}

func TestUpdateUserAllFields(t *testing.T) {
    oldUser := createRandomUser(t)

    newFullName := util.RandomOwner()
    newEmail := util.RandomEmail()
    newPassword := util.RandomString(6)
    newHashedPassword, err := util.HashPassword(newPassword)
    require.NoError(t, err)

    updatedUser, err := testStore.UpdateUser(context.Background(), UpdateUserParams{
        Username: oldUser.Username,
        FullName: pgtype.Text{
            String: newFullName,
            Valid:  true,
        },
        Email: pgtype.Text{
            String: newEmail,
            Valid:  true,
        },
        HashedPassword: pgtype.Text{
            String: newHashedPassword,
            Valid:  true,
        },
    })

    require.NoError(t, err)
    require.NotEqual(t, oldUser.HashedPassword, updatedUser.HashedPassword)
    require.Equal(t, newHashedPassword, updatedUser.HashedPassword)
    require.NotEqual(t, oldUser.Email, updatedUser.Email)
    require.Equal(t, newEmail, updatedUser.Email)
    require.NotEqual(t, oldUser.FullName, updatedUser.FullName)
    require.Equal(t, newFullName, updatedUser.FullName)
}
```

FILE: simplebank/db/sqlc/verify_email.sql.go

```
// Code generated by sqlc. DO NOT EDIT.
// versions:
//   sqlc v1.22.0
// source: verify_email.sql

package db

import (
    "context"
)

const createVerifyEmail = `-- name: CreateVerifyEmail :one
INSERT INTO verify_emails (
    username,
    email,
    secret_code
) VALUES (
    $1, $2, $3
```

```
) RETURNING id, username, email, secret_code, is_used, created_at, expired_at
`

type CreateVerifyEmailParams struct {
    Username   string `json:"username"`
    Email      string `json:"email"`
    SecretCode string `json:"secret_code"`
}

func (q *Queries) CreateVerifyEmail(ctx context.Context, arg CreateVerifyEmailParams)
(VerifyEmail, error) {
    row := q.db.QueryRow(ctx, createVerifyEmail, arg.Username, arg.Email, arg.SecretCode)
    var i VerifyEmail
    err := row.Scan(
        &i.ID,
        &i.Username,
        &i.Email,
        &i.SecretCode,
        &i.IsUsed,
        &i.CreatedAt,
        &i.ExpiredAt,
    )
    return i, err
}

const updateVerifyEmail = `-- name: UpdateVerifyEmail :one
UPDATE verify_emails
SET
    is_used = TRUE
WHERE
    id = $1
    AND secret_code = $2
    AND is_used = FALSE
    AND expired_at > now()
RETURNING id, username, email, secret_code, is_used, created_at, expired_at
`

type UpdateVerifyEmailParams struct {
    ID         int64  `json:"id"`
    SecretCode string `json:"secret_code"`
}

func (q *Queries) UpdateVerifyEmail(ctx context.Context, arg UpdateVerifyEmailParams)
(VerifyEmail, error) {
    row := q.db.QueryRow(ctx, updateVerifyEmail, arg.ID, arg.SecretCode)
    var i VerifyEmail
    err := row.Scan(
        &i.ID,
        &i.Username,
        &i.Email,
        &i.SecretCode,
        &i.IsUsed,
        &i.CreatedAt,
        &i.ExpiredAt,
    )
    return i, err
}
```

FILE: simplebank/doc/statik/statik.go

```
// Code generated by statik. DO NOT EDIT.

package statik

import (
    "github.com/rakyll/statik/fs"
)


func init() {
    data := "PK\x03\x04\x14\x00\x08\x00\x08\x00\x90@1W\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\
x00\x00\x11\x00
\x00favicon-16x16.pngUT\x05\x00\x01\x11\xb3\x06e\x00\x99\x02f\xfd\x89PNG\x0d\n\x1a\n\x ...
[LINE TRUNCATED]
        fs.Register(data)
    }
```

FILE: simplebank/gapi/authorization.go

```
package gapi

import (
    "context"
    "fmt"
    "strings"

    "github.com/techschool/simplebank/token"
    "google.golang.org/grpc/metadata"
)

const (
    authorizationHeader = "authorization"
    authorizationBearer = "bearer"
)

func (server *Server) authorizeUser(ctx context.Context, accessibleRoles []string)
(*token.Payload, error) {
    md, ok := metadata.FromIncomingContext(ctx)
    if !ok {
        return nil, fmt.Errorf("missing metadata")
    }

    values := md.Get(authorizationHeader)
    if len(values) == 0 {
        return nil, fmt.Errorf("missing authorization header")
    }

    authHeader := values[0]
    fields := strings.Fields(authHeader)
    if len(fields) < 2 {
        return nil, fmt.Errorf("invalid authorization header format")
    }

    authType := strings.ToLower(fields[0])
    if authType != authorizationBearer {
        return nil, fmt.Errorf("unsupported authorization type: %s", authType)
    }

    accessToken := fields[1]
    payload, err := server.tokenMaker.VerifyToken(accessToken, token.TokenTypeAccessToken)
    if err != nil {
        return nil, fmt.Errorf("invalid access token: %s", err)
    }

    if !hasPermission(payload.Role, accessibleRoles) {
        return nil, fmt.Errorf("permission denied")
    }

    return payload, nil
}

func hasPermission(userRole string, accessibleRoles []string) bool {
    for _, role := range accessibleRoles {
        if userRole == role {
            return true
        }
    }
    return false
}
```

FILE: simplebank/gapi/converter.go

```
package gapi

import (
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "google.golang.org/protobuf/types/known/timestamppb"
)

func convertUser(user db.User) *pb.User {
    return &pb.User{
        Username:          user.Username,
        FullName:          user.FullName,
        Email:             user.Email,
        PasswordChangedAt: timestamppb.New(user.PasswordChangedAt),
        CreatedAt:         timestamppb.New(user.CreatedAt),
    }
}
```

FILE: simplebank/gapi/error.go

```
package gapi

import (
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func fieldViolation(field string, err error) *errdetails.BadRequest_FieldViolation {
    return &errdetails.BadRequest_FieldViolation{
        Field:       field,
        Description: err.Error(),
    }
}

func invalidArgumentError(violations []*errdetails.BadRequest_FieldViolation) error {
    badRequest := &errdetails.BadRequest{FieldViolations: violations}
    statusInvalid := status.New(codes.InvalidArgument, "invalid parameters")

    statusDetails, err := statusInvalid.WithDetails(badRequest)
    if err != nil {
        return statusInvalid.Err()
    }

    return statusDetails.Err()
}

func unauthenticatedError(err error) error {
    return status.Errorf(codes.Unauthenticated, "unauthorized: %s", err)
}
```

FILE: simplebank/gapi/logger.go

```
package gapi

import (
    "context"
    "net/http"
    "time"

    "github.com/rs/zerolog/log"
    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func GrpcLogger(
    ctx context.Context,
    req interface{},
    info *grpc.UnaryServerInfo,
    handler grpc.UnaryHandler,
) (resp interface{}, err error) {
    startTime := time.Now()
    result, err := handler(ctx, req)
    duration := time.Since(startTime)

    statusCode := codes.Unknown
    if st, ok := status.FromError(err); ok {
        statusCode = st.Code()
```

```
    }

    logger := log.Info()
    if err != nil {
        logger = log.Error().Err(err)
    }

    logger.Str("protocol", "grpc").
        Str("method", info.FullMethod).
        Int("status_code", int(statusCode)).
        Str("status_text", statusCode.String()).
        Dur("duration", duration).
        Msg("received a gRPC request")

    return result, err
}

type ResponseRecorder struct {
    http.ResponseWriter
    StatusCode int
    Body       []byte
}

func (rec *ResponseRecorder) WriteHeader(statusCode int) {
    rec.StatusCode = statusCode
    rec.ResponseWriter.WriteHeader(statusCode)
}

func (rec *ResponseRecorder) Write(body []byte) (int, error) {
    rec.Body = body
    return rec.ResponseWriter.Write(body)
}

func HttpLogger(handler http.Handler) http.Handler {
    return http.HandlerFunc(func(res http.ResponseWriter, req *http.Request) {
        startTime := time.Now()
        rec := &ResponseRecorder{
            ResponseWriter: res,
            StatusCode:     http.StatusOK,
        }
        handler.ServeHTTP(rec, req)
        duration := time.Since(startTime)

        logger := log.Info()
        if rec.StatusCode != http.StatusOK {
            logger = log.Error().Bytes("body", rec.Body)
        }

        logger.Str("protocol", "http").
            Str("method", req.Method).
            Str("path", req.RequestURI).
            Int("status_code", rec.StatusCode).
            Str("status_text", http.StatusText(rec.StatusCode)).
            Dur("duration", duration).
            Msg("received a HTTP request")
    })
}
```

FILE: simplebank/gapi/main_test.go

```
package gapi

import (
    "context"
    "fmt"
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/worker"
    "google.golang.org/grpc/metadata"
)

func newTestServer(t *testing.T, store db.Store, taskDistributor worker.TaskDistributor)
*Server {
    config := util.Config{
        TokenSymmetricKey:   util.RandomString(32),
        AccessTokenDuration: time.Minute,
    }

    server, err := NewServer(config, store, taskDistributor)
    require.NoError(t, err)

    return server
}

func newContextWithBearerToken(t *testing.T, tokenMaker token.Maker, username string, role
string, duration time.Duration, tokenType token.TokenType) context.Context {
    accessToken, _, err := tokenMaker.CreateToken(username, role, duration, tokenType)
    require.NoError(t, err)

    bearerToken := fmt.Sprintf("%s %s", authorizationBearer, accessToken)
    md := metadata.MD{
        authorizationHeader: []string{
            bearerToken,
        },
    }

    return metadata.NewIncomingContext(context.Background(), md)
}
```

FILE: simplebank/gapi/metadata.go

```
package gapi

import (
    "context"
    "google.golang.org/grpc/metadata"
    "google.golang.org/grpc/peer"
)

const (
    grpcGatewayUserAgentHeader = "grpcgateway-user-agent"
    userAgentHeader            = "user-agent"
    xForwardedForHeader        = "x-forwarded-for"
)

type Metadata struct {
    UserAgent string
    ClientIP  string
}

func (server *Server) extractMetadata(ctx context.Context) *Metadata {
    mtdt := &Metadata{}

    if md, ok := metadata.FromIncomingContext(ctx); ok {
        if userAgents := md.Get(grpcGatewayUserAgentHeader); len(userAgents) > 0 {
            mtdt.UserAgent = userAgents[0]
        }

        if userAgents := md.Get(userAgentHeader); len(userAgents) > 0 {
            mtdt.UserAgent = userAgents[0]
        }

        if clientIPs := md.Get(xForwardedForHeader); len(clientIPs) > 0 {
            mtdt.ClientIP = clientIPs[0]
        }
    }

    if p, ok := peer.FromContext(ctx); ok {
        mtdt.ClientIP = p.Addr.String()
    }

    return mtdt
}
```

FILE: simplebank/gapi/rpc_create_user.go

```
package gapi

import (
    "context"
    "time"

    "github.com/hibiken/asynq"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/val"
    "github.com/techschool/simplebank/worker"
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func (server *Server) CreateUser(ctx context.Context, req *pb.CreateUserRequest)
(*pb.CreateUserResponse, error) {
    violations := validateCreateUserRequest(req)
    if violations != nil {
        return nil, invalidArgumentError(violations)
    }

    hashedPassword, err := util.HashPassword(req.GetPassword())
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to hash password: %s", err)
    }

    arg := db.CreateUserTxParams{
        CreateUserParams: db.CreateUserParams{
            Username:       req.GetUsername(),
            HashedPassword: hashedPassword,
            FullName:       req.GetFullName(),
            Email:          req.GetEmail(),
        },
        AfterCreate: func(user db.User) error {
            taskPayload := &worker.PayloadSendVerifyEmail{
                Username: user.Username,
            }
            opts := []asynq.Option{
                asynq.MaxRetry(10),
                asynq.ProcessIn(10 * time.Second),
                asynq.Queue(worker.QueueCritical),
            }

            return server.taskDistributor.DistributeTaskSendVerifyEmail(ctx, taskPayload,
opts...)
        },
    }

    txResult, err := server.store.CreateUserTx(ctx, arg)
    if err != nil {
        if db.ErrorCode(err) == db.UniqueViolation {
            return nil, status.Error(codes.AlreadyExists, err.Error())
        }
        return nil, status.Errorf(codes.Internal, "failed to create user: %s", err)
    }

    rsp := &pb.CreateUserResponse{
        User: convertUser(txResult.User),
    }
    return rsp, nil
}

func validateCreateUserRequest(req *pb.CreateUserRequest) (violations
[]*errdetails.BadRequest_FieldViolation) {
```

```
    if err := val.ValidateUsername(req.GetUsername()); err != nil {
        violations = append(violations, fieldViolation("username", err))
    }

    if err := val.ValidatePassword(req.GetPassword()); err != nil {
        violations = append(violations, fieldViolation("password", err))
    }

    if err := val.ValidateFullName(req.GetFullName()); err != nil {
        violations = append(violations, fieldViolation("full_name", err))
    }

    if err := val.ValidateEmail(req.GetEmail()); err != nil {
        violations = append(violations, fieldViolation("email", err))
    }

    return violations
}
```

FILE: simplebank/gapi/rpc_create_user_test.go

```
package gapi

import (
    "context"
    "database/sql"
    "fmt"
    "reflect"
    "testing"

    "github.com/golang/mock/gomock"
    "github.com/stretchr/testify/require"
    mockdb "github.com/techschool/simplebank/db/mock"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/worker"
    mockwk "github.com/techschool/simplebank/worker/mock"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

type eqCreateUserTxParamsMatcher struct {
    arg      db.CreateUserTxParams
    password string
    user     db.User
}

func (expected eqCreateUserTxParamsMatcher) Matches(x interface{}) bool {
    actualArg, ok := x.(db.CreateUserTxParams)
    if !ok {
        return false
    }

    err := util.CheckPassword(expected.password, actualArg.HashedPassword)
    if err != nil {
        return false
    }

    expected.arg.HashedPassword = actualArg.HashedPassword
    if !reflect.DeepEqual(expected.arg.CreateUserParams, actualArg.CreateUserParams) {
        return false
    }

    err = actualArg.AfterCreate(expected.user)
    return err == nil
}

func (e eqCreateUserTxParamsMatcher) String() string {
    return fmt.Sprintf("matches arg %v and password %v", e.arg, e.password)
}

func EqCreateUserTxParams(arg db.CreateUserTxParams, password string, user db.User)
gomock.Matcher {
    return eqCreateUserTxParamsMatcher{arg, password, user}
}

func randomUser(t *testing.T, role string) (user db.User, password string) {
    password = util.RandomString(6)
    hashedPassword, err := util.HashPassword(password)
    require.NoError(t, err)

    user = db.User{
        Username:       util.RandomOwner(),
        Role:           role,
        HashedPassword: hashedPassword,
        FullName:       util.RandomOwner(),
        Email:          util.RandomEmail(),
    }
    return
}

func TestCreateUserAPI(t *testing.T) {
    user, password := randomUser(t, util.DepositorRole)

    testCases := []struct {
        name          string
        req           *pb.CreateUserRequest
        buildStubs    func(store *mockdb.MockStore, taskDistributor
*mockwk.MockTaskDistributor)
        checkResponse func(t *testing.T, res *pb.CreateUserResponse, err error)
    }{
        {
            name: "OK",
            req: &pb.CreateUserRequest{
                Username: user.Username,
                Password: password,
                FullName: user.FullName,
                Email:    user.Email,
            },
            buildStubs: func(store *mockdb.MockStore, taskDistributor
*mockwk.MockTaskDistributor) {
                arg := db.CreateUserTxParams{
                    CreateUserParams: db.CreateUserParams{
                        Username: user.Username,
                        FullName: user.FullName,
                        Email:    user.Email,
                    },
                }
                store.EXPECT().
                    CreateUserTx(gomock.Any(), EqCreateUserTxParams(arg, password, user)).
                    Times(1).
                    Return(db.CreateUserTxResult{User: user}, nil)

                taskPayload := &worker.PayloadSendVerifyEmail{
                    Username: user.Username,
                }
                taskDistributor.EXPECT().
                    DistributeTaskSendVerifyEmail(gomock.Any(), taskPayload, gomock.Any()).
                    Times(1).
                    Return(nil)
            },
            checkResponse: func(t *testing.T, res *pb.CreateUserResponse, err error) {
                require.NoError(t, err)
                require.NotNil(t, res)
                createdUser := res.GetUser()
                require.Equal(t, user.Username, createdUser.Username)
                require.Equal(t, user.FullName, createdUser.FullName)
                require.Equal(t, user.Email, createdUser.Email)
            },
        },
        {
            name: "InternalError",
            req: &pb.CreateUserRequest{
                Username: user.Username,
                Password: password,
                FullName: user.FullName,
                Email:    user.Email,
            },
            buildStubs: func(store *mockdb.MockStore, taskDistributor
*mockwk.MockTaskDistributor) {
                store.EXPECT().
                    CreateUserTx(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.CreateUserTxResult{}, sql.ErrConnDone)

                taskDistributor.EXPECT().
                    DistributeTaskSendVerifyEmail(gomock.Any(), gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(t *testing.T, res *pb.CreateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.Internal, st.Code())
            },
        },
        {
            name: "DuplicateUsername",
            req: &pb.CreateUserRequest{
                Username: user.Username,
                Password: password,
                FullName: user.FullName,
                Email:    user.Email,
            },
            buildStubs: func(store *mockdb.MockStore, taskDistributor
*mockwk.MockTaskDistributor) {
                store.EXPECT().
                    CreateUserTx(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.CreateUserTxResult{}, db.ErrUniqueViolation)

                taskDistributor.EXPECT().
                    DistributeTaskSendVerifyEmail(gomock.Any(), gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(t *testing.T, res *pb.CreateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.AlreadyExists, st.Code())
            },
        },
        {
            name: "InvalidEmail",
            req: &pb.CreateUserRequest{
                Username: user.Username,
                Password: password,
                FullName: user.FullName,
                Email:    "invalid-email",
            },
            buildStubs: func(store *mockdb.MockStore, taskDistributor
*mockwk.MockTaskDistributor) {
                store.EXPECT().
                    CreateUserTx(gomock.Any(), gomock.Any()).
                    Times(0)

                taskDistributor.EXPECT().
                    DistributeTaskSendVerifyEmail(gomock.Any(), gomock.Any(), gomock.Any()).
                    Times(0)
            },
            checkResponse: func(t *testing.T, res *pb.CreateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.InvalidArgument, st.Code())
            },
```

```
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            storeCtrl := gomock.NewController(t)
            defer storeCtrl.Finish()
            store := mockdb.NewMockStore(storeCtrl)

            taskCtrl := gomock.NewController(t)
            defer taskCtrl.Finish()
            taskDistributor := mockwk.NewMockTaskDistributor(taskCtrl)

            tc.buildStubs(store, taskDistributor)
            server := newTestServer(t, store, taskDistributor)

            res, err := server.CreateUser(context.Background(), tc.req)
            tc.checkResponse(t, res, err)
        })
    }
}
```

FILE: simplebank/gapi/rpc_login_user.go

```
package gapi

import (
    "context"
    "errors"

    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/val"
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
    "google.golang.org/protobuf/types/known/timestamppb"
)

func (server *Server) LoginUser(ctx context.Context, req *pb.LoginUserRequest)
(*pb.LoginUserResponse, error) {
    violations := validateLoginUserRequest(req)
    if violations != nil {
        return nil, invalidArgumentError(violations)
    }

    user, err := server.store.GetUser(ctx, req.GetUsername())
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            return nil, status.Errorf(codes.NotFound, "user not found")
        }
        return nil, status.Errorf(codes.Internal, "failed to find user")
    }

    err = util.CheckPassword(req.Password, user.HashedPassword)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "incorrect password")
    }

    accessToken, accessPayload, err := server.tokenMaker.CreateToken(
        user.Username,
        user.Role,
        server.config.AccessTokenDuration,
        token.TokenTypeAccessToken,
    )
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to create access token")
    }

    refreshToken, refreshPayload, err := server.tokenMaker.CreateToken(
        user.Username,
        user.Role,
        server.config.RefreshTokenDuration,
        token.TokenTypeRefreshToken,
    )
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to create refresh token")
    }

    mtdt := server.extractMetadata(ctx)
    session, err := server.store.CreateSession(ctx, db.CreateSessionParams{
        ID:           refreshPayload.ID,
        Username:     user.Username,
        RefreshToken: refreshToken,
        UserAgent:    mtdt.UserAgent,
        ClientIp:     mtdt.ClientIP,
        IsBlocked:    false,
        ExpiresAt:    refreshPayload.ExpiredAt,
    })
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to create session")
    }

    rsp := &pb.LoginUserResponse{
        User:                  convertUser(user),
        SessionId:             session.ID.String(),
        AccessToken:           accessToken,
        RefreshToken:          refreshToken,
        AccessTokenExpiresAt:  timestamppb.New(accessPayload.ExpiredAt),
        RefreshTokenExpiresAt: timestamppb.New(refreshPayload.ExpiredAt),
    }
    return rsp, nil
}

func validateLoginUserRequest(req *pb.LoginUserRequest) (violations
[]*errdetails.BadRequest_FieldViolation) {
    if err := val.ValidateUsername(req.GetUsername()); err != nil {
        violations = append(violations, fieldViolation("username", err))
    }

    if err := val.ValidatePassword(req.GetPassword()); err != nil {
        violations = append(violations, fieldViolation("password", err))
    }

    return violations
}
```

FILE: simplebank/gapi/rpc_update_user.go

```
package gapi

import (
    "context"
    "errors"
    "time"

    "github.com/jackc/pgx/v5/pgtype"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/val"
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func (server *Server) UpdateUser(ctx context.Context, req *pb.UpdateUserRequest)
(*pb.UpdateUserResponse, error) {
    authPayload, err := server.authorizeUser(ctx, []string{util.BankerRole,
util.DepositorRole})
    if err != nil {
        return nil, unauthenticatedError(err)
    }

    violations := validateUpdateUserRequest(req)
    if violations != nil {
        return nil, invalidArgumentError(violations)
    }

    if authPayload.Role != util.BankerRole && authPayload.Username != req.GetUsername() {
        return nil, status.Errorf(codes.PermissionDenied, "cannot update other user's info")
    }

    arg := db.UpdateUserParams{
        Username: req.GetUsername(),
        FullName: pgtype.Text{
            String: req.GetFullName(),
            Valid:  req.FullName != nil,
        },
        Email: pgtype.Text{
            String: req.GetEmail(),
            Valid:  req.Email != nil,
        },
    }

    if req.Password != nil {
        hashedPassword, err := util.HashPassword(req.GetPassword())
        if err != nil {
            return nil, status.Errorf(codes.Internal, "failed to hash password: %s", err)
        }

        arg.HashedPassword = pgtype.Text{
            String: hashedPassword,
            Valid:  true,
        }

        arg.PasswordChangedAt = pgtype.Timestamptz{
            Time:  time.Now(),
            Valid: true,
        }
    }

    user, err := server.store.UpdateUser(ctx, arg)
    if err != nil {
        if errors.Is(err, db.ErrRecordNotFound) {
            return nil, status.Errorf(codes.NotFound, "user not found")
        }
        return nil, status.Errorf(codes.Internal, "failed to update user: %s", err)
    }

    rsp := &pb.UpdateUserResponse{
        User: convertUser(user),
    }
    return rsp, nil
}

func validateUpdateUserRequest(req *pb.UpdateUserRequest) (violations
[]*errdetails.BadRequest_FieldViolation) {
    if err := val.ValidateUsername(req.GetUsername()); err != nil {
        violations = append(violations, fieldViolation("username", err))
    }

    if req.Password != nil {
        if err := val.ValidatePassword(req.GetPassword()); err != nil {
            violations = append(violations, fieldViolation("password", err))
        }
    }

    if req.FullName != nil {
        if err := val.ValidateFullName(req.GetFullName()); err != nil {
            violations = append(violations, fieldViolation("full_name", err))
        }
    }
```

```
    if req.Email != nil {
        if err := val.ValidateEmail(req.GetEmail()); err != nil {
            violations = append(violations, fieldViolation("email", err))
        }
    }

    return violations
}
```

FILE: simplebank/gapi/rpc_update_user_test.go

```
package gapi

import (
    "context"
    "testing"
    "time"

    "github.com/golang/mock/gomock"
    "github.com/jackc/pgx/v5/pgtype"
    "github.com/stretchr/testify/require"
    mockdb "github.com/techschool/simplebank/db/mock"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func TestUpdateUserAPI(t *testing.T) {
    user, _ := randomUser(t, util.DepositorRole)
    other, _ := randomUser(t, util.DepositorRole)
    banker, _ := randomUser(t, util.BankerRole)

    newName := util.RandomOwner()
    newEmail := util.RandomEmail()
    invalidEmail := "invalid-email"

    testCases := []struct {
        name          string
        req           *pb.UpdateUserRequest
        buildStubs    func(store *mockdb.MockStore)
        buildContext  func(t *testing.T, tokenMaker token.Maker) context.Context
        checkResponse func(t *testing.T, res *pb.UpdateUserResponse, err error)
    }{
        {
            name: "OK",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                arg := db.UpdateUserParams{
                    Username: user.Username,
                    FullName: pgtype.Text{
                        String: newName,
                        Valid:  true,
                    },
                    Email: pgtype.Text{
                        String: newEmail,
                        Valid:  true,
                    },
                }
                updatedUser := db.User{
                    Username:          user.Username,
                    HashedPassword:    user.HashedPassword,
                    FullName:          newName,
                    Email:             newEmail,
                    PasswordChangedAt: user.PasswordChangedAt,
                    CreatedAt:         user.CreatedAt,
                    IsEmailVerified:   user.IsEmailVerified,
                }
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Eq(arg)).
                    Times(1).
                    Return(updatedUser, nil)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, user.Username, user.Role,
time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.NoError(t, err)
                require.NotNil(t, res)
                updatedUser := res.GetUser()
                require.Equal(t, user.Username, updatedUser.Username)
                require.Equal(t, newName, updatedUser.FullName)
                require.Equal(t, newEmail, updatedUser.Email)
            },
        },
        {
            name: "BankerCanUpdateUserInfo",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                arg := db.UpdateUserParams{
                    Username: user.Username,
                    FullName: pgtype.Text{
                        String: newName,
                        Valid:  true,
                    },
                    Email: pgtype.Text{
                        String: newEmail,
                        Valid:  true,
                    },
                }
                updatedUser := db.User{
                    Username:          user.Username,
                    HashedPassword:    user.HashedPassword,
                    FullName:          newName,
                    Email:             newEmail,
                    PasswordChangedAt: user.PasswordChangedAt,
                    CreatedAt:         user.CreatedAt,
                    IsEmailVerified:   user.IsEmailVerified,
                }
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Eq(arg)).
                    Times(1).
                    Return(updatedUser, nil)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, banker.Username, banker.Role,
time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.NoError(t, err)
                require.NotNil(t, res)
                updatedUser := res.GetUser()
                require.Equal(t, user.Username, updatedUser.Username)
                require.Equal(t, newName, updatedUser.FullName)
                require.Equal(t, newEmail, updatedUser.Email)
            },
        },
        {
            name: "OtherDepositorCannotUpdateThisUserInfo",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, other.Username, other.Role,
time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.PermissionDenied, st.Code())
            },
        },
        {
            name: "UserNotFound",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(1).
                    Return(db.User{}, db.ErrRecordNotFound)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, user.Username, user.Role,
time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.NotFound, st.Code())
            },
        },
        {
            name: "InvalidEmail",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &invalidEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, user.Username, user.Role,
time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.InvalidArgument, st.Code())
            },
        },
        {
            name: "ExpiredToken",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
```

```
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, user.Username, user.Role,
-time.Minute, token.TokenTypeAccessToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.Unauthenticated, st.Code())
            },
        },
        {
            name: "WrongTokenType",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return newContextWithBearerToken(t, tokenMaker, user.Username, user.Role,
time.Minute, token.TokenTypeRefreshToken)
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.Unauthenticated, st.Code())
            },
        },
        {
            name: "NoAuthorization",
            req: &pb.UpdateUserRequest{
                Username: user.Username,
                FullName: &newName,
                Email:    &newEmail,
            },
            buildStubs: func(store *mockdb.MockStore) {
                store.EXPECT().
                    UpdateUser(gomock.Any(), gomock.Any()).
                    Times(0)
            },
            buildContext: func(t *testing.T, tokenMaker token.Maker) context.Context {
                return context.Background()
            },
            checkResponse: func(t *testing.T, res *pb.UpdateUserResponse, err error) {
                require.Error(t, err)
                st, ok := status.FromError(err)
                require.True(t, ok)
                require.Equal(t, codes.Unauthenticated, st.Code())
            },
        },
    }

    for i := range testCases {
        tc := testCases[i]

        t.Run(tc.name, func(t *testing.T) {
            storeCtrl := gomock.NewController(t)
            defer storeCtrl.Finish()
            store := mockdb.NewMockStore(storeCtrl)

            tc.buildStubs(store)
            server := newTestServer(t, store, nil)

            ctx := tc.buildContext(t, server.tokenMaker)
            res, err := server.UpdateUser(ctx, tc.req)
            tc.checkResponse(t, res, err)
        })
    }
}
```

FILE: simplebank/gapi/rpc_verify_email.go

```
package gapi

import (
    "context"

    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/val"
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
)

func (server *Server) VerifyEmail(ctx context.Context, req *pb.VerifyEmailRequest)
(*pb.VerifyEmailResponse, error) {
    violations := validateVerifyEmailRequest(req)
    if violations != nil {
        return nil, invalidArgumentError(violations)
    }

    txResult, err := server.store.VerifyEmailTx(ctx, db.VerifyEmailTxParams{
        EmailId:    req.GetEmailId(),
        SecretCode: req.GetSecretCode(),
    })
    if err != nil {
        return nil, status.Errorf(codes.Internal, "failed to verify email")
    }

    rsp := &pb.VerifyEmailResponse{
        IsVerified: txResult.User.IsEmailVerified,
    }
    return rsp, nil
}

func validateVerifyEmailRequest(req *pb.VerifyEmailRequest) (violations
[]*errdetails.BadRequest_FieldViolation) {
    if err := val.ValidateEmailId(req.GetEmailId()); err != nil {
        violations = append(violations, fieldViolation("email_id", err))
    }

    if err := val.ValidateSecretCode(req.GetSecretCode()); err != nil {
        violations = append(violations, fieldViolation("secret_code", err))
    }

    return violations
}
```

FILE: simplebank/gapi/server.go

```
package gapi

import (
    "fmt"

    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/token"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/worker"
)

// Server serves gRPC requests for our banking service.
type Server struct {
    pb.UnimplementedSimpleBankServer
    config          util.Config
    store           db.Store
    tokenMaker      token.Maker
    taskDistributor worker.TaskDistributor
}

// NewServer creates a new gRPC server.
func NewServer(config util.Config, store db.Store, taskDistributor worker.TaskDistributor)
(*Server, error) {
    tokenMaker, err := token.NewPasetoMaker(config.TokenSymmetricKey)
    if err != nil {
        return nil, fmt.Errorf("cannot create token maker: %w", err)
    }

    server := &Server{
        config:          config,
        store:           store,
        tokenMaker:      tokenMaker,
        taskDistributor: taskDistributor,
    }

    return server, nil
}
```

FILE: simplebank/mail/sender.go

```
package mail

import (
    "fmt"
    "net/smtp"

    "github.com/jordan-wright/email"
)

const (
    smtpAuthAddress   = "smtp.gmail.com"
    smtpServerAddress = "smtp.gmail.com:587"
)

type EmailSender interface {
    SendEmail(
        subject string,
        content string,
        to []string,
        cc []string,
        bcc []string,
        attachFiles []string,
    ) error
}

type GmailSender struct {
    name              string
    fromEmailAddress  string
    fromEmailPassword string
}

func NewGmailSender(name string, fromEmailAddress string, fromEmailPassword string) EmailSender
{
    return &GmailSender{
        name:              name,
        fromEmailAddress:  fromEmailAddress,
        fromEmailPassword: fromEmailPassword,
    }
}

func (sender *GmailSender) SendEmail(
    subject string,
    content string,
    to []string,
    cc []string,
    bcc []string,
    attachFiles []string,
) error {
```

```
    e := email.NewEmail()
    e.From = fmt.Sprintf("%s <%s>", sender.name, sender.fromEmailAddress)
    e.Subject = subject
    e.HTML = []byte(content)
    e.To = to
    e.Cc = cc
    e.Bcc = bcc

    for _, f := range attachFiles {
        _, err := e.AttachFile(f)
        if err != nil {
            return fmt.Errorf("failed to attach file %s: %w", f, err)
        }
    }

    smtpAuth := smtp.PlainAuth("", sender.fromEmailAddress, sender.fromEmailPassword,
smtpAuthAddress)
    return e.Send(smtpServerAddress, smtpAuth)
}
```

FILE: simplebank/mail/sender_test.go

```
package mail

import (
    "testing"

    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func TestSendEmailWithGmail(t *testing.T) {
    if testing.Short() {
        t.Skip()
    }

    config, err := util.LoadConfig("..")
    require.NoError(t, err)

    sender := NewGmailSender(config.EmailSenderName, config.EmailSenderAddress,
config.EmailSenderPassword)

    subject := "A test email"
    content := `
    <h1>Hello world</h1>
    <p>This is a test message from <a href="http://techschool.guru">Tech School</a></p>
    `
    to := []string{"techschool.guru@gmail.com"}
    attachFiles := []string{"../README.md"}

    err = sender.SendEmail(subject, content, to, nil, nil, attachFiles)
    require.NoError(t, err)
}
```

FILE: simplebank/main.go

```
package main

import (
    "context"
    "errors"
    "net"
    "net/http"
    "os"
    "os/signal"
    "syscall"

    "github.com/golang-migrate/migrate/v4"
    _ "github.com/golang-migrate/migrate/v4/database/postgres"
    _ "github.com/golang-migrate/migrate/v4/source/file"
    "github.com/grpc-ecosystem/grpc-gateway/v2/runtime"
    "github.com/hibiken/asynq"
    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/rakyll/statik/fs"
    "github.com/rs/cors"
    "github.com/rs/zerolog"
    "github.com/rs/zerolog/log"
    "github.com/techschool/simplebank/api"
    db "github.com/techschool/simplebank/db/sqlc"
    _ "github.com/techschool/simplebank/doc/statik"
    "github.com/techschool/simplebank/gapi"
    "github.com/techschool/simplebank/mail"
    "github.com/techschool/simplebank/pb"
    "github.com/techschool/simplebank/util"
    "github.com/techschool/simplebank/worker"
    "golang.org/x/sync/errgroup"
    "google.golang.org/grpc"
    "google.golang.org/grpc/reflection"
    "google.golang.org/protobuf/encoding/protojson"
)

var interruptSignals = []os.Signal{
    os.Interrupt,
    syscall.SIGTERM,
    syscall.SIGINT,
}

func main() {
    config, err := util.LoadConfig(".")
    if err != nil {
        log.Fatal().Err(err).Msg("cannot load config")
    }

    if config.Environment == "development" {
        log.Logger = log.Output(zerolog.ConsoleWriter{Out: os.Stderr})
    }

    ctx, stop := signal.NotifyContext(context.Background(), interruptSignals...)
    defer stop()

    connPool, err := pgxpool.New(ctx, config.DBSource)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot connect to db")
    }

    runDBMigration(config.MigrationURL, config.DBSource)

    store := db.NewStore(connPool)

    redisOpt := asynq.RedisClientOpt{
        Addr: config.RedisAddress,
    }

    taskDistributor := worker.NewRedisTaskDistributor(redisOpt)

    waitGroup, ctx := errgroup.WithContext(ctx)

    runTaskProcessor(ctx, waitGroup, config, redisOpt, store)
    runGatewayServer(ctx, waitGroup, config, store, taskDistributor)
    runGrpcServer(ctx, waitGroup, config, store, taskDistributor)

    err = waitGroup.Wait()
    if err != nil {
        log.Fatal().Err(err).Msg("error from wait group")
    }
}

func runDBMigration(migrationURL string, dbSource string) {
    migration, err := migrate.New(migrationURL, dbSource)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create new migrate instance")
    }

    if err = migration.Up(); err != nil && err != migrate.ErrNoChange {
        log.Fatal().Err(err).Msg("failed to run migrate up")
    }

    log.Info().Msg("db migrated successfully")
}

func runTaskProcessor(
    ctx context.Context,
    waitGroup *errgroup.Group,
    config util.Config,
    redisOpt asynq.RedisClientOpt,
    store db.Store,
) {
    mailer := mail.NewGmailSender(config.EmailSenderName, config.EmailSenderAddress,
config.EmailSenderPassword)
    taskProcessor := worker.NewRedisTaskProcessor(redisOpt, store, mailer)

    log.Info().Msg("start task processor")
    err := taskProcessor.Start()
    if err != nil {
        log.Fatal().Err(err).Msg("failed to start task processor")
    }

    waitGroup.Go(func() error {
        <-ctx.Done()
        log.Info().Msg("graceful shutdown task processor")

        taskProcessor.Shutdown()
        log.Info().Msg("task processor is stopped")

        return nil
    })
}

func runGrpcServer(
    ctx context.Context,
    waitGroup *errgroup.Group,
    config util.Config,
    store db.Store,
    taskDistributor worker.TaskDistributor,
) {
    server, err := gapi.NewServer(config, store, taskDistributor)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create server")
    }

    gprcLogger := grpc.UnaryInterceptor(gapi.GrpcLogger)
    grpcServer := grpc.NewServer(gprcLogger)
    pb.RegisterSimpleBankServer(grpcServer, server)
    reflection.Register(grpcServer)

    listener, err := net.Listen("tcp", config.GRPCServerAddress)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create listener")
    }

    waitGroup.Go(func() error {
        log.Info().Msgf("start gRPC server at %s", listener.Addr().String())

        err = grpcServer.Serve(listener)
        if err != nil {
            if errors.Is(err, grpc.ErrServerStopped) {
                return nil
            }
            log.Error().Err(err).Msg("gRPC server failed to serve")
            return err
        }

        return nil
    })

    waitGroup.Go(func() error {
        <-ctx.Done()
        log.Info().Msg("graceful shutdown gRPC server")
```

```
        grpcServer.GracefulStop()
        log.Info().Msg("gRPC server is stopped")

        return nil
    })
}

func runGatewayServer(
    ctx context.Context,
    waitGroup *errgroup.Group,
    config util.Config,
    store db.Store,
    taskDistributor worker.TaskDistributor,
) {
    server, err := gapi.NewServer(config, store, taskDistributor)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create server")
    }

    jsonOption := runtime.WithMarshalerOption(runtime.MIMEWildcard, &runtime.JSONPb{
        MarshalOptions: protojson.MarshalOptions{
            UseProtoNames: true,
        },
        UnmarshalOptions: protojson.UnmarshalOptions{
            DiscardUnknown: true,
        },
    })

    grpcMux := runtime.NewServeMux(jsonOption)

    err = pb.RegisterSimpleBankHandlerServer(ctx, grpcMux, server)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot register handler server")
    }

    mux := http.NewServeMux()
    mux.Handle("/", grpcMux)

    statikFS, err := fs.New()
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create statik fs")
    }

    swaggerHandler := http.StripPrefix("/swagger/", http.FileServer(statikFS))
    mux.Handle("/swagger/", swaggerHandler)

    c := cors.New(cors.Options{
        AllowedOrigins: config.AllowedOrigins,
        AllowedMethods: []string{
            http.MethodHead,
            http.MethodOptions,
            http.MethodGet,
            http.MethodPost,
            http.MethodPut,
            http.MethodPatch,
            http.MethodDelete,
        },
        AllowedHeaders: []string{
            "Content-Type",
            "Authorization",
        },
        AllowCredentials: true,
    })
    handler := c.Handler(gapi.HttpLogger(mux))

    httpServer := &http.Server{
        Handler: handler,
        Addr:    config.HTTPServerAddress,
    }

    waitGroup.Go(func() error {
        log.Info().Msgf("start HTTP gateway server at %s", httpServer.Addr)
        err = httpServer.ListenAndServe()
        if err != nil {
            if errors.Is(err, http.ErrServerClosed) {
                return nil
            }
            log.Error().Err(err).Msg("HTTP gateway server failed to serve")
            return err
        }
        return nil
    })

    waitGroup.Go(func() error {
        <-ctx.Done()
        log.Info().Msg("graceful shutdown HTTP gateway server")

        err := httpServer.Shutdown(context.Background())
        if err != nil {
            log.Error().Err(err).Msg("failed to shutdown HTTP gateway server")
            return err
        }

        log.Info().Msg("HTTP gateway server is stopped")
        return nil
    })
}

func runGinServer(config util.Config, store db.Store) {
    server, err := api.NewServer(config, store)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot create server")
    }

    err = server.Start(config.HTTPServerAddress)
    if err != nil {
        log.Fatal().Err(err).Msg("cannot start server")
    }
}
```

FILE: simplebank/pb/rpc_create_user.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: rpc_create_user.proto

package pb

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
    sync "sync"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type CreateUserRequest struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    Username string `protobuf:"bytes,1,opt,name=username,proto3" json:"username,omitempty"`
    FullName string `protobuf:"bytes,2,opt,name=full_name,json=fullName,proto3"
json:"full_name,omitempty"`
    Email    string `protobuf:"bytes,3,opt,name=email,proto3" json:"email,omitempty"`
    Password string `protobuf:"bytes,4,opt,name=password,proto3" json:"password,omitempty"`
}

func (x *CreateUserRequest) Reset() {
    *x = CreateUserRequest{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_create_user_proto_msgTypes[0]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *CreateUserRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*CreateUserRequest) ProtoMessage() {}

func (x *CreateUserRequest) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_create_user_proto_msgTypes[0]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use CreateUserRequest.ProtoReflect.Descriptor instead.
func (*CreateUserRequest) Descriptor() ([]byte, []int) {
    return file_rpc_create_user_proto_rawDescGZIP(), []int{0}
}

func (x *CreateUserRequest) GetUsername() string {
    if x != nil {
        return x.Username
    }
    return ""
}

func (x *CreateUserRequest) GetFullName() string {
    if x != nil {
        return x.FullName
    }
    return ""
}

func (x *CreateUserRequest) GetEmail() string {
    if x != nil {
        return x.Email
    }
    return ""
}

func (x *CreateUserRequest) GetPassword() string {
    if x != nil {
        return x.Password
    }
    return ""
}

type CreateUserResponse struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    User *User `protobuf:"bytes,1,opt,name=user,proto3" json:"user,omitempty"`
}

func (x *CreateUserResponse) Reset() {
    *x = CreateUserResponse{}
```

```
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_create_user_proto_msgTypes[1]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *CreateUserResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*CreateUserResponse) ProtoMessage() {}

func (x *CreateUserResponse) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_create_user_proto_msgTypes[1]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use CreateUserResponse.ProtoReflect.Descriptor instead.
func (*CreateUserResponse) Descriptor() ([]byte, []int) {
    return file_rpc_create_user_proto_rawDescGZIP(), []int{1}
}

func (x *CreateUserResponse) GetUser() *User {
    if x != nil {
        return x.User
    }
    return nil
}

var File_rpc_create_user_proto protoreflect.FileDescriptor

var file_rpc_create_user_proto_rawDesc = []byte{
    0x0a, 0x15, 0x72, 0x70, 0x63, 0x5f, 0x63, 0x72, 0x65, 0x61, 0x74, 0x65, 0x5f, 0x75, 0x73,
0x65,
    0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70, 0x62, 0x1a, 0x0a, 0x75, 0x73,
0x65,
    0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x22, 0x7e, 0x0a, 0x11, 0x43, 0x72, 0x65, 0x61,
0x74,
    0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74, 0x12, 0x1a, 0x0a,
0x08,
    0x75, 0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x01, 0x20, 0x01, 0x28, 0x09, 0x52,
0x08,
    0x75, 0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x12, 0x1b, 0x0a, 0x09, 0x66, 0x75, 0x6c,
0x6c,
    0x5f, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x02, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08, 0x66, 0x75,
0x6c,
    0x6c, 0x4e, 0x61, 0x6d, 0x65, 0x12, 0x14, 0x0a, 0x05, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x18,
0x03,
    0x20, 0x01, 0x28, 0x09, 0x52, 0x05, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x12, 0x1a, 0x0a, 0x08,
0x70,
    0x61, 0x73, 0x73, 0x77, 0x6f, 0x72, 0x64, 0x18, 0x04, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08,
0x70,
    0x61, 0x73, 0x73, 0x77, 0x6f, 0x72, 0x64, 0x22, 0x32, 0x0a, 0x12, 0x43, 0x72, 0x65, 0x61,
0x74,
    0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x12, 0x1c,
0x0a,
    0x04, 0x75, 0x73, 0x65, 0x72, 0x18, 0x01, 0x20, 0x01, 0x28, 0x0b, 0x32, 0x08, 0x2e, 0x70,
0x62,
    0x2e, 0x55, 0x73, 0x65, 0x72, 0x52, 0x04, 0x75, 0x73, 0x65, 0x72, 0x42, 0x25, 0x5a, 0x23,
0x67,
    0x69, 0x74, 0x68, 0x75, 0x62, 0x2e, 0x63, 0x6f, 0x6d, 0x2f, 0x74, 0x65, 0x63, 0x68, 0x73,
0x63,
    0x68, 0x6f, 0x6f, 0x6c, 0x2f, 0x73, 0x69, 0x6d, 0x70, 0x6c, 0x65, 0x62, 0x61, 0x6e, 0x6b,
0x2f,
    0x70, 0x62, 0x62, 0x06, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x33,
}

var (
    file_rpc_create_user_proto_rawDescOnce sync.Once
    file_rpc_create_user_proto_rawDescData = file_rpc_create_user_proto_rawDesc
)

func file_rpc_create_user_proto_rawDescGZIP() []byte {
    file_rpc_create_user_proto_rawDescOnce.Do(func() {
        file_rpc_create_user_proto_rawDescData =
protoimpl.X.CompressGZIP(file_rpc_create_user_proto_rawDescData)
    })
    return file_rpc_create_user_proto_rawDescData
}

var file_rpc_create_user_proto_msgTypes = make([]protoimpl.MessageInfo, 2)
var file_rpc_create_user_proto_goTypes = []interface{}{
    (*CreateUserRequest)(nil),  // 0: pb.CreateUserRequest
    (*CreateUserResponse)(nil), // 1: pb.CreateUserResponse
    (*User)(nil),               // 2: pb.User
}
var file_rpc_create_user_proto_depIdxs = []int32{
    2, // 0: pb.CreateUserResponse.user:type_name -> pb.User
    1, // [1:1] is the sub-list for method output_type
    1, // [1:1] is the sub-list for method input_type
    1, // [1:1] is the sub-list for extension type_name
    1, // [1:1] is the sub-list for extension extendee
    0, // [0:1] is the sub-list for field type_name
}

func init() { file_rpc_create_user_proto_init() }
func file_rpc_create_user_proto_init() {
    if File_rpc_create_user_proto != nil {
        return
    }
    file_user_proto_init()
    if !protoimpl.UnsafeEnabled {
        file_rpc_create_user_proto_msgTypes[0].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*CreateUserRequest); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
        file_rpc_create_user_proto_msgTypes[1].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*CreateUserResponse); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
    }
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_rpc_create_user_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   2,
            NumExtensions: 0,
            NumServices:   0,
        },
        GoTypes:           file_rpc_create_user_proto_goTypes,
        DependencyIndexes: file_rpc_create_user_proto_depIdxs,
        MessageInfos:      file_rpc_create_user_proto_msgTypes,
    }.Build()
    File_rpc_create_user_proto = out.File
    file_rpc_create_user_proto_rawDesc = nil
    file_rpc_create_user_proto_goTypes = nil
    file_rpc_create_user_proto_depIdxs = nil
}
```

FILE: simplebank/pb/rpc_login_user.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: rpc_login_user.proto

package pb

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    timestamppb "google.golang.org/protobuf/types/known/timestamppb"
    reflect "reflect"
    sync "sync"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type LoginUserRequest struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    Username string `protobuf:"bytes,1,opt,name=username,proto3" json:"username,omitempty"`
    Password string `protobuf:"bytes,2,opt,name=password,proto3" json:"password,omitempty"`
}

func (x *LoginUserRequest) Reset() {
    *x = LoginUserRequest{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_login_user_proto_msgTypes[0]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *LoginUserRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*LoginUserRequest) ProtoMessage() {}

func (x *LoginUserRequest) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_login_user_proto_msgTypes[0]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}
```

```
// Deprecated: Use LoginUserRequest.ProtoReflect.Descriptor instead.
func (*LoginUserRequest) Descriptor() ([]byte, []int) {
    return file_rpc_login_user_proto_rawDescGZIP(), []int{0}
}

func (x *LoginUserRequest) GetUsername() string {
    if x != nil {
        return x.Username
    }
    return ""
}

func (x *LoginUserRequest) GetPassword() string {
    if x != nil {
        return x.Password
    }
    return ""
}

type LoginUserResponse struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    User                  *User                  `protobuf:"bytes,1,opt,name=user,proto3"
json:"user,omitempty"`
    SessionId             string
`protobuf:"bytes,2,opt,name=session_id,json=sessionId,proto3" json:"session_id,omitempty"`
    AccessToken           string
`protobuf:"bytes,3,opt,name=access_token,json=accessToken,proto3"
json:"access_token,omitempty"`
    RefreshToken          string
`protobuf:"bytes,4,opt,name=refresh_token,json=refreshToken,proto3"
json:"refresh_token,omitempty"`
    AccessTokenExpiresAt  *timestamppb.Timestamp
`protobuf:"bytes,5,opt,name=access_token_expires_at,json=accessTokenExpiresAt,proto3"
json:"access_token_expires_at,omitempty"`
    RefreshTokenExpiresAt *timestamppb.Timestamp
`protobuf:"bytes,6,opt,name=refresh_token_expires_at,json=refreshTokenExpiresAt,proto3"
json:"refresh_token_expires_at,omitempty"`
}

func (x *LoginUserResponse) Reset() {
    *x = LoginUserResponse{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_login_user_proto_msgTypes[1]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *LoginUserResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*LoginUserResponse) ProtoMessage() {}

func (x *LoginUserResponse) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_login_user_proto_msgTypes[1]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use LoginUserResponse.ProtoReflect.Descriptor instead.
func (*LoginUserResponse) Descriptor() ([]byte, []int) {
    return file_rpc_login_user_proto_rawDescGZIP(), []int{1}
}

func (x *LoginUserResponse) GetUser() *User {
    if x != nil {
        return x.User
    }
    return nil
}

func (x *LoginUserResponse) GetSessionId() string {
    if x != nil {
        return x.SessionId
    }
    return ""
}

func (x *LoginUserResponse) GetAccessToken() string {
    if x != nil {
        return x.AccessToken
    }
    return ""
}

func (x *LoginUserResponse) GetRefreshToken() string {
    if x != nil {
        return x.RefreshToken
    }
    return ""
}

func (x *LoginUserResponse) GetAccessTokenExpiresAt() *timestamppb.Timestamp {
    if x != nil {
        return x.AccessTokenExpiresAt
    }
    return nil
}

func (x *LoginUserResponse) GetRefreshTokenExpiresAt() *timestamppb.Timestamp {
    if x != nil {
        return x.RefreshTokenExpiresAt
    }
    return nil
}

var File_rpc_login_user_proto protoreflect.FileDescriptor

var file_rpc_login_user_proto_rawDesc = []byte{
    0x0a, 0x14, 0x72, 0x70, 0x63, 0x5f, 0x6c, 0x6f, 0x67, 0x69, 0x6e, 0x5f, 0x75, 0x73, 0x65,
0x72,
    0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70, 0x62, 0x1a, 0x0a, 0x75, 0x73, 0x65,
0x72,
    0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x1a, 0x1f, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65, 0x2f,
0x70,
    0x72, 0x6f, 0x74, 0x6f, 0x62, 0x75, 0x66, 0x2f, 0x74, 0x69, 0x6d, 0x65, 0x73, 0x74, 0x61,
0x6d,
    0x70, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x22, 0x4a, 0x0a, 0x10, 0x4c, 0x6f, 0x67, 0x69,
0x6e,
    0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74, 0x12, 0x1a, 0x0a, 0x08,
0x75,
    0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x01, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08,
0x75,
    0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x12, 0x1a, 0x0a, 0x08, 0x70, 0x61, 0x73, 0x73,
0x77,
    0x6f, 0x72, 0x64, 0x18, 0x02, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08, 0x70, 0x61, 0x73, 0x73,
0x77,
    0x6f, 0x72, 0x64, 0x22, 0xc0, 0x02, 0x0a, 0x11, 0x4c, 0x6f, 0x67, 0x69, 0x6e, 0x55, 0x73,
0x65,
    0x72, 0x52, 0x65, 0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x12, 0x1c, 0x0a, 0x04, 0x75, 0x73,
0x65,
    0x72, 0x18, 0x01, 0x20, 0x01, 0x28, 0x0b, 0x32, 0x08, 0x2e, 0x70, 0x62, 0x2e, 0x55, 0x73,
0x65,
    0x72, 0x52, 0x04, 0x75, 0x73, 0x65, 0x72, 0x12, 0x1d, 0x0a, 0x0a, 0x73, 0x65, 0x73, 0x73,
0x69,
    0x6f, 0x6e, 0x5f, 0x69, 0x64, 0x18, 0x02, 0x20, 0x01, 0x28, 0x09, 0x52, 0x09, 0x73, 0x65,
0x73,
    0x73, 0x69, 0x6f, 0x6e, 0x49, 0x64, 0x12, 0x21, 0x0a, 0x0c, 0x61, 0x63, 0x63, 0x65, 0x73,
0x73,
    0x5f, 0x74, 0x6f, 0x6b, 0x65, 0x6e, 0x18, 0x03, 0x20, 0x01, 0x28, 0x09, 0x52, 0x0b, 0x61,
0x63,
    0x63, 0x65, 0x73, 0x73, 0x54, 0x6f, 0x6b, 0x65, 0x6e, 0x12, 0x23, 0x0a, 0x0d, 0x72, 0x65,
0x66,
    0x72, 0x65, 0x73, 0x68, 0x5f, 0x74, 0x6f, 0x6b, 0x65, 0x6e, 0x18, 0x04, 0x20, 0x01, 0x28,
0x09,
    0x52, 0x0c, 0x72, 0x65, 0x66, 0x72, 0x65, 0x73, 0x68, 0x54, 0x6f, 0x6b, 0x65, 0x6e, 0x12,
0x51,
    0x0a, 0x17, 0x61, 0x63, 0x63, 0x65, 0x73, 0x73, 0x5f, 0x74, 0x6f, 0x6b, 0x65, 0x6e, 0x5f,
0x65,
    0x78, 0x70, 0x69, 0x72, 0x65, 0x73, 0x5f, 0x61, 0x74, 0x18, 0x05, 0x20, 0x01, 0x28, 0x0b,
0x32,
    0x1a, 0x2e, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x62,
0x75,
    0x66, 0x2e, 0x54, 0x69, 0x6d, 0x65, 0x73, 0x74, 0x61, 0x6d, 0x70, 0x52, 0x14, 0x61, 0x63,
0x63,
    0x65, 0x73, 0x73, 0x54, 0x6f, 0x6b, 0x65, 0x6e, 0x45, 0x78, 0x70, 0x69, 0x72, 0x65, 0x73,
0x41,
    0x74, 0x12, 0x53, 0x0a, 0x18, 0x72, 0x65, 0x66, 0x72, 0x65, 0x73, 0x68, 0x5f, 0x74, 0x6f,
0x6b,
    0x65, 0x6e, 0x5f, 0x65, 0x78, 0x70, 0x69, 0x72, 0x65, 0x73, 0x5f, 0x61, 0x74, 0x18, 0x06,
0x20,
    0x01, 0x28, 0x0b, 0x32, 0x1a, 0x2e, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65, 0x2e, 0x70, 0x72,
0x6f,
    0x74, 0x6f, 0x62, 0x75, 0x66, 0x2e, 0x54, 0x69, 0x6d, 0x65, 0x73, 0x74, 0x61, 0x6d, 0x70,
0x52,
    0x15, 0x72, 0x65, 0x66, 0x72, 0x65, 0x73, 0x68, 0x54, 0x6f, 0x6b, 0x65, 0x6e, 0x45, 0x78,
0x70,
    0x69, 0x72, 0x65, 0x73, 0x41, 0x74, 0x42, 0x25, 0x5a, 0x23, 0x67, 0x69, 0x74, 0x68, 0x75,
0x62,
    0x2e, 0x63, 0x6f, 0x6d, 0x2f, 0x74, 0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c,
0x2f,
    0x73, 0x69, 0x6d, 0x70, 0x6c, 0x65, 0x62, 0x61, 0x6e, 0x6b, 0x2f, 0x70, 0x62, 0x62, 0x06,
0x70,
    0x72, 0x6f, 0x74, 0x6f, 0x33,
}

var (
    file_rpc_login_user_proto_rawDescOnce sync.Once
    file_rpc_login_user_proto_rawDescData = file_rpc_login_user_proto_rawDesc
)

func file_rpc_login_user_proto_rawDescGZIP() []byte {
    file_rpc_login_user_proto_rawDescOnce.Do(func() {
        file_rpc_login_user_proto_rawDescData =
protoimpl.X.CompressGZIP(file_rpc_login_user_proto_rawDescData)
    })
    return file_rpc_login_user_proto_rawDescData
}

var file_rpc_login_user_proto_msgTypes = make([]protoimpl.MessageInfo, 2)
var file_rpc_login_user_proto_goTypes = []interface{}{
    (*LoginUserRequest)(nil),      // 0: pb.LoginUserRequest
    (*LoginUserResponse)(nil),     // 1: pb.LoginUserResponse
    (*User)(nil),                  // 2: pb.User
    (*timestamppb.Timestamp)(nil), // 3: google.protobuf.Timestamp
}
var file_rpc_login_user_proto_depIdxs = []int32{
    2, // 0: pb.LoginUserResponse.user:type_name -> pb.User
    3, // 1: pb.LoginUserResponse.access_token_expires_at:type_name ->
google.protobuf.Timestamp
    3, // 2: pb.LoginUserResponse.refresh_token_expires_at:type_name ->
google.protobuf.Timestamp
    3, // [3:3] is the sub-list for method output_type
    3, // [3:3] is the sub-list for method input_type
    3, // [3:3] is the sub-list for extension type_name
```

```
    3, // [3:3] is the sub-list for extension extendee
    0, // [0:3] is the sub-list for field type_name
}

func init() { file_rpc_login_user_proto_init() }
func file_rpc_login_user_proto_init() {
    if File_rpc_login_user_proto != nil {
        return
    }
    file_user_proto_init()
    if !protoimpl.UnsafeEnabled {
        file_rpc_login_user_proto_msgTypes[0].Exporter = func(v interface{}, i int) interface{}
{
            switch v := v.(*LoginUserRequest); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
        file_rpc_login_user_proto_msgTypes[1].Exporter = func(v interface{}, i int) interface{}
{
            switch v := v.(*LoginUserResponse); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
    }
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_rpc_login_user_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   2,
            NumExtensions: 0,
            NumServices:   0,
        },
        GoTypes:           file_rpc_login_user_proto_goTypes,
        DependencyIndexes: file_rpc_login_user_proto_depIdxs,
        MessageInfos:      file_rpc_login_user_proto_msgTypes,
    }.Build()
    File_rpc_login_user_proto = out.File
    file_rpc_login_user_proto_rawDesc = nil
    file_rpc_login_user_proto_goTypes = nil
    file_rpc_login_user_proto_depIdxs = nil
}
```

FILE: simplebank/pb/rpc_update_user.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: rpc_update_user.proto

package pb

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
    sync "sync"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type UpdateUserRequest struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    Username string  `protobuf:"bytes,1,opt,name=username,proto3" json:"username,omitempty"`
    FullName *string `protobuf:"bytes,2,opt,name=full_name,json=fullName,proto3,oneof"
json:"full_name,omitempty"`
    Email    *string `protobuf:"bytes,3,opt,name=email,proto3,oneof" json:"email,omitempty"`
    Password *string `protobuf:"bytes,4,opt,name=password,proto3,oneof"
json:"password,omitempty"`
}

func (x *UpdateUserRequest) Reset() {
    *x = UpdateUserRequest{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_update_user_proto_msgTypes[0]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *UpdateUserRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*UpdateUserRequest) ProtoMessage() {}

func (x *UpdateUserRequest) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_update_user_proto_msgTypes[0]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use UpdateUserRequest.ProtoReflect.Descriptor instead.
func (*UpdateUserRequest) Descriptor() ([]byte, []int) {
    return file_rpc_update_user_proto_rawDescGZIP(), []int{0}
}

func (x *UpdateUserRequest) GetUsername() string {
    if x != nil {
        return x.Username
    }
    return ""
}

func (x *UpdateUserRequest) GetFullName() string {
    if x != nil && x.FullName != nil {
        return *x.FullName
    }
    return ""
}

func (x *UpdateUserRequest) GetEmail() string {
    if x != nil && x.Email != nil {
        return *x.Email
    }
    return ""
}

func (x *UpdateUserRequest) GetPassword() string {
    if x != nil && x.Password != nil {
        return *x.Password
    }
    return ""
}

type UpdateUserResponse struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    User *User `protobuf:"bytes,1,opt,name=user,proto3" json:"user,omitempty"`
}

func (x *UpdateUserResponse) Reset() {
    *x = UpdateUserResponse{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_update_user_proto_msgTypes[1]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *UpdateUserResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*UpdateUserResponse) ProtoMessage() {}

func (x *UpdateUserResponse) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_update_user_proto_msgTypes[1]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use UpdateUserResponse.ProtoReflect.Descriptor instead.
func (*UpdateUserResponse) Descriptor() ([]byte, []int) {
    return file_rpc_update_user_proto_rawDescGZIP(), []int{1}
}

func (x *UpdateUserResponse) GetUser() *User {
    if x != nil {
        return x.User
    }
    return nil
}

var File_rpc_update_user_proto protoreflect.FileDescriptor

var file_rpc_update_user_proto_rawDesc = []byte{
    0x0a, 0x15, 0x72, 0x70, 0x63, 0x5f, 0x75, 0x70, 0x64, 0x61, 0x74, 0x65, 0x5f, 0x75, 0x73,
0x65,
    0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70, 0x62, 0x1a, 0x0a, 0x75, 0x73,
0x65,
    0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x22, 0xb2, 0x01, 0x0a, 0x11, 0x55, 0x70, 0x64,
0x61,
    0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74, 0x12, 0x1a,
0x0a,
    0x08, 0x75, 0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x01, 0x20, 0x01, 0x28, 0x09,
0x52,
    0x08, 0x75, 0x73, 0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x12, 0x20, 0x0a, 0x09, 0x66, 0x75,
0x6c,
```

```
    0x6c, 0x5f, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x02, 0x20, 0x01, 0x28, 0x09, 0x48, 0x00, 0x52,
0x08,
    0x66, 0x75, 0x6c, 0x6c, 0x4e, 0x61, 0x6d, 0x65, 0x88, 0x01, 0x01, 0x12, 0x19, 0x0a, 0x05,
0x65,
    0x6d, 0x61, 0x69, 0x6c, 0x18, 0x03, 0x20, 0x01, 0x28, 0x09, 0x48, 0x01, 0x52, 0x05, 0x65,
0x6d,
    0x61, 0x69, 0x6c, 0x88, 0x01, 0x01, 0x12, 0x1f, 0x0a, 0x08, 0x70, 0x61, 0x73, 0x73, 0x77,
0x6f,
    0x72, 0x64, 0x18, 0x04, 0x20, 0x01, 0x28, 0x09, 0x48, 0x02, 0x52, 0x08, 0x70, 0x61, 0x73,
0x73,
    0x77, 0x6f, 0x72, 0x64, 0x88, 0x01, 0x01, 0x42, 0x0c, 0x0a, 0x0a, 0x5f, 0x66, 0x75, 0x6c,
0x6c,
    0x5f, 0x6e, 0x61, 0x6d, 0x65, 0x42, 0x08, 0x0a, 0x06, 0x5f, 0x65, 0x6d, 0x61, 0x69, 0x6c,
0x42,
    0x0b, 0x0a, 0x09, 0x5f, 0x70, 0x61, 0x73, 0x73, 0x77, 0x6f, 0x72, 0x64, 0x22, 0x32, 0x0a,
0x12,
    0x55, 0x70, 0x64, 0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x73, 0x70, 0x6f,
0x6e,
    0x73, 0x65, 0x12, 0x1c, 0x0a, 0x04, 0x75, 0x73, 0x65, 0x72, 0x18, 0x01, 0x20, 0x01, 0x28,
0x0b,
    0x32, 0x08, 0x2e, 0x70, 0x62, 0x2e, 0x55, 0x73, 0x65, 0x72, 0x52, 0x04, 0x75, 0x73, 0x65,
0x72,
    0x42, 0x25, 0x5a, 0x23, 0x67, 0x69, 0x74, 0x68, 0x75, 0x62, 0x2e, 0x63, 0x6f, 0x6d, 0x2f,
0x74,
    0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x2f, 0x73, 0x69, 0x6d, 0x70, 0x6c,
0x65,
    0x62, 0x61, 0x6e, 0x6b, 0x2f, 0x70, 0x62, 0x62, 0x06, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x33,
}

var (
    file_rpc_update_user_proto_rawDescOnce sync.Once
    file_rpc_update_user_proto_rawDescData = file_rpc_update_user_proto_rawDesc
)

func file_rpc_update_user_proto_rawDescGZIP() []byte {
    file_rpc_update_user_proto_rawDescOnce.Do(func() {
        file_rpc_update_user_proto_rawDescData =
protoimpl.X.CompressGZIP(file_rpc_update_user_proto_rawDescData)
    })
    return file_rpc_update_user_proto_rawDescData
}

var file_rpc_update_user_proto_msgTypes = make([]protoimpl.MessageInfo, 2)
var file_rpc_update_user_proto_goTypes = []interface{}{
    (*UpdateUserRequest)(nil),  // 0: pb.UpdateUserRequest
    (*UpdateUserResponse)(nil), // 1: pb.UpdateUserResponse
    (*User)(nil),               // 2: pb.User
}
var file_rpc_update_user_proto_depIdxs = []int32{
    2, // 0: pb.UpdateUserResponse.user:type_name -> pb.User
    1, // [1:1] is the sub-list for method output_type
    1, // [1:1] is the sub-list for method input_type
    1, // [1:1] is the sub-list for extension type_name
    1, // [1:1] is the sub-list for extension extendee
    0, // [0:1] is the sub-list for field type_name
}

func init() { file_rpc_update_user_proto_init() }
func file_rpc_update_user_proto_init() {
    if File_rpc_update_user_proto != nil {
        return
    }
    file_user_proto_init()
    if !protoimpl.UnsafeEnabled {
        file_rpc_update_user_proto_msgTypes[0].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*UpdateUserRequest); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
        file_rpc_update_user_proto_msgTypes[1].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*UpdateUserResponse); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
    }
    file_rpc_update_user_proto_msgTypes[0].OneofWrappers = []interface{}{}
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_rpc_update_user_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   2,
            NumExtensions: 0,
            NumServices:   0,
        },
        GoTypes:           file_rpc_update_user_proto_goTypes,
        DependencyIndexes: file_rpc_update_user_proto_depIdxs,
        MessageInfos:      file_rpc_update_user_proto_msgTypes,
    }.Build()
    File_rpc_update_user_proto = out.File
    file_rpc_update_user_proto_rawDesc = nil
    file_rpc_update_user_proto_goTypes = nil
    file_rpc_update_user_proto_depIdxs = nil
}
```

FILE: simplebank/pb/rpc_verify_email.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: rpc_verify_email.proto

package pb

import (
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
    sync "sync"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type VerifyEmailRequest struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    EmailId    int64  `protobuf:"varint,1,opt,name=email_id,json=emailId,proto3"
json:"email_id,omitempty"`
    SecretCode string `protobuf:"bytes,2,opt,name=secret_code,json=secretCode,proto3"
json:"secret_code,omitempty"`
}

func (x *VerifyEmailRequest) Reset() {
    *x = VerifyEmailRequest{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_verify_email_proto_msgTypes[0]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *VerifyEmailRequest) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*VerifyEmailRequest) ProtoMessage() {}

func (x *VerifyEmailRequest) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_verify_email_proto_msgTypes[0]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use VerifyEmailRequest.ProtoReflect.Descriptor instead.
func (*VerifyEmailRequest) Descriptor() ([]byte, []int) {
    return file_rpc_verify_email_proto_rawDescGZIP(), []int{0}
}

func (x *VerifyEmailRequest) GetEmailId() int64 {
    if x != nil {
        return x.EmailId
    }
    return 0
}

func (x *VerifyEmailRequest) GetSecretCode() string {
    if x != nil {
        return x.SecretCode
    }
    return ""
}

type VerifyEmailResponse struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    IsVerified bool `protobuf:"varint,1,opt,name=is_verified,json=isVerified,proto3"
json:"is_verified,omitempty"`
}

func (x *VerifyEmailResponse) Reset() {
    *x = VerifyEmailResponse{}
    if protoimpl.UnsafeEnabled {
        mi := &file_rpc_verify_email_proto_msgTypes[1]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *VerifyEmailResponse) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*VerifyEmailResponse) ProtoMessage() {}
```

```
func (x *VerifyEmailResponse) ProtoReflect() protoreflect.Message {
    mi := &file_rpc_verify_email_proto_msgTypes[1]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use VerifyEmailResponse.ProtoReflect.Descriptor instead.
func (*VerifyEmailResponse) Descriptor() ([]byte, []int) {
    return file_rpc_verify_email_proto_rawDescGZIP(), []int{1}
}

func (x *VerifyEmailResponse) GetIsVerified() bool {
    if x != nil {
        return x.IsVerified
    }
    return false
}

var File_rpc_verify_email_proto protoreflect.FileDescriptor

var file_rpc_verify_email_proto_rawDesc = []byte{
    0x0a, 0x16, 0x72, 0x70, 0x63, 0x5f, 0x76, 0x65, 0x72, 0x69, 0x66, 0x79, 0x5f, 0x65, 0x6d,
0x61,
    0x69, 0x6c, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70, 0x62, 0x22, 0x50, 0x0a,
0x12,
    0x56, 0x65, 0x72, 0x69, 0x66, 0x79, 0x45, 0x6d, 0x61, 0x69, 0x6c, 0x52, 0x65, 0x71, 0x75,
0x65,
    0x73, 0x74, 0x12, 0x19, 0x0a, 0x08, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x5f, 0x69, 0x64, 0x18,
0x01,
    0x20, 0x01, 0x28, 0x03, 0x52, 0x07, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x49, 0x64, 0x12, 0x1f,
0x0a,
    0x0b, 0x73, 0x65, 0x63, 0x72, 0x65, 0x74, 0x5f, 0x63, 0x6f, 0x64, 0x65, 0x18, 0x02, 0x20,
0x01,
    0x28, 0x09, 0x52, 0x0a, 0x73, 0x65, 0x63, 0x72, 0x65, 0x74, 0x43, 0x6f, 0x64, 0x65, 0x22,
0x36,
    0x0a, 0x13, 0x56, 0x65, 0x72, 0x69, 0x66, 0x79, 0x45, 0x6d, 0x61, 0x69, 0x6c, 0x52, 0x65,
0x73,
    0x70, 0x6f, 0x6e, 0x73, 0x65, 0x12, 0x1f, 0x0a, 0x0b, 0x69, 0x73, 0x5f, 0x76, 0x65, 0x72,
0x69,
    0x66, 0x69, 0x65, 0x64, 0x18, 0x01, 0x20, 0x01, 0x28, 0x08, 0x52, 0x0a, 0x69, 0x73, 0x56,
0x65,
    0x72, 0x69, 0x66, 0x69, 0x65, 0x64, 0x42, 0x25, 0x5a, 0x23, 0x67, 0x69, 0x74, 0x68, 0x75,
0x62,
    0x2e, 0x63, 0x6f, 0x6d, 0x2f, 0x74, 0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c,
0x2f,
    0x73, 0x69, 0x6d, 0x70, 0x6c, 0x65, 0x62, 0x61, 0x6e, 0x6b, 0x2f, 0x70, 0x62, 0x62, 0x06,
0x70,
    0x72, 0x6f, 0x74, 0x6f, 0x33,
}

var (
    file_rpc_verify_email_proto_rawDescOnce sync.Once
    file_rpc_verify_email_proto_rawDescData = file_rpc_verify_email_proto_rawDesc
)

func file_rpc_verify_email_proto_rawDescGZIP() []byte {
    file_rpc_verify_email_proto_rawDescOnce.Do(func() {
        file_rpc_verify_email_proto_rawDescData =
protoimpl.X.CompressGZIP(file_rpc_verify_email_proto_rawDescData)
    })
    return file_rpc_verify_email_proto_rawDescData
}

var file_rpc_verify_email_proto_msgTypes = make([]protoimpl.MessageInfo, 2)
var file_rpc_verify_email_proto_goTypes = []interface{}{
    (*VerifyEmailRequest)(nil),  // 0: pb.VerifyEmailRequest
    (*VerifyEmailResponse)(nil), // 1: pb.VerifyEmailResponse
}
var file_rpc_verify_email_proto_depIdxs = []int32{
    0, // [0:0] is the sub-list for method output_type
    0, // [0:0] is the sub-list for method input_type
    0, // [0:0] is the sub-list for extension type_name
    0, // [0:0] is the sub-list for extension extendee
    0, // [0:0] is the sub-list for field type_name
}

func init() { file_rpc_verify_email_proto_init() }
func file_rpc_verify_email_proto_init() {
    if File_rpc_verify_email_proto != nil {
        return
    }
    if !protoimpl.UnsafeEnabled {
        file_rpc_verify_email_proto_msgTypes[0].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*VerifyEmailRequest); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
        file_rpc_verify_email_proto_msgTypes[1].Exporter = func(v interface{}, i int)
interface{} {
            switch v := v.(*VerifyEmailResponse); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
    }
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_rpc_verify_email_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   2,
            NumExtensions: 0,
            NumServices:   0,
        },
        GoTypes:           file_rpc_verify_email_proto_goTypes,
        DependencyIndexes: file_rpc_verify_email_proto_depIdxs,
        MessageInfos:      file_rpc_verify_email_proto_msgTypes,
    }.Build()
    File_rpc_verify_email_proto = out.File
    file_rpc_verify_email_proto_rawDesc = nil
    file_rpc_verify_email_proto_goTypes = nil
    file_rpc_verify_email_proto_depIdxs = nil
}
```

FILE: simplebank/pb/service_simple_bank.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: service_simple_bank.proto

package pb

import (
    _ "github.com/grpc-ecosystem/grpc-gateway/v2/protoc-gen-openapiv2/options"
    _ "google.golang.org/genproto/googleapis/api/annotations"
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    reflect "reflect"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

var File_service_simple_bank_proto protoreflect.FileDescriptor

var file_service_simple_bank_proto_rawDesc = []byte{
    0x0a, 0x19, 0x73, 0x65, 0x72, 0x76, 0x69, 0x63, 0x65, 0x5f, 0x73, 0x69, 0x6d, 0x70, 0x6c,
0x65,
    0x5f, 0x62, 0x61, 0x6e, 0x6b, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70, 0x62,
0x1a,
    0x1c, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65, 0x2f, 0x61, 0x70, 0x69, 0x2f, 0x61, 0x6e, 0x6e,
0x6f,
    0x74, 0x61, 0x74, 0x69, 0x6f, 0x6e, 0x73, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x1a, 0x15,
0x72,
    0x70, 0x63, 0x5f, 0x63, 0x72, 0x65, 0x61, 0x74, 0x65, 0x5f, 0x75, 0x73, 0x65, 0x72, 0x2e,
0x70,
    0x72, 0x6f, 0x74, 0x6f, 0x1a, 0x15, 0x72, 0x70, 0x63, 0x5f, 0x75, 0x70, 0x64, 0x61, 0x74,
0x65,
    0x5f, 0x75, 0x73, 0x65, 0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x1a, 0x14, 0x72, 0x70,
0x63,
    0x5f, 0x6c, 0x6f, 0x67, 0x69, 0x6e, 0x5f, 0x75, 0x73, 0x65, 0x72, 0x2e, 0x70, 0x72, 0x6f,
0x74,
    0x6f, 0x1a, 0x16, 0x72, 0x70, 0x63, 0x5f, 0x76, 0x65, 0x72, 0x69, 0x66, 0x79, 0x5f, 0x65,
0x6d,
    0x61, 0x69, 0x6c, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x1a, 0x2e, 0x70, 0x72, 0x6f, 0x74,
0x6f,
    0x63, 0x2d, 0x67, 0x65, 0x6e, 0x2d, 0x6f, 0x70, 0x65, 0x6e, 0x61, 0x70, 0x69, 0x76, 0x32,
0x2f,
    0x6f, 0x70, 0x74, 0x69, 0x6f, 0x6e, 0x73, 0x2f, 0x61, 0x6e, 0x6e, 0x6f, 0x74, 0x61, 0x74,
0x69,
    0x6f, 0x6e, 0x73, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x32, 0xe3, 0x04, 0x0a, 0x0a, 0x53,
0x69,
    0x6d, 0x70, 0x6c, 0x65, 0x42, 0x61, 0x6e, 0x6b, 0x12, 0x8e, 0x01, 0x0a, 0x0a, 0x43, 0x72,
0x65,
    0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x12, 0x15, 0x2e, 0x70, 0x62, 0x2e, 0x43, 0x72,
0x65,
    0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74, 0x1a,
0x16,
    0x2e, 0x70, 0x62, 0x2e, 0x43, 0x72, 0x65, 0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x52,
0x65,
    0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x22, 0x51, 0x92, 0x41, 0x34, 0x12, 0x0f, 0x43, 0x72,
0x65,
    0x61, 0x74, 0x65, 0x20, 0x6e, 0x65, 0x77, 0x20, 0x75, 0x73, 0x65, 0x72, 0x1a, 0x21, 0x55,
0x73,
    0x65, 0x20, 0x74, 0x68, 0x69, 0x73, 0x20, 0x41, 0x50, 0x49, 0x20, 0x74, 0x6f, 0x20, 0x63,
0x72,
    0x65, 0x61, 0x74, 0x65, 0x20, 0x61, 0x20, 0x6e, 0x65, 0x77, 0x20, 0x75, 0x73, 0x65, 0x72,
0x82,
    0xd3, 0xe4, 0x93, 0x02, 0x14, 0x22, 0x0f, 0x2f, 0x76, 0x31, 0x2f, 0x63, 0x72, 0x65, 0x61,
0x74,
    0x65, 0x5f, 0x75, 0x73, 0x65, 0x72, 0x3a, 0x01, 0x2a, 0x12, 0x84, 0x01, 0x0a, 0x0a, 0x55,
0x70,
    0x64, 0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x12, 0x15, 0x2e, 0x70, 0x62, 0x2e, 0x55,
0x70,
    0x64, 0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74,
0x1a,
    0x16, 0x2e, 0x70, 0x62, 0x2e, 0x55, 0x70, 0x64, 0x61, 0x74, 0x65, 0x55, 0x73, 0x65, 0x72,
0x52,
    0x65, 0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x22, 0x47, 0x92, 0x41, 0x2a, 0x12, 0x0b, 0x55,
0x70,
```

```
    0x64, 0x61, 0x74, 0x65, 0x20, 0x75, 0x73, 0x65, 0x72, 0x1a, 0x1b, 0x55, 0x73, 0x65, 0x20,
0x74,
    0x68, 0x69, 0x73, 0x20, 0x41, 0x50, 0x49, 0x20, 0x74, 0x6f, 0x20, 0x75, 0x70, 0x64, 0x61,
0x74,
    0x65, 0x20, 0x75, 0x73, 0x65, 0x72, 0x82, 0xd3, 0xe4, 0x93, 0x02, 0x14, 0x32, 0x0f, 0x2f,
0x76,
    0x31, 0x2f, 0x75, 0x70, 0x64, 0x61, 0x74, 0x65, 0x5f, 0x75, 0x73, 0x65, 0x72, 0x3a, 0x01,
0x2a,
    0x12, 0xa3, 0x01, 0x0a, 0x09, 0x4c, 0x6f, 0x67, 0x69, 0x6e, 0x55, 0x73, 0x65, 0x72, 0x12,
0x14,
    0x2e, 0x70, 0x62, 0x2e, 0x4c, 0x6f, 0x67, 0x69, 0x6e, 0x55, 0x73, 0x65, 0x72, 0x52, 0x65,
0x71,
    0x75, 0x65, 0x73, 0x74, 0x1a, 0x15, 0x2e, 0x70, 0x62, 0x2e, 0x4c, 0x6f, 0x67, 0x69, 0x6e,
0x55,
    0x73, 0x65, 0x72, 0x52, 0x65, 0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x22, 0x69, 0x92, 0x41,
0x4d,
    0x12, 0x0a, 0x4c, 0x6f, 0x67, 0x69, 0x6e, 0x20, 0x75, 0x73, 0x65, 0x72, 0x1a, 0x3f, 0x55,
0x73,
    0x65, 0x20, 0x74, 0x68, 0x69, 0x73, 0x20, 0x41, 0x50, 0x49, 0x20, 0x74, 0x6f, 0x20, 0x6c,
0x6f,
    0x67, 0x69, 0x6e, 0x20, 0x75, 0x73, 0x65, 0x72, 0x20, 0x61, 0x6e, 0x64, 0x20, 0x67, 0x65,
0x74,
    0x20, 0x61, 0x63, 0x63, 0x65, 0x73, 0x73, 0x20, 0x74, 0x6f, 0x6b, 0x65, 0x6e, 0x20, 0x26,
0x20,
    0x72, 0x65, 0x66, 0x72, 0x65, 0x73, 0x68, 0x20, 0x74, 0x6f, 0x6b, 0x65, 0x6e, 0x82, 0xd3,
0xe4,
    0x93, 0x02, 0x13, 0x22, 0x0e, 0x2f, 0x76, 0x31, 0x2f, 0x6c, 0x6f, 0x67, 0x69, 0x6e, 0x5f,
0x75,
    0x73, 0x65, 0x72, 0x3a, 0x01, 0x2a, 0x12, 0x96, 0x01, 0x0a, 0x0b, 0x56, 0x65, 0x72, 0x69,
0x66,
    0x79, 0x45, 0x6d, 0x61, 0x69, 0x6c, 0x12, 0x16, 0x2e, 0x70, 0x62, 0x2e, 0x56, 0x65, 0x72,
0x69,
    0x66, 0x79, 0x45, 0x6d, 0x61, 0x69, 0x6c, 0x52, 0x65, 0x71, 0x75, 0x65, 0x73, 0x74, 0x1a,
0x17,
    0x2e, 0x70, 0x62, 0x2e, 0x56, 0x65, 0x72, 0x69, 0x66, 0x79, 0x45, 0x6d, 0x61, 0x69, 0x6c,
0x52,
    0x65, 0x73, 0x70, 0x6f, 0x6e, 0x73, 0x65, 0x22, 0x56, 0x92, 0x41, 0x3b, 0x12, 0x0c, 0x56,
0x65,
    0x72, 0x69, 0x66, 0x79, 0x20, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x1a, 0x2b, 0x55, 0x73, 0x65,
0x20,
    0x74, 0x68, 0x69, 0x73, 0x20, 0x41, 0x50, 0x49, 0x20, 0x74, 0x6f, 0x20, 0x76, 0x65, 0x72,
0x69,
    0x66, 0x79, 0x20, 0x75, 0x73, 0x65, 0x72, 0x27, 0x73, 0x20, 0x65, 0x6d, 0x61, 0x69, 0x6c,
0x20,
    0x61, 0x64, 0x64, 0x72, 0x65, 0x73, 0x73, 0x82, 0xd3, 0xe4, 0x93, 0x02, 0x12, 0x12, 0x10,
0x2f,
    0x76, 0x31, 0x2f, 0x76, 0x65, 0x72, 0x69, 0x66, 0x79, 0x5f, 0x65, 0x6d, 0x61, 0x69, 0x6c,
0x42,
    0x89, 0x01, 0x5a, 0x23, 0x67, 0x69, 0x74, 0x68, 0x75, 0x62, 0x2e, 0x63, 0x6f, 0x6d, 0x2f,
0x74,
    0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x2f, 0x73, 0x69, 0x6d, 0x70, 0x6c,
0x65,
    0x62, 0x61, 0x6e, 0x6b, 0x2f, 0x70, 0x62, 0x92, 0x41, 0x61, 0x12, 0x5f, 0x0a, 0x0f, 0x53,
0x69,
    0x6d, 0x70, 0x6c, 0x65, 0x20, 0x42, 0x61, 0x6e, 0x6b, 0x20, 0x41, 0x50, 0x49, 0x22, 0x47,
0x0a,
    0x0b, 0x54, 0x65, 0x63, 0x68, 0x20, 0x53, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x12, 0x1d, 0x68,
0x74,
    0x74, 0x70, 0x73, 0x3a, 0x2f, 0x2f, 0x67, 0x69, 0x74, 0x68, 0x75, 0x62, 0x2e, 0x63, 0x6f,
0x6d,
    0x2f, 0x74, 0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x1a, 0x19, 0x74, 0x65,
0x63,
    0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x2e, 0x67, 0x75, 0x72, 0x75, 0x40, 0x67, 0x6d,
0x61,
    0x69, 0x6c, 0x2e, 0x63, 0x6f, 0x6d, 0x32, 0x03, 0x31, 0x2e, 0x32, 0x62, 0x06, 0x70, 0x72,
0x6f,
    0x74, 0x6f, 0x33,
}

var file_service_simple_bank_proto_goTypes = []interface{}{
    (*CreateUserRequest)(nil),   // 0: pb.CreateUserRequest
    (*UpdateUserRequest)(nil),   // 1: pb.UpdateUserRequest
    (*LoginUserRequest)(nil),    // 2: pb.LoginUserRequest
    (*VerifyEmailRequest)(nil),  // 3: pb.VerifyEmailRequest
    (*CreateUserResponse)(nil),  // 4: pb.CreateUserResponse
    (*UpdateUserResponse)(nil),  // 5: pb.UpdateUserResponse
    (*LoginUserResponse)(nil),   // 6: pb.LoginUserResponse
    (*VerifyEmailResponse)(nil), // 7: pb.VerifyEmailResponse
}
var file_service_simple_bank_proto_depIdxs = []int32{
    0, // 0: pb.SimpleBank.CreateUser:input_type -> pb.CreateUserRequest
    1, // 1: pb.SimpleBank.UpdateUser:input_type -> pb.UpdateUserRequest
    2, // 2: pb.SimpleBank.LoginUser:input_type -> pb.LoginUserRequest
    3, // 3: pb.SimpleBank.VerifyEmail:input_type -> pb.VerifyEmailRequest
    4, // 4: pb.SimpleBank.CreateUser:output_type -> pb.CreateUserResponse
    5, // 5: pb.SimpleBank.UpdateUser:output_type -> pb.UpdateUserResponse
    6, // 6: pb.SimpleBank.LoginUser:output_type -> pb.LoginUserResponse
    7, // 7: pb.SimpleBank.VerifyEmail:output_type -> pb.VerifyEmailResponse
    4, // [4:8] is the sub-list for method output_type
    0, // [0:4] is the sub-list for method input_type
    0, // [0:0] is the sub-list for extension type_name
    0, // [0:0] is the sub-list for extension extendee
    0, // [0:0] is the sub-list for field type_name
}

func init() { file_service_simple_bank_proto_init() }
func file_service_simple_bank_proto_init() {
    if File_service_simple_bank_proto != nil {
        return
    }
    file_rpc_create_user_proto_init()
    file_rpc_update_user_proto_init()
    file_rpc_login_user_proto_init()
    file_rpc_verify_email_proto_init()
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_service_simple_bank_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   0,
            NumExtensions: 0,
            NumServices:   1,
        },
        GoTypes:           file_service_simple_bank_proto_goTypes,
        DependencyIndexes: file_service_simple_bank_proto_depIdxs,
    }.Build()
    File_service_simple_bank_proto = out.File
    file_service_simple_bank_proto_rawDesc = nil
    file_service_simple_bank_proto_goTypes = nil
    file_service_simple_bank_proto_depIdxs = nil
}
```

FILE: simplebank/pb/service_simple_bank.pb.gw.go

```
// Code generated by protoc-gen-grpc-gateway. DO NOT EDIT.
// source: service_simple_bank.proto

/*
Package pb is a reverse proxy.

It translates gRPC into RESTful JSON APIs.
*/
package pb

import (
    "context"
    "io"
    "net/http"

    "github.com/grpc-ecosystem/grpc-gateway/v2/runtime"
    "github.com/grpc-ecosystem/grpc-gateway/v2/utilities"
    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/grpclog"
    "google.golang.org/grpc/metadata"
    "google.golang.org/grpc/status"
    "google.golang.org/protobuf/proto"
)

// Suppress "imported and not used" errors
var _ codes.Code
var _ io.Reader
var _ status.Status
var _ = runtime.String
var _ = utilities.NewDoubleArray
var _ = metadata.Join

func request_SimpleBank_CreateUser_0(ctx context.Context, marshaler runtime.Marshaler, client
SimpleBankClient, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMetadata, ... [LINE TRUNCATED]
    var protoReq CreateUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := client.CreateUser(ctx, &protoReq, grpc.Header(&metadata.HeaderMD),
grpc.Trailer(&metadata.TrailerMD))
    return msg, metadata, err

}

func local_request_SimpleBank_CreateUser_0(ctx context.Context, marshaler runtime.Marshaler,
server SimpleBankServer, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMet ... [LINE TRUNCATED]
    var protoReq CreateUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := server.CreateUser(ctx, &protoReq)
    return msg, metadata, err

}

func request_SimpleBank_UpdateUser_0(ctx context.Context, marshaler runtime.Marshaler, client
SimpleBankClient, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMetadata, ... [LINE TRUNCATED]
    var protoReq UpdateUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := client.UpdateUser(ctx, &protoReq, grpc.Header(&metadata.HeaderMD),
grpc.Trailer(&metadata.TrailerMD))
    return msg, metadata, err
```

```
}

func local_request_SimpleBank_UpdateUser_0(ctx context.Context, marshaler runtime.Marshaler,
server SimpleBankServer, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMet ... [LINE TRUNCATED]
    var protoReq UpdateUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := server.UpdateUser(ctx, &protoReq)
    return msg, metadata, err

}

func request_SimpleBank_LoginUser_0(ctx context.Context, marshaler runtime.Marshaler, client
SimpleBankClient, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMetadata,  ... [LINE TRUNCATED]
    var protoReq LoginUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := client.LoginUser(ctx, &protoReq, grpc.Header(&metadata.HeaderMD),
grpc.Trailer(&metadata.TrailerMD))
    return msg, metadata, err

}

func local_request_SimpleBank_LoginUser_0(ctx context.Context, marshaler runtime.Marshaler,
server SimpleBankServer, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMeta ... [LINE TRUNCATED]
    var protoReq LoginUserRequest
    var metadata runtime.ServerMetadata

    newReader, berr := utilities.IOReaderFactory(req.Body)
    if berr != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", berr)
    }
    if err := marshaler.NewDecoder(newReader()).Decode(&protoReq); err != nil && err != io.EOF
{
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := server.LoginUser(ctx, &protoReq)
    return msg, metadata, err

}

var (
    filter_SimpleBank_VerifyEmail_0 = &utilities.DoubleArray{Encoding: map[string]int{}, Base:
[]int(nil), Check: []int(nil)}
)

func request_SimpleBank_VerifyEmail_0(ctx context.Context, marshaler runtime.Marshaler, client
SimpleBankClient, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMetadata ... [LINE TRUNCATED]
    var protoReq VerifyEmailRequest
    var metadata runtime.ServerMetadata

    if err := req.ParseForm(); err != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }
    if err := runtime.PopulateQueryParameters(&protoReq, req.Form,
filter_SimpleBank_VerifyEmail_0); err != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := client.VerifyEmail(ctx, &protoReq, grpc.Header(&metadata.HeaderMD),
grpc.Trailer(&metadata.TrailerMD))
    return msg, metadata, err

}

func local_request_SimpleBank_VerifyEmail_0(ctx context.Context, marshaler runtime.Marshaler,
server SimpleBankServer, req *http.Request, pathParams map[string]string) (proto.Message,
runtime.ServerMe ... [LINE TRUNCATED]
    var protoReq VerifyEmailRequest
    var metadata runtime.ServerMetadata

    if err := req.ParseForm(); err != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }
    if err := runtime.PopulateQueryParameters(&protoReq, req.Form,
filter_SimpleBank_VerifyEmail_0); err != nil {
        return nil, metadata, status.Errorf(codes.InvalidArgument, "%v", err)
    }

    msg, err := server.VerifyEmail(ctx, &protoReq)
    return msg, metadata, err

}

// RegisterSimpleBankHandlerServer registers the http handlers for service SimpleBank to "mux".
// UnaryRPC     :call SimpleBankServer directly.
// StreamingRPC :currently unsupported pending https://github.com/grpc/grpc-go/issues/906.
// Note that using this registration option will cause many gRPC library features to stop
working. Consider using RegisterSimpleBankHandlerFromEndpoint instead.
func RegisterSimpleBankHandlerServer(ctx context.Context, mux *runtime.ServeMux, server
SimpleBankServer) error {

    mux.Handle("POST", pattern_SimpleBank_CreateUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        var stream runtime.ServerTransportStream
        ctx = grpc.NewContextWithServerTransportStream(ctx, &stream)
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateIncomingContext(ctx, mux, req, "/pb.SimpleBank/CreateUser",
runtime.WithHTTPPathPattern("/v1/create_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := local_request_SimpleBank_CreateUser_0(ctx, inboundMarshaler, server,
req, pathParams)
        md.HeaderMD, md.TrailerMD = metadata.Join(md.HeaderMD, stream.Header()),
metadata.Join(md.TrailerMD, stream.Trailer())
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_CreateUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("PATCH", pattern_SimpleBank_UpdateUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        var stream runtime.ServerTransportStream
        ctx = grpc.NewContextWithServerTransportStream(ctx, &stream)
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateIncomingContext(ctx, mux, req, "/pb.SimpleBank/UpdateUser",
runtime.WithHTTPPathPattern("/v1/update_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := local_request_SimpleBank_UpdateUser_0(ctx, inboundMarshaler, server,
req, pathParams)
        md.HeaderMD, md.TrailerMD = metadata.Join(md.HeaderMD, stream.Header()),
metadata.Join(md.TrailerMD, stream.Trailer())
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_UpdateUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("POST", pattern_SimpleBank_LoginUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        var stream runtime.ServerTransportStream
        ctx = grpc.NewContextWithServerTransportStream(ctx, &stream)
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateIncomingContext(ctx, mux, req, "/pb.SimpleBank/LoginUser",
runtime.WithHTTPPathPattern("/v1/login_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := local_request_SimpleBank_LoginUser_0(ctx, inboundMarshaler, server,
req, pathParams)
        md.HeaderMD, md.TrailerMD = metadata.Join(md.HeaderMD, stream.Header()),
metadata.Join(md.TrailerMD, stream.Trailer())
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_LoginUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("GET", pattern_SimpleBank_VerifyEmail_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        var stream runtime.ServerTransportStream
        ctx = grpc.NewContextWithServerTransportStream(ctx, &stream)
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateIncomingContext(ctx, mux, req, "/pb.SimpleBank/VerifyEmail",
runtime.WithHTTPPathPattern("/v1/verify_email"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
```

```
        }
        resp, md, err := local_request_SimpleBank_VerifyEmail_0(ctx, inboundMarshaler, server,
req, pathParams)
        md.HeaderMD, md.TrailerMD = metadata.Join(md.HeaderMD, stream.Header()),
metadata.Join(md.TrailerMD, stream.Trailer())
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_VerifyEmail_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    return nil
}

// RegisterSimpleBankHandlerFromEndpoint is same as RegisterSimpleBankHandler but
// automatically dials to "endpoint" and closes the connection when "ctx" gets done.
func RegisterSimpleBankHandlerFromEndpoint(ctx context.Context, mux *runtime.ServeMux, endpoint
string, opts []grpc.DialOption) (err error) {
    conn, err := grpc.Dial(endpoint, opts...)
    if err != nil {
        return err
    }
    defer func() {
        if err != nil {
            if cerr := conn.Close(); cerr != nil {
                grpclog.Infof("Failed to close conn to %s: %v", endpoint, cerr)
            }
            return
        }
        go func() {
            <-ctx.Done()
            if cerr := conn.Close(); cerr != nil {
                grpclog.Infof("Failed to close conn to %s: %v", endpoint, cerr)
            }
        }()
    }()

    return RegisterSimpleBankHandler(ctx, mux, conn)
}

// RegisterSimpleBankHandler registers the http handlers for service SimpleBank to "mux".
// The handlers forward requests to the grpc endpoint over "conn".
func RegisterSimpleBankHandler(ctx context.Context, mux *runtime.ServeMux, conn
*grpc.ClientConn) error {
    return RegisterSimpleBankHandlerClient(ctx, mux, NewSimpleBankClient(conn))
}

// RegisterSimpleBankHandlerClient registers the http handlers for service SimpleBank
// to "mux". The handlers forward requests to the grpc endpoint over the given implementation
of "SimpleBankClient".
// Note: the gRPC framework executes interceptors within the gRPC handler. If the passed in
"SimpleBankClient"
// doesn't go through the normal gRPC flow (creating a gRPC client etc.) then it will be up to
the passed in
// "SimpleBankClient" to call the correct interceptors.
func RegisterSimpleBankHandlerClient(ctx context.Context, mux *runtime.ServeMux, client
SimpleBankClient) error {

    mux.Handle("POST", pattern_SimpleBank_CreateUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateContext(ctx, mux, req, "/pb.SimpleBank/CreateUser",
runtime.WithHTTPPathPattern("/v1/create_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := request_SimpleBank_CreateUser_0(ctx, inboundMarshaler, client, req,
pathParams)
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_CreateUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("PATCH", pattern_SimpleBank_UpdateUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateContext(ctx, mux, req, "/pb.SimpleBank/UpdateUser",
runtime.WithHTTPPathPattern("/v1/update_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := request_SimpleBank_UpdateUser_0(ctx, inboundMarshaler, client, req,
pathParams)
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_UpdateUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("POST", pattern_SimpleBank_LoginUser_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateContext(ctx, mux, req, "/pb.SimpleBank/LoginUser",
runtime.WithHTTPPathPattern("/v1/login_user"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := request_SimpleBank_LoginUser_0(ctx, inboundMarshaler, client, req,
pathParams)
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_LoginUser_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    mux.Handle("GET", pattern_SimpleBank_VerifyEmail_0, func(w http.ResponseWriter, req
*http.Request, pathParams map[string]string) {
        ctx, cancel := context.WithCancel(req.Context())
        defer cancel()
        inboundMarshaler, outboundMarshaler := runtime.MarshalerForRequest(mux, req)
        var err error
        ctx, err = runtime.AnnotateContext(ctx, mux, req, "/pb.SimpleBank/VerifyEmail",
runtime.WithHTTPPathPattern("/v1/verify_email"))
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }
        resp, md, err := request_SimpleBank_VerifyEmail_0(ctx, inboundMarshaler, client, req,
pathParams)
        ctx = runtime.NewServerMetadataContext(ctx, md)
        if err != nil {
            runtime.HTTPError(ctx, mux, outboundMarshaler, w, req, err)
            return
        }

        forward_SimpleBank_VerifyEmail_0(ctx, mux, outboundMarshaler, w, req, resp,
mux.GetForwardResponseOptions()...)

    })

    return nil
}

var (
    pattern_SimpleBank_CreateUser_0 = runtime.MustPattern(runtime.NewPattern(1, []int{2, 0, 2,
1}, []string{"v1", "create_user"}, ""))

    pattern_SimpleBank_UpdateUser_0 = runtime.MustPattern(runtime.NewPattern(1, []int{2, 0, 2,
1}, []string{"v1", "update_user"}, ""))

    pattern_SimpleBank_LoginUser_0 = runtime.MustPattern(runtime.NewPattern(1, []int{2, 0, 2,
1}, []string{"v1", "login_user"}, ""))

    pattern_SimpleBank_VerifyEmail_0 = runtime.MustPattern(runtime.NewPattern(1, []int{2, 0, 2,
1}, []string{"v1", "verify_email"}, ""))
)

var (
    forward_SimpleBank_CreateUser_0 = runtime.ForwardResponseMessage

    forward_SimpleBank_UpdateUser_0 = runtime.ForwardResponseMessage

    forward_SimpleBank_LoginUser_0 = runtime.ForwardResponseMessage

    forward_SimpleBank_VerifyEmail_0 = runtime.ForwardResponseMessage
)
```

FILE: simplebank/pb/service_simple_bank_grpc.pb.go

```
// Code generated by protoc-gen-go-grpc. DO NOT EDIT.
// versions:
// - protoc-gen-go-grpc v1.2.0
// - protoc             v4.24.3
// source: service_simple_bank.proto

package pb

import (
    context "context"
    grpc "google.golang.org/grpc"
    codes "google.golang.org/grpc/codes"
    status "google.golang.org/grpc/status"
)

// This is a compile-time assertion to ensure that this generated file
// is compatible with the grpc package it is being compiled against.
// Requires gRPC-Go v1.32.0 or later.
const _ = grpc.SupportPackageIsVersion7

// SimpleBankClient is the client API for SimpleBank service.
//
// For semantics around ctx use and closing/ending streaming RPCs, please refer to
https://pkg.go.dev/google.golang.org/grpc/?tab=doc#ClientConn.NewStream.
type SimpleBankClient interface {
```

```
    CreateUser(ctx context.Context, in *CreateUserRequest, opts ...grpc.CallOption)
(*CreateUserResponse, error)
    UpdateUser(ctx context.Context, in *UpdateUserRequest, opts ...grpc.CallOption)
(*UpdateUserResponse, error)
    LoginUser(ctx context.Context, in *LoginUserRequest, opts ...grpc.CallOption)
(*LoginUserResponse, error)
    VerifyEmail(ctx context.Context, in *VerifyEmailRequest, opts ...grpc.CallOption)
(*VerifyEmailResponse, error)
}

type simpleBankClient struct {
    cc grpc.ClientConnInterface
}

func NewSimpleBankClient(cc grpc.ClientConnInterface) SimpleBankClient {
    return &simpleBankClient{cc}
}

func (c *simpleBankClient) CreateUser(ctx context.Context, in *CreateUserRequest, opts
...grpc.CallOption) (*CreateUserResponse, error) {
    out := new(CreateUserResponse)
    err := c.cc.Invoke(ctx, "/pb.SimpleBank/CreateUser", in, out, opts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

func (c *simpleBankClient) UpdateUser(ctx context.Context, in *UpdateUserRequest, opts
...grpc.CallOption) (*UpdateUserResponse, error) {
    out := new(UpdateUserResponse)
    err := c.cc.Invoke(ctx, "/pb.SimpleBank/UpdateUser", in, out, opts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

func (c *simpleBankClient) LoginUser(ctx context.Context, in *LoginUserRequest, opts
...grpc.CallOption) (*LoginUserResponse, error) {
    out := new(LoginUserResponse)
    err := c.cc.Invoke(ctx, "/pb.SimpleBank/LoginUser", in, out, opts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

func (c *simpleBankClient) VerifyEmail(ctx context.Context, in *VerifyEmailRequest, opts
...grpc.CallOption) (*VerifyEmailResponse, error) {
    out := new(VerifyEmailResponse)
    err := c.cc.Invoke(ctx, "/pb.SimpleBank/VerifyEmail", in, out, opts...)
    if err != nil {
        return nil, err
    }
    return out, nil
}

// SimpleBankServer is the server API for SimpleBank service.
// All implementations must embed UnimplementedSimpleBankServer
// for forward compatibility
type SimpleBankServer interface {
    CreateUser(context.Context, *CreateUserRequest) (*CreateUserResponse, error)
    UpdateUser(context.Context, *UpdateUserRequest) (*UpdateUserResponse, error)
    LoginUser(context.Context, *LoginUserRequest) (*LoginUserResponse, error)
    VerifyEmail(context.Context, *VerifyEmailRequest) (*VerifyEmailResponse, error)
    mustEmbedUnimplementedSimpleBankServer()
}

// UnimplementedSimpleBankServer must be embedded to have forward compatible implementations.
type UnimplementedSimpleBankServer struct {
}

func (UnimplementedSimpleBankServer) CreateUser(context.Context, *CreateUserRequest)
(*CreateUserResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method CreateUser not implemented")
}
func (UnimplementedSimpleBankServer) UpdateUser(context.Context, *UpdateUserRequest)
(*UpdateUserResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method UpdateUser not implemented")
}
func (UnimplementedSimpleBankServer) LoginUser(context.Context, *LoginUserRequest)
(*LoginUserResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method LoginUser not implemented")
}
func (UnimplementedSimpleBankServer) VerifyEmail(context.Context, *VerifyEmailRequest)
(*VerifyEmailResponse, error) {
    return nil, status.Errorf(codes.Unimplemented, "method VerifyEmail not implemented")
}
func (UnimplementedSimpleBankServer) mustEmbedUnimplementedSimpleBankServer() {}

// UnsafeSimpleBankServer may be embedded to opt out of forward compatibility for this service.
// Use of this interface is not recommended, as added methods to SimpleBankServer will
// result in compilation errors.
type UnsafeSimpleBankServer interface {
    mustEmbedUnimplementedSimpleBankServer()
}

func RegisterSimpleBankServer(s grpc.ServiceRegistrar, srv SimpleBankServer) {
    s.RegisterService(&SimpleBank_ServiceDesc, srv)
}

func _SimpleBank_CreateUser_Handler(srv interface{}, ctx context.Context, dec func(interface{})
error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(CreateUserRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(SimpleBankServer).CreateUser(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:     srv,
        FullMethod: "/pb.SimpleBank/CreateUser",
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(SimpleBankServer).CreateUser(ctx, req.(*CreateUserRequest))
    }
    return interceptor(ctx, in, info, handler)
}

func _SimpleBank_UpdateUser_Handler(srv interface{}, ctx context.Context, dec func(interface{})
error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(UpdateUserRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(SimpleBankServer).UpdateUser(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:     srv,
        FullMethod: "/pb.SimpleBank/UpdateUser",
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(SimpleBankServer).UpdateUser(ctx, req.(*UpdateUserRequest))
    }
    return interceptor(ctx, in, info, handler)
}

func _SimpleBank_LoginUser_Handler(srv interface{}, ctx context.Context, dec func(interface{})
error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(LoginUserRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(SimpleBankServer).LoginUser(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:     srv,
        FullMethod: "/pb.SimpleBank/LoginUser",
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(SimpleBankServer).LoginUser(ctx, req.(*LoginUserRequest))
    }
    return interceptor(ctx, in, info, handler)
}

func _SimpleBank_VerifyEmail_Handler(srv interface{}, ctx context.Context, dec
func(interface{}) error, interceptor grpc.UnaryServerInterceptor) (interface{}, error) {
    in := new(VerifyEmailRequest)
    if err := dec(in); err != nil {
        return nil, err
    }
    if interceptor == nil {
        return srv.(SimpleBankServer).VerifyEmail(ctx, in)
    }
    info := &grpc.UnaryServerInfo{
        Server:     srv,
        FullMethod: "/pb.SimpleBank/VerifyEmail",
    }
    handler := func(ctx context.Context, req interface{}) (interface{}, error) {
        return srv.(SimpleBankServer).VerifyEmail(ctx, req.(*VerifyEmailRequest))
    }
    return interceptor(ctx, in, info, handler)
}

// SimpleBank_ServiceDesc is the grpc.ServiceDesc for SimpleBank service.
// It's only intended for direct use with grpc.RegisterService,
// and not to be introspected or modified (even as a copy)
var SimpleBank_ServiceDesc = grpc.ServiceDesc{
    ServiceName: "pb.SimpleBank",
    HandlerType: (*SimpleBankServer)(nil),
    Methods: []grpc.MethodDesc{
        {
            MethodName: "CreateUser",
            Handler:    _SimpleBank_CreateUser_Handler,
        },
        {
            MethodName: "UpdateUser",
            Handler:    _SimpleBank_UpdateUser_Handler,
        },
        {
            MethodName: "LoginUser",
            Handler:    _SimpleBank_LoginUser_Handler,
        },
        {
            MethodName: "VerifyEmail",
            Handler:    _SimpleBank_VerifyEmail_Handler,
        },
    },
    Streams:  []grpc.StreamDesc{},
    Metadata: "service_simple_bank.proto",
}
```

FILE: simplebank/pb/user.pb.go

```
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.28.1
//     protoc        v4.24.3
// source: user.proto

package pb

import (
```

```
    protoreflect "google.golang.org/protobuf/reflect/protoreflect"
    protoimpl "google.golang.org/protobuf/runtime/protoimpl"
    timestamppb "google.golang.org/protobuf/types/known/timestamppb"
    reflect "reflect"
    sync "sync"
)

const (
    // Verify that this generated code is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(20 - protoimpl.MinVersion)
    // Verify that runtime/protoimpl is sufficiently up-to-date.
    _ = protoimpl.EnforceVersion(protoimpl.MaxVersion - 20)
)

type User struct {
    state         protoimpl.MessageState
    sizeCache     protoimpl.SizeCache
    unknownFields protoimpl.UnknownFields

    Username          string                 `protobuf:"bytes,1,opt,name=username,proto3"
json:"username,omitempty"`
    FullName          string
`protobuf:"bytes,2,opt,name=full_name,json=fullName,proto3" json:"full_name,omitempty"`
    Email             string                 `protobuf:"bytes,3,opt,name=email,proto3"
json:"email,omitempty"`
    PasswordChangedAt *timestamppb.Timestamp
`protobuf:"bytes,4,opt,name=password_changed_at,json=passwordChangedAt,proto3"
json:"password_changed_at,omitempty"`
    CreatedAt         *timestamppb.Timestamp
`protobuf:"bytes,5,opt,name=created_at,json=createdAt,proto3" json:"created_at,omitempty"`
}

func (x *User) Reset() {
    *x = User{}
    if protoimpl.UnsafeEnabled {
        mi := &file_user_proto_msgTypes[0]
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        ms.StoreMessageInfo(mi)
    }
}

func (x *User) String() string {
    return protoimpl.X.MessageStringOf(x)
}

func (*User) ProtoMessage() {}

func (x *User) ProtoReflect() protoreflect.Message {
    mi := &file_user_proto_msgTypes[0]
    if protoimpl.UnsafeEnabled && x != nil {
        ms := protoimpl.X.MessageStateOf(protoimpl.Pointer(x))
        if ms.LoadMessageInfo() == nil {
            ms.StoreMessageInfo(mi)
        }
        return ms
    }
    return mi.MessageOf(x)
}

// Deprecated: Use User.ProtoReflect.Descriptor instead.
func (*User) Descriptor() ([]byte, []int) {
    return file_user_proto_rawDescGZIP(), []int{0}
}

func (x *User) GetUsername() string {
    if x != nil {
        return x.Username
    }
    return ""
}

func (x *User) GetFullName() string {
    if x != nil {
        return x.FullName
    }
    return ""
}

func (x *User) GetEmail() string {
    if x != nil {
        return x.Email
    }
    return ""
}

func (x *User) GetPasswordChangedAt() *timestamppb.Timestamp {
    if x != nil {
        return x.PasswordChangedAt
    }
    return nil
}

func (x *User) GetCreatedAt() *timestamppb.Timestamp {
    if x != nil {
        return x.CreatedAt
    }
    return nil
}

var File_user_proto protoreflect.FileDescriptor

var file_user_proto_rawDesc = []byte{
    0x0a, 0x0a, 0x75, 0x73, 0x65, 0x72, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x12, 0x02, 0x70,
0x62,
    0x1a, 0x1f, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65, 0x2f, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x62,
0x75,
    0x66, 0x2f, 0x74, 0x69, 0x6d, 0x65, 0x73, 0x74, 0x61, 0x6d, 0x70, 0x2e, 0x70, 0x72, 0x6f,
0x74,
    0x6f, 0x22, 0xdc, 0x01, 0x0a, 0x04, 0x55, 0x73, 0x65, 0x72, 0x12, 0x1a, 0x0a, 0x08, 0x75,
0x73,
    0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x18, 0x01, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08, 0x75,
0x73,
    0x65, 0x72, 0x6e, 0x61, 0x6d, 0x65, 0x12, 0x1b, 0x0a, 0x09, 0x66, 0x75, 0x6c, 0x6c, 0x5f,
0x6e,
    0x61, 0x6d, 0x65, 0x18, 0x02, 0x20, 0x01, 0x28, 0x09, 0x52, 0x08, 0x66, 0x75, 0x6c, 0x6c,
0x4e,
    0x61, 0x6d, 0x65, 0x12, 0x14, 0x0a, 0x05, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x18, 0x03, 0x20,
0x01,
    0x28, 0x09, 0x52, 0x05, 0x65, 0x6d, 0x61, 0x69, 0x6c, 0x12, 0x4a, 0x0a, 0x13, 0x70, 0x61,
0x73,
    0x73, 0x77, 0x6f, 0x72, 0x64, 0x5f, 0x63, 0x68, 0x61, 0x6e, 0x67, 0x65, 0x64, 0x5f, 0x61,
0x74,
    0x18, 0x04, 0x20, 0x01, 0x28, 0x0b, 0x32, 0x1a, 0x2e, 0x67, 0x6f, 0x6f, 0x67, 0x6c, 0x65,
0x2e,
    0x70, 0x72, 0x6f, 0x74, 0x6f, 0x62, 0x75, 0x66, 0x2e, 0x54, 0x69, 0x6d, 0x65, 0x73, 0x74,
0x61,
    0x6d, 0x70, 0x52, 0x11, 0x70, 0x61, 0x73, 0x73, 0x77, 0x6f, 0x72, 0x64, 0x43, 0x68, 0x61,
0x6e,
    0x67, 0x65, 0x64, 0x41, 0x74, 0x12, 0x39, 0x0a, 0x0a, 0x63, 0x72, 0x65, 0x61, 0x74, 0x65,
0x64,
    0x5f, 0x61, 0x74, 0x18, 0x05, 0x20, 0x01, 0x28, 0x0b, 0x32, 0x1a, 0x2e, 0x67, 0x6f, 0x6f,
0x67,
    0x6c, 0x65, 0x2e, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x62, 0x75, 0x66, 0x2e, 0x54, 0x69, 0x6d,
0x65,
    0x73, 0x74, 0x61, 0x6d, 0x70, 0x52, 0x09, 0x63, 0x72, 0x65, 0x61, 0x74, 0x65, 0x64, 0x41,
0x74,
    0x42, 0x25, 0x5a, 0x23, 0x67, 0x69, 0x74, 0x68, 0x75, 0x62, 0x2e, 0x63, 0x6f, 0x6d, 0x2f,
0x74,
    0x65, 0x63, 0x68, 0x73, 0x63, 0x68, 0x6f, 0x6f, 0x6c, 0x2f, 0x73, 0x69, 0x6d, 0x70, 0x6c,
0x65,
    0x62, 0x61, 0x6e, 0x6b, 0x2f, 0x70, 0x62, 0x62, 0x06, 0x70, 0x72, 0x6f, 0x74, 0x6f, 0x33,
}

var (
    file_user_proto_rawDescOnce sync.Once
    file_user_proto_rawDescData = file_user_proto_rawDesc
)

func file_user_proto_rawDescGZIP() []byte {
    file_user_proto_rawDescOnce.Do(func() {
        file_user_proto_rawDescData = protoimpl.X.CompressGZIP(file_user_proto_rawDescData)
    })
    return file_user_proto_rawDescData
}

var file_user_proto_msgTypes = make([]protoimpl.MessageInfo, 1)
var file_user_proto_goTypes = []interface{}{
    (*User)(nil),                  // 0: pb.User
    (*timestamppb.Timestamp)(nil), // 1: google.protobuf.Timestamp
}
var file_user_proto_depIdxs = []int32{
    1, // 0: pb.User.password_changed_at:type_name -> google.protobuf.Timestamp
    1, // 1: pb.User.created_at:type_name -> google.protobuf.Timestamp
    2, // [2:2] is the sub-list for method output_type
    2, // [2:2] is the sub-list for method input_type
    2, // [2:2] is the sub-list for extension type_name
    2, // [2:2] is the sub-list for extension extendee
    0, // [0:2] is the sub-list for field type_name
}

func init() { file_user_proto_init() }
func file_user_proto_init() {
    if File_user_proto != nil {
        return
    }
    if !protoimpl.UnsafeEnabled {
        file_user_proto_msgTypes[0].Exporter = func(v interface{}, i int) interface{} {
            switch v := v.(*User); i {
            case 0:
                return &v.state
            case 1:
                return &v.sizeCache
            case 2:
                return &v.unknownFields
            default:
                return nil
            }
        }
    }
    type x struct{}
    out := protoimpl.TypeBuilder{
        File: protoimpl.DescBuilder{
            GoPackagePath: reflect.TypeOf(x{}).PkgPath(),
            RawDescriptor: file_user_proto_rawDesc,
            NumEnums:      0,
            NumMessages:   1,
            NumExtensions: 0,
            NumServices:   0,
        },
        GoTypes:           file_user_proto_goTypes,
        DependencyIndexes: file_user_proto_depIdxs,
        MessageInfos:      file_user_proto_msgTypes,
    }.Build()
    File_user_proto = out.File
    file_user_proto_rawDesc = nil
    file_user_proto_goTypes = nil
    file_user_proto_depIdxs = nil
}
```

FILE: simplebank/token/jwt_maker.go

```
package token

import (
    "errors"
    "fmt"
    "time"
```

```
    "github.com/golang-jwt/jwt/v5"
)

const minSecretKeySize = 32

// JWTMaker is a JSON Web Token maker
type JWTMaker struct {
    secretKey string
}

// NewJWTMaker creates a new JWTMaker
func NewJWTMaker(secretKey string) (Maker, error) {
    if len(secretKey) < minSecretKeySize {
        return nil, fmt.Errorf("invalid key size: must be at least %d characters",
minSecretKeySize)
    }
    return &JWTMaker{secretKey}, nil
}

// CreateToken creates a new token for a specific username and duration
func (maker *JWTMaker) CreateToken(username string, role string, duration time.Duration,
tokenType TokenType) (string, *Payload, error) {
    payload, err := NewPayload(username, role, duration, tokenType)
    if err != nil {
        return "", payload, err
    }

    jwtToken := jwt.NewWithClaims(jwt.SigningMethodHS256, payload)
    token, err := jwtToken.SignedString([]byte(maker.secretKey))
    return token, payload, err
}

// VerifyToken checks if the token is valid or not
func (maker *JWTMaker) VerifyToken(token string, tokenType TokenType) (*Payload, error) {
    keyFunc := func(token *jwt.Token) (interface{}, error) {
        _, ok := token.Method.(*jwt.SigningMethodHMAC)
        if !ok {
            return nil, ErrInvalidToken
        }
        return []byte(maker.secretKey), nil
    }

    jwtToken, err := jwt.ParseWithClaims(token, &Payload{}, keyFunc)
    if err != nil {
        if errors.Is(err, jwt.ErrTokenExpired) {
            return nil, ErrExpiredToken
        }
        return nil, ErrInvalidToken
    }

    payload, ok := jwtToken.Claims.(*Payload)
    if !ok {
        return nil, ErrInvalidToken
    }

    err = payload.Valid(tokenType)
    if err != nil {
        return nil, err
    }

    return payload, nil
}
```

FILE: simplebank/token/jwt_maker_test.go

```
package token

import (
    "testing"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func TestJWTMaker(t *testing.T) {
    maker, err := NewJWTMaker(util.RandomString(32))
    require.NoError(t, err)

    username := util.RandomOwner()
    role := util.DepositorRole
    duration := time.Minute

    issuedAt := time.Now()
    expiredAt := issuedAt.Add(duration)

    token, payload, err := maker.CreateToken(username, role, duration, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)

    require.NotZero(t, payload.ID)
    require.Equal(t, username, payload.Username)
    require.Equal(t, role, payload.Role)
    require.WithinDuration(t, issuedAt, payload.IssuedAt, time.Second)
    require.WithinDuration(t, expiredAt, payload.ExpiredAt, time.Second)
}

func TestExpiredJWTToken(t *testing.T) {
    maker, err := NewJWTMaker(util.RandomString(32))
    require.NoError(t, err)

    token, payload, err := maker.CreateToken(util.RandomOwner(), util.DepositorRole,
-time.Minute, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeAccessToken)
    require.Error(t, err)
    require.EqualError(t, err, ErrExpiredToken.Error())
    require.Nil(t, payload)
}

func TestInvalidJWTTokenAlgNone(t *testing.T) {
    payload, err := NewPayload(util.RandomOwner(), util.DepositorRole, time.Minute,
TokenTypeAccessToken)
    require.NoError(t, err)

    jwtToken := jwt.NewWithClaims(jwt.SigningMethodNone, payload)
    token, err := jwtToken.SignedString(jwt.UnsafeAllowNoneSignatureType)
    require.NoError(t, err)

    maker, err := NewJWTMaker(util.RandomString(32))
    require.NoError(t, err)

    payload, err = maker.VerifyToken(token, TokenTypeAccessToken)
    require.Error(t, err)
    require.EqualError(t, err, ErrInvalidToken.Error())
    require.Nil(t, payload)
}

func TestJWTWrongTokenType(t *testing.T) {
    maker, err := NewJWTMaker(util.RandomString(32))
    require.NoError(t, err)

    token, payload, err := maker.CreateToken(util.RandomOwner(), util.DepositorRole,
time.Minute, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeRefreshToken)
    require.Error(t, err)
    require.EqualError(t, err, ErrInvalidToken.Error())
    require.Nil(t, payload)
}
```

FILE: simplebank/token/maker.go

```
package token

import (
    "time"
)

// Maker is an interface for managing tokens
type Maker interface {
    // CreateToken creates a new token for a specific username and duration
    CreateToken(username string, role string, duration time.Duration, tokenType TokenType)
(string, *Payload, error)

    // VerifyToken checks if the token is valid or not
    VerifyToken(token string, tokenType TokenType) (*Payload, error)
}
```

FILE: simplebank/token/paseto_maker.go

```
package token

import (
    "fmt"
    "time"

    "github.com/aead/chacha20poly1305"
    "github.com/o1egl/paseto"
)

// PasetoMaker is a PASETO token maker
type PasetoMaker struct {
    paseto       *paseto.V2
    symmetricKey []byte
}

// NewPasetoMaker creates a new PasetoMaker
func NewPasetoMaker(symmetricKey string) (Maker, error) {
    if len(symmetricKey) != chacha20poly1305.KeySize {
        return nil, fmt.Errorf("invalid key size: must be exactly %d characters",
chacha20poly1305.KeySize)
    }

    maker := &PasetoMaker{
        paseto:       paseto.NewV2(),
        symmetricKey: []byte(symmetricKey),
    }

    return maker, nil
}

// CreateToken creates a new token for a specific username and duration
func (maker *PasetoMaker) CreateToken(username string, role string, duration time.Duration,
tokenType TokenType) (string, *Payload, error) {
    payload, err := NewPayload(username, role, duration, tokenType)
    if err != nil {
        return "", payload, err
    }

    token, err := maker.paseto.Encrypt(maker.symmetricKey, payload, nil)
    return token, payload, err
}

// VerifyToken checks if the token is valid or not
func (maker *PasetoMaker) VerifyToken(token string, tokenType TokenType) (*Payload, error) {
```

```
    payload := &Payload{}

    err := maker.paseto.Decrypt(token, maker.symmetricKey, payload, nil)
    if err != nil {
        return nil, ErrInvalidToken
    }

    err = payload.Valid(tokenType)
    if err != nil {
        return nil, err
    }

    return payload, nil
}
```

FILE: simplebank/token/paseto_maker_test.go

```
package token

import (
    "testing"
    "time"

    "github.com/stretchr/testify/require"
    "github.com/techschool/simplebank/util"
)

func TestPasetoMaker(t *testing.T) {
    maker, err := NewPasetoMaker(util.RandomString(32))
    require.NoError(t, err)

    username := util.RandomOwner()
    role := util.DepositorRole
    duration := time.Minute

    issuedAt := time.Now()
    expiredAt := issuedAt.Add(duration)

    token, payload, err := maker.CreateToken(username, role, duration, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)

    require.NotZero(t, payload.ID)
    require.Equal(t, username, payload.Username)
    require.Equal(t, role, payload.Role)
    require.WithinDuration(t, issuedAt, payload.IssuedAt, time.Second)
    require.WithinDuration(t, expiredAt, payload.ExpiredAt, time.Second)
}

func TestExpiredPasetoToken(t *testing.T) {
    maker, err := NewPasetoMaker(util.RandomString(32))
    require.NoError(t, err)

    token, payload, err := maker.CreateToken(util.RandomOwner(), util.DepositorRole,
-time.Minute, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeAccessToken)
    require.Error(t, err)
    require.EqualError(t, err, ErrExpiredToken.Error())
    require.Nil(t, payload)
}

func TestPasetoWrongTokenType(t *testing.T) {
    maker, err := NewPasetoMaker(util.RandomString(32))
    require.NoError(t, err)

    token, payload, err := maker.CreateToken(util.RandomOwner(), util.DepositorRole,
time.Minute, TokenTypeAccessToken)
    require.NoError(t, err)
    require.NotEmpty(t, token)
    require.NotEmpty(t, payload)

    payload, err = maker.VerifyToken(token, TokenTypeRefreshToken)
    require.Error(t, err)
    require.EqualError(t, err, ErrInvalidToken.Error())
    require.Nil(t, payload)
}
```

FILE: simplebank/token/payload.go

```
package token

import (
    "errors"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
)

// Different types of error returned by the VerifyToken function
var (
    ErrInvalidToken = errors.New("token is invalid")
    ErrExpiredToken = errors.New("token has expired")
)

type TokenType byte

const (
    TokenTypeAccessToken  = 1
    TokenTypeRefreshToken = 2
)

// Payload contains the payload data of the token
type Payload struct {
    ID        uuid.UUID `json:"id"`
    Type      TokenType `json:"token_type"`
    Username  string    `json:"username"`
    Role      string    `json:"role"`
    IssuedAt  time.Time `json:"issued_at"`
    ExpiredAt time.Time `json:"expired_at"`
}

// NewPayload creates a new token payload with a specific username and duration
func NewPayload(username string, role string, duration time.Duration, tokenType TokenType)
(*Payload, error) {
    tokenID, err := uuid.NewRandom()
    if err != nil {
        return nil, err
    }

    payload := &Payload{
        ID:        tokenID,
        Type:      tokenType,
        Username:  username,
        Role:      role,
        IssuedAt:  time.Now(),
        ExpiredAt: time.Now().Add(duration),
    }
    return payload, nil
}

// Valid checks if the token payload is valid or not
func (payload *Payload) Valid(tokenType TokenType) error {
    if payload.Type != tokenType {
        return ErrInvalidToken
    }
    if time.Now().After(payload.ExpiredAt) {
        return ErrExpiredToken
    }
    return nil
}

func (payload *Payload) GetExpirationTime() (*jwt.NumericDate, error) {
    return &jwt.NumericDate{
        Time: payload.ExpiredAt,
    }, nil
}

func (payload *Payload) GetIssuedAt() (*jwt.NumericDate, error) {
    return &jwt.NumericDate{
        Time: payload.IssuedAt,
    }, nil
}

func (payload *Payload) GetNotBefore() (*jwt.NumericDate, error) {
    return &jwt.NumericDate{
        Time: payload.IssuedAt,
    }, nil
}

func (payload *Payload) GetIssuer() (string, error) {
    return "", nil
}

func (payload *Payload) GetSubject() (string, error) {
    return "", nil
}

func (payload *Payload) GetAudience() (jwt.ClaimStrings, error) {
    return jwt.ClaimStrings{}, nil
}
```

FILE: simplebank/util/config.go

```
package util

import (
    "time"

    "github.com/spf13/viper"
)

// Config stores all configuration of the application.
// The values are read by viper from a config file or environment variable.
type Config struct {
    Environment          string        `mapstructure:"ENVIRONMENT"`
    AllowedOrigins       []string      `mapstructure:"ALLOWED_ORIGINS"`
    DBSource             string        `mapstructure:"DB_SOURCE"`
    MigrationURL         string        `mapstructure:"MIGRATION_URL"`
    RedisAddress         string        `mapstructure:"REDIS_ADDRESS"`
    HTTPServerAddress    string        `mapstructure:"HTTP_SERVER_ADDRESS"`
    GRPCServerAddress    string        `mapstructure:"GRPC_SERVER_ADDRESS"`
    TokenSymmetricKey    string        `mapstructure:"TOKEN_SYMMETRIC_KEY"`
    AccessTokenDuration  time.Duration `mapstructure:"ACCESS_TOKEN_DURATION"`
    RefreshTokenDuration time.Duration `mapstructure:"REFRESH_TOKEN_DURATION"`
    EmailSenderName      string        `mapstructure:"EMAIL_SENDER_NAME"`
    EmailSenderAddress   string        `mapstructure:"EMAIL_SENDER_ADDRESS"`
    EmailSenderPassword  string        `mapstructure:"EMAIL_SENDER_PASSWORD"`
}

// LoadConfig reads configuration from file or environment variables.
func LoadConfig(path string) (config Config, err error) {
    viper.AddConfigPath(path)
    viper.SetConfigName("app")
    viper.SetConfigType("env")

    viper.AutomaticEnv()

    err = viper.ReadInConfig()
    if err != nil {
```

```
        return
    }

    err = viper.Unmarshal(&config)
    return
}
```

FILE: simplebank/util/currency.go

```
package util

// Constants for all supported currencies
const (
    USD = "USD"
    EUR = "EUR"
    CAD = "CAD"
)

// IsSupportedCurrency returns true if the currency is supported
func IsSupportedCurrency(currency string) bool {
    switch currency {
    case USD, EUR, CAD:
        return true
    }
    return false
}
```

FILE: simplebank/util/password.go

```
package util

import (
    "fmt"

    "golang.org/x/crypto/bcrypt"
)

// HashPassword returns the bcrypt hash of the password
func HashPassword(password string) (string, error) {
    hashedPassword, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
    if err != nil {
        return "", fmt.Errorf("failed to hash password: %w", err)
    }
    return string(hashedPassword), nil
}

// CheckPassword checks if the provided password is correct or not
func CheckPassword(password string, hashedPassword string) error {
    return bcrypt.CompareHashAndPassword([]byte(hashedPassword), []byte(password))
}
```

FILE: simplebank/util/password_test.go

```
package util

import (
    "testing"

    "github.com/stretchr/testify/require"
    "golang.org/x/crypto/bcrypt"
)

func TestPassword(t *testing.T) {
    password := RandomString(6)

    hashedPassword1, err := HashPassword(password)
    require.NoError(t, err)
    require.NotEmpty(t, hashedPassword1)

    err = CheckPassword(password, hashedPassword1)
    require.NoError(t, err)

    wrongPassword := RandomString(6)
    err = CheckPassword(wrongPassword, hashedPassword1)
    require.EqualError(t, err, bcrypt.ErrMismatchedHashAndPassword.Error())

    hashedPassword2, err := HashPassword(password)
    require.NoError(t, err)
    require.NotEmpty(t, hashedPassword2)
    require.NotEqual(t, hashedPassword1, hashedPassword2)
}
```

FILE: simplebank/util/random.go

```
package util

import (
    "fmt"
    "math/rand"
    "strings"
    "time"
)

const alphabet = "abcdefghijklmnopqrstuvwxyz"

func init() {
    rand.Seed(time.Now().UnixNano())
}

// RandomInt generates a random integer between min and max
func RandomInt(min, max int64) int64 {
    return min + rand.Int63n(max-min+1)
}

// RandomString generates a random string of length n
func RandomString(n int) string {
    var sb strings.Builder
    k := len(alphabet)

    for i := 0; i < n; i++ {
        c := alphabet[rand.Intn(k)]
        sb.WriteByte(c)
    }

    return sb.String()
}

// RandomOwner generates a random owner name
func RandomOwner() string {
    return RandomString(6)
}

// RandomMoney generates a random amount of money
func RandomMoney() int64 {
    return RandomInt(0, 1000)
}

// RandomCurrency generates a random currency code
func RandomCurrency() string {
    currencies := []string{USD, EUR, CAD}
    n := len(currencies)
    return currencies[rand.Intn(n)]
}

// RandomEmail generates a random email
func RandomEmail() string {
    return fmt.Sprintf("%s@email.com", RandomString(6))
}
```

FILE: simplebank/util/role.go

```
package util

const (
    DepositorRole = "depositor"
    BankerRole    = "banker"
)
```

FILE: simplebank/val/validator.go

```
package val

import (
    "fmt"
    "net/mail"
    "regexp"
)

var (
    isValidUsername = regexp.MustCompile(`^[a-z0-9_]+$`).MatchString
    isValidFullName = regexp.MustCompile(`^[a-zA-Z\s]+$`).MatchString
)

func ValidateString(value string, minLength int, maxLength int) error {
    n := len(value)
    if n < minLength || n > maxLength {
        return fmt.Errorf("must contain from %d-%d characters", minLength, maxLength)
    }
    return nil
}

func ValidateUsername(value string) error {
    if err := ValidateString(value, 3, 100); err != nil {
        return err
    }
    if !isValidUsername(value) {
        return fmt.Errorf("must contain only lowercase letters, digits, or underscore")
    }
    return nil
}

func ValidateFullName(value string) error {
    if err := ValidateString(value, 3, 100); err != nil {
        return err
    }
    if !isValidFullName(value) {
        return fmt.Errorf("must contain only letters or spaces")
    }
    return nil
}

func ValidatePassword(value string) error {
    return ValidateString(value, 6, 100)
}

func ValidateEmail(value string) error {
    if err := ValidateString(value, 3, 200); err != nil {
        return err
    }
    if _, err := mail.ParseAddress(value); err != nil {
        return fmt.Errorf("is not a valid email address")
    }
    return nil
}

func ValidateEmailId(value int64) error {
    if value <= 0 {
        return fmt.Errorf("must be a positive integer")
    }
    return nil
}

func ValidateSecretCode(value string) error {
    return ValidateString(value, 32, 128)
}
```

FILE: simplebank/worker/distributor.go

```
package worker

import (
    "context"
```

```
    "github.com/hibiken/asynq"
)

type TaskDistributor interface {
    DistributeTaskSendVerifyEmail(
        ctx context.Context,
        payload *PayloadSendVerifyEmail,
        opts ...asynq.Option,
    ) error
}

type RedisTaskDistributor struct {
    client *asynq.Client
}

func NewRedisTaskDistributor(redisOpt asynq.RedisClientOpt) TaskDistributor {
    client := asynq.NewClient(redisOpt)
    return &RedisTaskDistributor{
        client: client,
    }
}
```

FILE: simplebank/worker/logger.go

```
package worker

import (
    "context"
    "fmt"

    "github.com/rs/zerolog"
    "github.com/rs/zerolog/log"
)

type Logger struct{}

func NewLogger() *Logger {
    return &Logger{}
}

func (logger *Logger) Print(level zerolog.Level, args ...interface{}) {
    log.WithLevel(level).Msg(fmt.Sprint(args...))
}

func (logger *Logger) Printf(ctx context.Context, format string, v ...interface{}) {
    log.WithLevel(zerolog.DebugLevel).Msgf(format, v...)
}

func (logger *Logger) Debug(args ...interface{}) {
    logger.Print(zerolog.DebugLevel, args...)
}

func (logger *Logger) Info(args ...interface{}) {
    logger.Print(zerolog.InfoLevel, args...)
}

func (logger *Logger) Warn(args ...interface{}) {
    logger.Print(zerolog.WarnLevel, args...)
}

func (logger *Logger) Error(args ...interface{}) {
    logger.Print(zerolog.ErrorLevel, args...)
}

func (logger *Logger) Fatal(args ...interface{}) {
    logger.Print(zerolog.FatalLevel, args...)
}
```

FILE: simplebank/worker/mock/distributor.go

```
// Code generated by MockGen. DO NOT EDIT.
// Source: github.com/techschool/simplebank/worker (interfaces: TaskDistributor)

// Package mockwk is a generated GoMock package.
package mockwk

import (
    context "context"
    gomock "github.com/golang/mock/gomock"
    asynq "github.com/hibiken/asynq"
    worker "github.com/techschool/simplebank/worker"
    reflect "reflect"
)

// MockTaskDistributor is a mock of TaskDistributor interface
type MockTaskDistributor struct {
    ctrl     *gomock.Controller
    recorder *MockTaskDistributorMockRecorder
}

// MockTaskDistributorMockRecorder is the mock recorder for MockTaskDistributor
type MockTaskDistributorMockRecorder struct {
    mock *MockTaskDistributor
}

// NewMockTaskDistributor creates a new mock instance
func NewMockTaskDistributor(ctrl *gomock.Controller) *MockTaskDistributor {
    mock := &MockTaskDistributor{ctrl: ctrl}
    mock.recorder = &MockTaskDistributorMockRecorder{mock}
    return mock
}

// EXPECT returns an object that allows the caller to indicate expected use
func (m *MockTaskDistributor) EXPECT() *MockTaskDistributorMockRecorder {
    return m.recorder
}

// DistributeTaskSendVerifyEmail mocks base method
func (m *MockTaskDistributor) DistributeTaskSendVerifyEmail(arg0 context.Context, arg1
*worker.PayloadSendVerifyEmail, arg2 ...asynq.Option) error {
    m.ctrl.T.Helper()
    varargs := []interface{}{arg0, arg1}
    for _, a := range arg2 {
        varargs = append(varargs, a)
    }
    ret := m.ctrl.Call(m, "DistributeTaskSendVerifyEmail", varargs...)
    ret0, _ := ret[0].(error)
    return ret0
}

// DistributeTaskSendVerifyEmail indicates an expected call of DistributeTaskSendVerifyEmail
func (mr *MockTaskDistributorMockRecorder) DistributeTaskSendVerifyEmail(arg0, arg1
interface{}, arg2 ...interface{}) *gomock.Call {
    mr.mock.ctrl.T.Helper()
    varargs := append([]interface{}{arg0, arg1}, arg2...)
    return mr.mock.ctrl.RecordCallWithMethodType(mr.mock, "DistributeTaskSendVerifyEmail",
reflect.TypeOf((*MockTaskDistributor)(nil).DistributeTaskSendVerifyEmail), varargs...)
}
```

FILE: simplebank/worker/processor.go

```
package worker

import (
    "context"

    "github.com/go-redis/redis/v8"
    "github.com/hibiken/asynq"
    "github.com/rs/zerolog/log"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/mail"
)

const (
    QueueCritical = "critical"
    QueueDefault  = "default"
)

type TaskProcessor interface {
    Start() error
    Shutdown()
    ProcessTaskSendVerifyEmail(ctx context.Context, task *asynq.Task) error
}

type RedisTaskProcessor struct {
    server *asynq.Server
    store  db.Store
    mailer mail.EmailSender
}

func NewRedisTaskProcessor(redisOpt asynq.RedisClientOpt, store db.Store, mailer
mail.EmailSender) TaskProcessor {
    logger := NewLogger()
    redis.SetLogger(logger)

    server := asynq.NewServer(
        redisOpt,
        asynq.Config{
            Queues: map[string]int{
                QueueCritical: 10,
                QueueDefault:  5,
            },
            ErrorHandler: asynq.ErrorHandlerFunc(func(ctx context.Context, task *asynq.Task,
err error) {
                log.Error().Err(err).Str("type", task.Type()).
                    Bytes("payload", task.Payload()).Msg("process task failed")
            }),
            Logger: logger,
        },
    )

    return &RedisTaskProcessor{
        server: server,
        store:  store,
        mailer: mailer,
    }
}

func (processor *RedisTaskProcessor) Start() error {
    mux := asynq.NewServeMux()

    mux.HandleFunc(TaskSendVerifyEmail, processor.ProcessTaskSendVerifyEmail)

    return processor.server.Start(mux)
}

func (processor *RedisTaskProcessor) Shutdown() {
    processor.server.Shutdown()
}
```

FILE: simplebank/worker/task_send_verify_email.go

```
package worker

import (
    "context"
    "encoding/json"
    "fmt"

    "github.com/hibiken/asynq"
    "github.com/rs/zerolog/log"
    db "github.com/techschool/simplebank/db/sqlc"
    "github.com/techschool/simplebank/util"
)

const TaskSendVerifyEmail = "task:send_verify_email"

type PayloadSendVerifyEmail struct {
    Username string `json:"username"`
}
```

```
func (distributor *RedisTaskDistributor) DistributeTaskSendVerifyEmail(
    ctx context.Context,
    payload *PayloadSendVerifyEmail,
    opts ...asynq.Option,
) error {
    jsonPayload, err := json.Marshal(payload)
    if err != nil {
        return fmt.Errorf("failed to marshal task payload: %w", err)
    }

    task := asynq.NewTask(TaskSendVerifyEmail, jsonPayload, opts...)
    info, err := distributor.client.EnqueueContext(ctx, task)
    if err != nil {
        return fmt.Errorf("failed to enqueue task: %w", err)
    }

    log.Info().Str("type", task.Type()).Bytes("payload", task.Payload()).
        Str("queue", info.Queue).Int("max_retry", info.MaxRetry).Msg("enqueued task")
    return nil
}

func (processor *RedisTaskProcessor) ProcessTaskSendVerifyEmail(ctx context.Context, task
*asynq.Task) error {
    var payload PayloadSendVerifyEmail
    if err := json.Unmarshal(task.Payload(), &payload); err != nil {
        return fmt.Errorf("failed to unmarshal payload: %w", asynq.SkipRetry)
    }

    user, err := processor.store.GetUser(ctx, payload.Username)
    if err != nil {
        return fmt.Errorf("failed to get user: %w", err)
    }

    verifyEmail, err := processor.store.CreateVerifyEmail(ctx, db.CreateVerifyEmailParams{
        Username:   user.Username,
        Email:      user.Email,
        SecretCode: util.RandomString(32),
    })
    if err != nil {
        return fmt.Errorf("failed to create verify email: %w", err)
    }

    subject := "Welcome to Simple Bank"
    // TODO: replace this URL with an environment variable that points to a front-end page
    verifyUrl :=
fmt.Sprintf("http://localhost:8080/v1/verify_email?email_id=%d&secret_code=%s",
        verifyEmail.ID, verifyEmail.SecretCode)
    content := fmt.Sprintf(`Hello %s,<br/>
    Thank you for registering with us!<br/>
    Please <a href="%s">click here</a> to verify your email address.<br/>
    `, user.FullName, verifyUrl)
    to := []string{user.Email}

    err = processor.mailer.SendEmail(subject, content, to, nil, nil, nil)
    if err != nil {
        return fmt.Errorf("failed to send verify email: %w", err)
    }

    log.Info().Str("type", task.Type()).Bytes("payload", task.Payload()).
        Str("email", user.Email).Msg("processed task")
    return nil
}
```